RESEARCH LEARNING HANDBOOK

AI 数据知识手册

从原始语料到预训练、中期训练、后训练与 RL 环境

覆盖范围

表示与获取	Token 化与计量；网页、PDF、代码、书籍与多模态原始数据
筛选与去重	启发式过滤、质量分类器、LLM 打分、生成式精炼；去重、去污染与审计
预训练组织	配比、阶段与规模律；合成预训练数据
中期训练	推理轨迹、仓库级代码、Agentic 数据与长上下文
后训练	SFT 与推理蒸馏；偏好、奖励模型与评判数据
RL 环境	RL 任务、验证器、环境与轨迹数据
多模态	多模态理解、视觉生成、语音、具身与世界模型数据
系统与方法	数据系统与工程；数据实验方法与评价

Jason

修订日期：2026 年 10 月 17 日

目录

阅读导引与符号约定 **6**

第一章 数据表示、Token 化与计量 **8**
1.1 子词分词的机制与预算 . . . 8
1.2 大词表与多语言覆盖 . . . 10
1.3 跨空白 token 与无 tokenizer 路线 . . . 11
1.4 特殊 token 与上下文构造 . . . 11
1.5 多模态 token 预算 . . . 13
1.6 数据量的统一口径 . . . 13

第二章 原始数据获取与内容抽取 **16**
2.1 网页语料：抓取、快照与漏斗 . . . 16
2.2 HTML 正文抽取 . . . 17
2.3 PDF 与文档解析 . . . 18
2.4 代码与交互数据 . . . 20
2.5 书籍、论文与垂直领域 . . . 20
2.6 多模态原始数据 . . . 21
2.7 元信息与溯源 . . . 22

第三章 过滤、打分与生成式精炼 **24**
3.1 数据处理深度的分层视图 . . . 24
3.2 启发式过滤 . . . 25
3.3 质量分类器 . . . 26
3.4 LLM 标注与蒸馏打分 . . . 27
3.5 分桶、集成与按质量上采样 . . . 28
3.6 反模板与反 AI 文本过滤 . . . 29
3.7 生成式精炼（L4） . . . 30
3.8 目标导向的数据选择 . . . 31
3.9 多模态质量过滤 . . . 31

第四章 去重、去污染与数据审计 **33**
4.1 三级去重流水线 . . . 33
4.2 MinHash 与 LSH 的参数 . . . 34
4.3 语义去重 . . . 35
4.4 有意重复与无意重复 . . . 36
4.5 去污染 . . . 37
4.6 数据审计 . . . 38

第五章 预训练配比、阶段与规模律 **40**

5.1 配比问题的形式化 . . . 40
5.2 多阶段预训练 . . . 41
5.3 各家的配比与取舍 . . . 42
5.4 代理模型群与回归配比 . . . 43
5.5 配比在开发过程中的复用 . . . 44
5.6 多模态联合预训练的配比 . . . 44
5.7 计算最优与过训练 . . . 45
5.8 数据受限：重复与有效数据量 . . . 45
5.9 领域饱和与规模上的排名反转 . . . 46

第六章 合成预训练数据 48
6.1 统一视图 . . . 48
6.2 网页改写 . . . 49
6.3 改写的组织方式：megadocs . . . 50
6.4 教科书与书级组织 . . . 50
6.5 题目格式、事实数据与小语料扩充 . . . 51
6.6 SFT 风格数据进入预训练 . . . 52
6.7 合成比例与规模律 . . . 52
6.8 两条路线与风险诊断 . . . 53

第七章 中期训练、仓库级代码与长上下文数据 56
7.1 中期训练的定位 . . . 56
7.2 中期训练的数据构成 . . . 57
7.3 推理轨迹与元推理数据 . . . 58
7.4 仓库级代码数据 . . . 59
7.5 Agentic 中期训练 . . . 59
7.6 自然长文档 . . . 60
7.7 合成长上下文数据 . . . 61
7.8 长度课程与短上下文保持 . . . 62

第八章 SFT 与推理蒸馏数据 64
8.1 SFT 数据的规模与构成 . . . 64
8.2 Query 侧：构造与过滤 . . . 64
8.3 Response 侧：教师与拒绝采样 . . . 65
8.4 推理数据配方 . . . 66
8.5 推理预算与思考模式 . . . 67
8.6 Agent 轨迹 SFT . . . 68
8.7 多语言、长上下文与格式数据 . . . 69
8.8 打包与训练细节 . . . 70

第九章 偏好、奖励模型与评判数据 72
9.1 偏好对与 Bradley–Terry 模型 . . . 72
9.2 Delta Learning 与规模化偏好数据 . . . 73
9.3 生成式奖励模型 . . . 74

9.4 Rubric 数据 . . . 75
9.5 规则化奖励与多模态、生成模型的奖励数据 . . . 76
9.6 Reward hacking 的诊断 . . . 77

第十章 RL 任务、环境与轨迹数据 80
10.1 RL 数据 = prompt + 验证器 + 环境 . . . 80
10.2 Prompt 池的筛选 . . . 81
10.3 数学与推理 . . . 83
10.4 竞赛代码与 kernel 优化 . . . 84
10.5 软件工程环境 . . . 84
10.6 终端环境 . . . 85
10.7 工具调用与工作流环境的合成 . . . 86
10.8 搜索与深度研究任务 . . . 88
10.9 GUI 与办公环境 . . . 88
10.10 多智能体与并行 . . . 89
10.11 从已有轨迹中挖掘 RL 数据 . . . 90
10.12 专家 RL 与 on-policy 蒸馏的数据流 . . . 90

第十一章 多模态理解数据 94
11.1 各阶段的数据与配比 . . . 94
11.2 图像描述与重写 caption . . . 96
11.3 交错图文与书籍 . . . 96
11.4 OCR 与文档解析 . . . 97
11.5 定位、计数与空间 . . . 98
11.6 STEM 与多模态推理 . . . 99
11.7 视频理解数据 . . . 99
11.8 GUI 与多模态 agent . . . 100

第十二章 视觉生成数据：图像、视频与音视频 102
12.1 图像数据基础设施 . . . 102
12.2 图像 caption 体系 . . . 103
12.3 编辑数据 . . . 103
12.4 视频数据漏斗 . . . 104
12.5 视频 caption . . . 105
12.6 分辨率与训练阶段 . . . 106
12.7 音视频联合数据 . . . 107
12.8 SFT 与 RL 数据 . . . 108
12.9 生成数据飞轮 . . . 108

第十三章 语音、具身与世界模型数据 110
13.1 语音预训练数据 . . . 110
13.2 语音生成与对话数据 . . . 111
13.3 机器人数据金字塔 . . . 112
13.4 人类视频与伪动作 . . . 113

13.5 仿真、神经轨迹与真机经验 115
13.6 交互世界模型数据 116

第十四章 数据系统与工程 119
14.1 存储格式与分片 119
14.2 文本处理框架 120
14.3 GPU 加速的数据处理 121
14.4 大规模视频处理与 caption 推理 121
14.5 合成数据生成系统 123
14.6 RL rollout 与训练的异步 123
14.7 环境服务 125
14.8 训练时的数据加载 126
14.9 多模态数据加载 127
14.10 打包与 mask 127
14.11 版本、血缘与数据卡 128
14.12 质量监控与数据飞轮 129

第十五章 数据实验方法与评价 131
15.1 代理模型实验 131
15.2 评测选择与噪声 132
15.3 消融设计 134
15.4 数据效率的度量 135
15.5 后训练数据的实验 136
15.6 数据归因 137

附录 A 术语与接口速查 139

一手资料与延伸阅读 141

阅读导引与符号约定

全书沿一条数据流展开：原始数据 → 可训练 token → 预训练分布 → 中期训练 → SFT 样本 → 偏好与奖励 → RL 环境 → 线上回流。第一至四章讨论数据怎样计量、从哪里来、怎样筛选和去重；第五至七章把筛好的数据组织成预训练与中期训练；第八至十章进入后训练，依次是 SFT、偏好与奖励、RL 任务与环境；第十一至十三章转向多模态理解、视觉生成、语音与具身数据；最后两章讨论支撑这一切的数据系统，以及判断一项数据工作是否真的有效的实验方法。

每章正文按“机制 → 公式 → 数值例子 → 产业实例”展开，章末练习提供推导、诊断与实验设计。

统一符号

符号	含义
b，D_b，$\lvert D_b\rvert$	数据桶（来源、主题或质量档），该桶的数据集，以及它的唯一 token 数
π_b，ρ_b	桶 b 的采样比例（配比）与重复次数
T，T_b	训练的总 token 预算，以及分给桶 b 的训练 token
N，D，C	参数量（MoE 取激活参数）、训练 token 数、训练计算量，$C \approx 6ND$
V，d	词表大小与隐藏维度
U，R，D'	唯一 token 数、额外的重复次数、等效的唯一数据量（5.8 节）
$q(x)$，s	文档的质量分；相似度或质量分位，按节定义
x，y，τ	prompt、回答与轨迹；τ 在 3.1 节表示阈值、在 5.1 节表示温度，按节定义
π_θ，π_S，π_T	策略；蒸馏中的学生与教师
G，p	每道题的 rollout 组大小，以及单次通过率
$\mathcal{V}$，$\mathcal{E}$	验证器与环境

文中的数字分两类。标明“教学假设”的算例只用来演示推导，数字是构造的；来自论文与技术报告的数字都附有出处，均为报告值，资料截至 2026 年 10 月 17 日。不同报告之间的比较口径，在正文中逐处注明。各机制图都是示意：实线箭头表示数据或处理的前向流动，虚线表示反馈、回流或附加条件。

数据杠杆的定位

路线	做什么	主要章节
A 选择	同一批原料，选得更好：过滤、打分、去重	3、4
B 生成	借模型补数据：改写、合成、蒸馏	6、8
C 交互	on-policy 数据、验证器与环境	9、10

路线	做什么	主要章节
D 组织	配比、阶段、课程、打包	5、7
E 系统	吞吐、成本、可复现	14

五条路线可以在同一次训练中组合，收益在“质量–有效 token –费用”三个轴上评价。

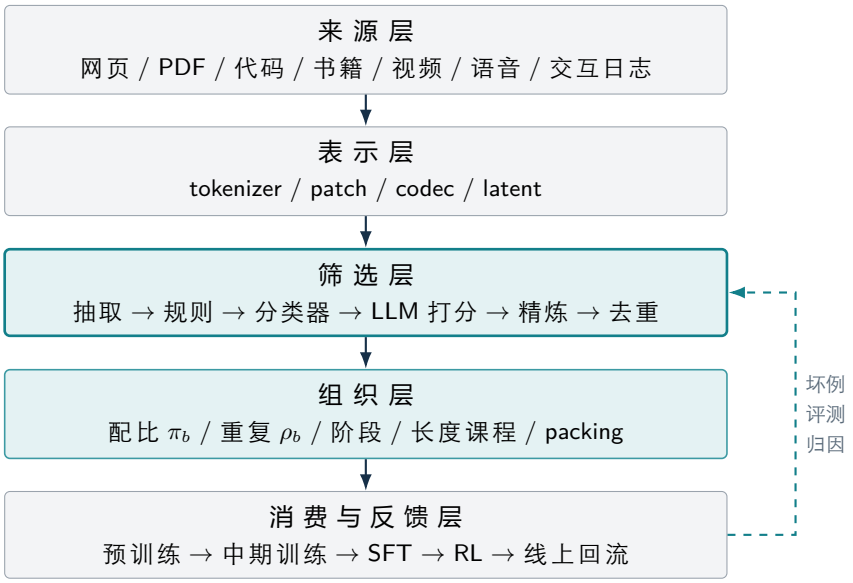


图 1　五层设计。读图：自上而下是数据从来源到被模型消费的五层，虚线表示坏例、评测与归因把问题送回筛选层。上表的五条路线分别作用在不同的层：选择与生成作用在筛选层，组织作用在组织层，交互作用在消费与反馈层，系统贯穿全部五层。

第一章　数据表示、Token 化与计量

数据进入模型之前，要先变成离散 token 或连续向量，而切分方式决定了同一份数据“有多少”。本章先讨论子词分词的机制与词表规模，再看跨空白 token、特殊 token 与多模态的 token 折算，最后统一“数据量”的几种口径。原始数据从哪里来、怎样抽取，留到第二章。

1.1　子词分词的机制与预算

tokenizer 的合并规则决定压缩率，词表大小决定参数量与 softmax 的成本。

BPE：按频次贪心合并

BPE 从 256 个字节出发，反复统计语料中相邻单元对的频次，把最高频的一对合并成新 token，直到词表达到目标大小 V [1]。编码时按合并的先后顺序（rank）依次应用规则。

教学语料取四个词及其频次：`low`×5、`lower`×2、`newest`×6、`widest`×3。初始频次最高的相邻对是 e s 与 s t（都是 9 次），其次是 w e（8 次），l o 与 o w（各 7 次）。依次合并四次：

步	合并	频次	合并后的切分
1	e+s → es	9	n e w es t；w i d es t
2	es+t → est	9	n e w est；w i d est
3	l+o → lo	7	lo w；lo w e r
4	lo+w → low	7	low；low e r

四次合并后，语料从 79 个字符单元压缩到 47 个 token，压缩比约 1.68。

注意第 1 步之后，“w e”的频次从 8 掉到 2：newest 中的 w 现在挨着 es，而不是 e。合并会改写后续的统计，所以 BPE 是贪心的，结果依赖合并顺序；同一份语料换一种打乱方式训练，词表会有细微差别。

字节级回退与 Unicode 规范化

以字节为基本单元，任何字符串都能编码，不存在 OOV。代价是 UTF-8 中一个汉字占 3 字节，如果词表里没有这个字的合并，它就会被拆成 3 个 token。中文、日文、印地文在英文主导的词表里“变贵”，原因就在这里。

规范化是另一个隐蔽的选择。SentencePiece 默认做 NFKC 规范化，会把全角的“A”变成“A”、把上标“²”变成“2”，这对自然语言是去噪，对代码和数学却会改变语义：`x²` 与 `x2` 是两个不同的东西。tiktoken 系列（GPT-4、Llama 3）不做规范化，原始字节原样进入模型，规范化只放在数据清洗阶段，并按领域区别对待。

Unigram：按概率选择切分

Unigram 为每个 token 赋一个概率 $p(t)$，字符串的切分取乘积最大者：

$$\hat{\mathbf{t}} = \arg\max_{\mathbf{t}\in S(x)} \prod_i p(t_i),$$

其中 $S(x)$ 是 x 所有合法切分的集合，用 Viterbi 求解。训练从一个很大的种子词表出发，交替做两件事：用 EM 估计 $p(t)$，再删掉那些“删掉后语料似然下降最少”的 token，每轮保留约 75%（SentencePiece 默认 `shrinking_factor=0.75`）。

Unigram 有一个 BPE 没有的能力：按 $P(\mathbf{t} \mid x)^\alpha$ 采样多种切分，即子词正则化。BPE 对同一个字符串只给出一种确定的切分。产业上，Llama 1/2 用 SentencePiece BPE、32K 词表，Llama 3 换成 tiktoken BPE、128K 词表，Gemma 系列用 SentencePiece、约 256K 词表。

预分词：合并之前先划边界

BPE 合并之前，先用正则表达式把文本切成“预 token”，合并永远不会跨越预 token 的边界。很多下游行为在这一步就定下来了。

数字的切法是最典型的例子。Llama 1/2 与 Qwen 逐位切分，数位对齐稳定，但序列变长；cl100k 与 Llama 3 用 `\p{N}{1,3}` 把数字从左往右按 1–3 位分组，“1234”被切成“123”和“4”，与千分位并不对齐。研究显示，把数字强制改成从右往左每三位一组之后，GPT-3.5/4 的算术准确率明显提高 [2]。

标点与换行的组合 token 是另一个例子。DeepSeek-V3 的预分词器引入了“标点 + 换行”的组合 token 来提高压缩率，副作用是 few-shot prompt 结尾没有换行时，模型看到的 token 边界与训练时不同；他们的对策是训练时随机拆开一部分这类 token [3]。代码缩进也有同样的问题：连续空格合成一个 token 后，Python 的 4/8 空格缩进很省 token，但训练语料中罕见的缩进宽度没有对应的 token，模型在这类缩进上更容易出错。任何压缩性的合并，都可能让推理时的边界偏离训练分布。

词表大小的参数与计算预算

输入 embedding 与输出 LM head 各有 $V \cdot d$ 个参数（不共享时是两份），每个 token 在 LM head 上的计算约为 $2Vd$ FLOPs。以 $d = 4096$ 为例，128,256 的词表每个矩阵 0.525B 参数，两个矩阵合计 1.05B，约占 8B 模型的 13%；词表扩到 262,144 后合计 2.15B，占比升到 27%。

计算上，262K 词表的 LM head 每个 token 约 2.1 GFLOPs，相当于 8B 稠密模型每 token 前向计算（约 16 GFLOPs）的 13%。显存上更麻烦：8K token 的 fp32 logits 就有 $8192 \times 262144 \times 4\,\text{B} \approx 8.6$ GB，必须分块计算交叉熵，或者使用融合 kernel。

大词表的回报是压缩率：同样的字节用更少的 token，每个字节的计算更少，同样的上下文窗口装得下更多内容。代价是低频 token 训练不足，会出现可以用 embedding 范数等指标检测的“glitch token”[4]，小模型中 embedding 的参数占比也会过高。词表规模律的研究给出的方向是，最优词表随计算预算增长；按其估计，Llama 2-70B 的最优词表至少应有 216K，是实际 32K 的 7 倍 [5]。

评估一个 tokenizer，先看三件事：目标语料上的每 token 字节数，数字与代码的切法，以及词表中训练不足的 token 有多少。

1.2 大词表与多语言覆盖

词表从 128K 走向 250K，增长主要来自非拉丁文字与代码；衡量它的指标，是同一内容在不同语言下要用多少 token。

产业词表的演化

2024 年，Llama 3 把词表从 32K 扩到 128,256：其中 100K 来自 tiktoken，另加 28K 非英语 token，英语的压缩率从每 token 3.17 个字符提高到 3.94 [6]。DeepSeek-V3 与 V4 保持约 128K（129,280）的字节级 BPE，V4 只为上下文构造新增了少量特殊 token [3, 7]。Kimi K2 的词表为 163,840。

2026 年的变化来自多语言与多模态。Qwen3.5 的词表从约 151K 扩到 248,320，语种覆盖从 119 种扩到 201 种，官方给出的结果是大多数语言的编码效率提升了 10–60% [8]；Gemma 3 与 Gemma 4 沿用 Gemini 的约 262K 词表 [9]。

Fertility：同一内容的 token 账单

定义平行语料上的相对 token 比：

$$\text{ratio}_\ell = \frac{\#\text{tokens}(x_\ell)}{\#\text{tokens}(x_{\text{en}})},$$

其中 x_ℓ 与 x_{en} 是同一段内容的语言 ℓ 版本与英文版本。对多种 tokenizer 的测量显示，不同语言之间的这个比值可以相差 15 倍 [10]。

这个比值同时影响训练、推理与上下文三处：同样的 token 预算里，该语言承载的内容更少；推理的费用与延迟按 token 计；同样的窗口里装下的信息也更少。教学假设：一篇 1000 词的英文新闻约 1300 token，若某语言的 ratio 为 3，同样的内容需要 3900 token，在 128K 的窗口里，该语言的“有效上下文”只相当于英语的三分之一。Qwen3.5 报告的 10–60% 编码效率提升，本质上就是把这些语言的 ratio 往下压。

词表扩充与 embedding 初始化

给已有模型扩充新语言，常见流程是先用目标语言的语料训练新的 BPE、合并进旧词表，再初始化新 token 的 embedding，最后继续训练：先只训练 embedding 与 LM head，再全参训练。

初始化有两种做法。取全部旧 embedding 的均值，保证初始输出分布不被扰乱；取该 token 在旧 tokenizer 下的子 token 的 embedding 均值，则能保留语义。扩充之后要检查两件事：旧语言的压缩率没有下降；新 token 在训练中出现得足够多，否则它们会成为新的 glitch token。

tokenizer 与语料的耦合

tokenizer 应在与预训练配比一致的样本上训练。训练 tokenizer 的样本里高频、预训练语料里却罕见的 token，embedding 会训练不足；预训练语料里大量出现、tokenizer 却没有覆盖的领域，会被切得很碎。实践中的做法，是按目标配比对代码、数学、多语言分别加权抽样，再训练 tokenizer。第五章会看到，配比本身也依赖 tokenizer，两者需要一起标定。

1.3 跨空白 token 与无 tokenizer 路线

BPE 的边界由预分词决定。放开空白边界，或者干脆不用 tokenizer，是提高每个单元信息密度的两条路线。

SuperBPE：允许跨词边界的 superword

标准 BPE 中，“by the way”永远至少是 3 个 token，因为空白处有预分词边界。SuperBPE 把训练分成两段：先照常用空白预分词学习子词，到过渡点 t 之后，再去掉空白预分词，学习跨词的 superword [11]。

在 200K 词表、8B 模型、约 330B token 的设置下，SuperBPE 平均每 token 6.63 字节（BPE 为 4.46 字节），同样的文本最多少用 33% 的 token；30 项下游任务平均高 4.0 分、赢下 25 项，MMLU 从 36.5 升到 44.7，推理计算减少 27%。

最耐人寻味的是，两者的 bits-per-byte 几乎相同（0.7465 对 0.7482）。语言模型损失一样，下游能力却差了 4 分。损失不能代替下游评测，第十五章还会回到这一点。

BLT：按熵动态切 patch

Byte Latent Transformer 直接建模字节 [12]。一个小型字节语言模型先计算下一字节的熵 $H(x_i)$，熵超过阈值的位置开启一个新 patch；局部编码器把每个 patch 内的字节编码成向量，交给全局的 latent Transformer；局部解码器再还原出字节。

可预测的长片段（例如常见后缀）会被并进一个长 patch，难预测的位置 patch 变短，计算量按难度分配。在 8B 规模的 FLOP 对照实验中，BLT 与 Llama 3 相当；平均 patch 更大时，推理 FLOPs 最多可以省约 50%。截至本书修订时，主流前沿模型仍使用 BPE 系的 tokenizer。

Bits-per-byte：跨 tokenizer 可比的损失

不同 tokenizer 下，每 token 的损失不能直接比较，应换算成每字节的比特数：

$$\mathrm{BPB} = \frac{\sum_i -\ln p(t_i)}{\ln 2 \cdot \#\mathrm{bytes}} = \frac{\bar{\ell}_{\mathrm{tok}}}{\ln 2} \cdot \frac{\#\mathrm{tokens}}{\#\mathrm{bytes}},$$

其中 $\bar{\ell}_{\mathrm{tok}}$ 是每 token 的平均交叉熵（nats）。

教学假设：tokenizer A 每 token 损失 2.0 nats、每 token 4.0 字节，$\mathrm{BPB} = 2.0/(0.693\times4.0) \approx 0.72$；tokenizer B 每 token 损失 2.3 nats、每 token 4.6 字节，$\mathrm{BPB} = 2.3/(0.693\times4.6) \approx 0.72$。两者其实一样好，直接比较每 token 的损失，会误以为 A 更好。第五章 Olmo 3 的配比回归，正是以 BPB 为目标 [13]。

1.4 特殊 token 与上下文构造

特殊 token 定义了数据的“语法”：谁在说话、思考从哪里开始、工具调用怎样分界、哪些位置计入损失。

对话模板：训练与推理逐字一致

对话数据会被渲染成带特殊 token 的序列，例如 `<|im_start|>assistant ⋯<|im_end|>`。模板上有三条规则必须守住。第一是训推一致：SFT 用的模板与推理时的模板逐字一致，空格、换行、BOS 的差异都会造成分布偏移。第二是注入防护：网页正文中字面出现的 `<|im_end|>` 要按普通文本编码（tiktoken 的 `allowed_special` 机制），不能被解析成控制 token。第三是预训练数据的清洗：转义原始语料中出现的特殊字符串，避免模型在预训练阶段就学会“伪造”控制 token。

2026 年的报告开始把模板当作数据工程的一部分单独说明，例如 Kimi K3 把全部 SFT 数据序列化为一种基于 XTML 的对话模板 [14]。

工具调用格式也是数据

工具调用的格式错误，会直接让 agent 失败。DeepSeek-V4 为此新增了 `|DSML|` 特殊 token，并改用基于 XML 的工具调用格式：字符串参数原样传入，标记为 `string="true"`；结构化参数以 JSON 传入，标记为 `string="false"` [15]。据官方说明，这比把 JSON 塞进字符串更少出现转义失败。

V4 还改变了思考内容的保留规则：对话中有工具调用时，跨用户消息保留之前的推理内容；没有工具时，每轮仍会清空推理内容，以保持上下文简洁 [15]。这类规则决定了多轮 agent 数据应该怎样构造，训练样本中哪些推理被保留、哪些被丢弃，必须与推理时的行为一致。

文档边界与样本级注意力 mask

预训练把多篇文档拼成长度为 L 的序列（packing）。只用因果 mask 时，后一篇文档能看到前一篇，产生跨文档的干扰。样本级（块对角）mask 只允许文档内的注意力，并把每篇文档的位置编号从零开始重新计算。Llama 3 采用这种 mask，并指出它在超长序列的继续预训练中尤其重要 [6]；DeepSeek-V4 同样使用样本级注意力 mask [7]。

教学假设：$L = 8192$，装入长度为 3000、4000、1192 的三篇文档。全因果注意力的配对数约为 $L^2/2 = 33.5\text{M}$，块对角 mask 下为 $(3000^2+4000^2+1192^2)/2 \approx 13.2\text{M}$，只有前者的 39%。用变长 FlashAttention（传入 `cu_seqlens`）实现时，省下的配对就是真正省下的计算。

Loss mask：哪些 token 参与梯度

$$\mathcal{L} = \frac{\sum_i m_i \, (-\log p_\theta(t_i \mid t_{<i}))}{\sum_i m_i}, \qquad m_i \in \{0, 1\},$$

其中 $m_i = 1$ 表示第 i 个 token 计入损失。SFT 中，系统提示与用户输入的 $m_i = 0$；agent 轨迹中，工具返回的内容也不计损失。GLM-5 把轨迹中的错误片段保留在上下文里，但同样不计损失 [16]；Nemotron 3 Ultra 把推理轨迹截断到不同的预算时，截断处人为加上的 `</think>` 不计损失，避免模型学会“主动提前结束思考” [17]。

归一化方式也要写进配置。按 token 平均，长样本的权重更大；先在样本内平均、再跨样本平均，每个样本的权重相同。两者在长短样本混合时会训练出不同的模型（第八章）。

1.5 多模态 token 预算

图像、视频、音频都要先折算成 token，才能和文本放进同一个预算里。

图像：patch、合并与动态分辨率

视觉编码器切出 $p \times p$ 的 patch，再把 $m \times m$ 个相邻 patch 合并成一个 LLM token：

$$N_{\text{img}} = \left\lceil \frac{H}{pm} \right\rceil \cdot \left\lceil \frac{W}{pm} \right\rceil .$$

Qwen2.5-VL 取 p=14、m=2，每个 token 覆盖 28×28 像素，一张 1024×1024 的图按上式为 $37 \times 37 = 1369$ 个 token；Qwen3-VL 把 p 改为 16，每个 token 覆盖 32×32 像素，同一张图正好 1024 个 token。

动态分辨率保留原始宽高比，用 `min_pixels` 与 `max_pixels` 控制 token 的上下限；另一条路线是切成固定大小的 tile，每个 tile 给固定数量的 token。两者的差别在两头：小图会不会被放大、浪费 token，大图会不会因为缩小而丢掉细节。

视频：帧率 × 每帧 token ÷ 时间合并

$$N_{\text{vid}} = \frac{T \cdot f}{k} \cdot N_{\text{frame}},$$

其中 T 为时长，f 为采样帧率，k 为时间维的合并倍数，N_{frame} 为每帧的 token 数。

教学假设：60 秒视频，2 fps 采样，448×448 分辨率，每 28×28 像素一个 token，即每帧 256 个 token。$k = 2$ 时共 $120/2 \times 256 = 15{,}360$ 个 token；Kimi K2.5 每 4 帧一组做时间池化（$k = 4$）[18]，降到 7,680 个，同样的窗口能装入两倍时长的视频。

音频：codec 帧率 × 码本层数

神经 codec 以帧率 r 输出，每帧有 K 层 RVQ 码本。把码本层展平成一条序列时，每秒有 rK 个 token：以 12.5 Hz、8 层码本为例（Mimi），每秒 100 个 token，1 小时语音约 36 万个 token。改用并行输出头或延迟模式（delay pattern），每秒只有 r 个位置，即 12.5 个，1 小时约 4.5 万个位置。语音模型能处理多长的上下文，很大程度上取决于选择哪种表示。

生成侧：latent token

视频生成模型在 VAE latent 上计 token。以 Wan 类 VAE（时间压缩 4 倍、空间压缩 8 倍）加 1×2×2 patch 为例，生成 5 秒、16 fps 的 720p 视频：81 帧折算成 $1 + 80/4 = 21$ 个 latent 帧，每个 latent 帧有 45×80 个 token，合计 75,600 个 token。视频生成数据的“规模”，最终要折算到这个口径上，才能和计算预算对齐（12.4 节）。

1.6 数据量的统一口径

“我们有 X 的数据”这句话，至少有四种单位和三种口径，混用会让配比和预算全部失真。

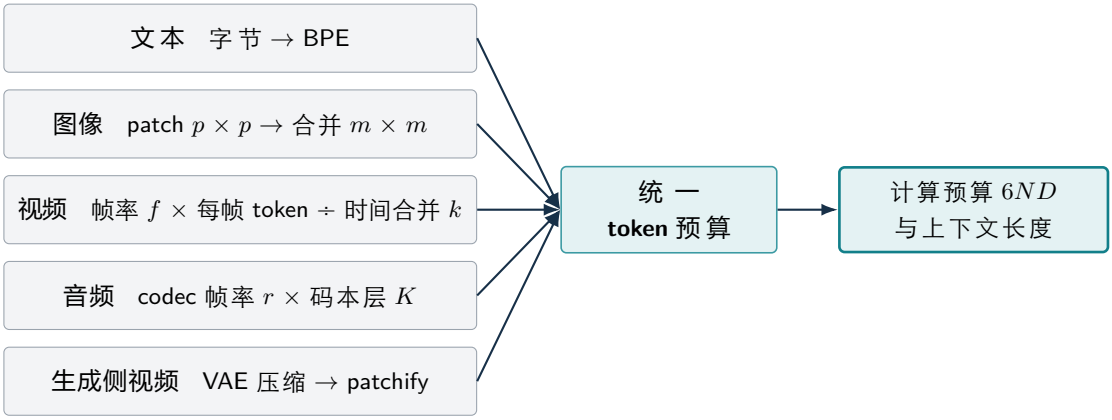


图 1.1 各模态进入模型前的 token 折算。读图：先看每条输入链路中决定 token 数的那几个参数，再到右侧统一成计算预算。理解模型与生成模型的 token 口径不同，前者按编码器 patch 计，后者按 VAE latent 计。[18, 19]

四种单位

字节适合存储，也适合跨 tokenizer 比较，英文网页约每 token 4 字节，Llama 3 为每 token 3.94 个字符 [6]。token 用于计算预算与配比，但会随 tokenizer 改变。文档数用于去重与过滤的统计：Olmo 3 的 Dolma 3 数据池有 9.31T token、99.7 亿篇文档，平均每篇约 930 个 token [13]。视频与语音以小时计，要按 1.5 节的方式换算成 token。

一个完整的预算例子（教学假设）：要得到 1T 英文网页 token，按每篇约 930 token 计，需要约 10.8 亿篇过滤后的网页。若启发式过滤保留 15%、去重再保留 25%（Olmo 3 的量级，见第三、四章），原始文档需要约 $10.8/(0.15 \times 0.25) \approx 288$ 亿篇。同样是 1T token，如果来自 PDF，每篇文档长一个数量级，需要的文档少得多，但每篇的抽取成本也高得多（第二章）。

原始、唯一与训练 token

同一个数据集常常同时报出三个数：去重之前的原始 token、全局去重之后的唯一 token，以及计入重复次数后的训练 token $\sum_b \rho_b |D_b|$，其中 ρ_b 是数据源 b 的重复次数。

三者的差距可以很大。按 Nemotron-CC 论文的统计，DCLM-Baseline 的 3.8T token 中，全局唯一的只有 1.0T；FineWeb-Edu 的 1.3T 中，全局唯一的只有 0.2T [20]。原因是很多数据集只在每个快照内部去重，跨快照的重复依然存在。

2026 年的 MAI-Thinking-1 报告把三种口径并列写出，是一个规范的范例 [21]。它的全部数据唯一 29.2T、训练消耗 30.0T；其中代码唯一 7.4T、训练 16.4T，平均重复 2.22 次；数学唯一只有 0.3T、训练 1.6T，平均重复 5.28 次；网页文本反过来，唯一 8.1T 只训练了 4.5T，平均 0.55 次；多语言唯一 8.1T，只用了 0.5T。“30T 预训练”背后，是对每个来源唯一量与重复次数的具体取舍（5.3 节）。

计算预算与数据墙

稠密 Transformer 的训练计算近似为 $C \approx 6ND$，其中 N 为参数量（MoE 取激活参数），D 为训练 token 数。DeepSeek-V3 激活 37B 参数、训练 14.8T token，$C \approx 3.3 \times 10^{24}$ FLOPs。

到 2026 年，前沿模型的预训练量已经到了 20–36T token：Qwen3 为 36T，DeepSeek-V4 为 32T（Flash）与 33T（Pro），MAI-Thinking-1 为 30T，GLM-5 为 28.5T，Nemotron 3 Ultra 为 20T。而对公开网页文本的估计约为 300T 有效 token，按当前趋势会在 2026–2032 年之间被用尽 [22]。

更新的变化是，可用的人类文本不只是变少，还在被稀释：2026 年 8 月，经过质量过滤的 Common Crawl token 中，已有约 31% 被检测为 AI 生成（3.6 节）[23]。后续几章的共同背景由此而来：提高唯一数据的质量（第三、四章），用改写和上采样替代简单重复（第五、六章），以及引入 PDF、多模态与交互数据（第二、十、十三章）。

本章思考与进阶练习

1.1 词表翻倍

题目 一个 8B 稠密模型（d=4096，embedding 与 LM head 不共享），词表从 128K 扩到 256K 后，同一语料的 token 数减少 8%。参数量和每字节的前向计算分别怎样变化?

解题思路 参数增加 $2 \times (262{,}144 - 128{,}256) \times 4096 \approx 1.1\text{B}$。计算上，非 embedding 参数约 6.95B，主干每 token 约 13.9 GFLOPs，LM head 从 1.05 升到 2.15 GFLOPs，每 token 计算从约 14.95 升到 16.05 GFLOPs（+7.4%）；乘以 0.92 之后，每字节计算降低约 1%，基本持平，盈亏点在 token 数减少约 6.9% 处。真正的代价是多出的参数与优化器状态，以及翻倍的 logits 显存。是否值得，要看多语言压缩率与下游指标。

1.2 BPB 换算

题目 模型 A 每 token 损失 1.9 nats，平均每 token 3.8 字节；模型 B 每 token 损失 2.2 nats，平均每 token 4.5 字节。哪个模型在字节层面更好?

解题思路 A 的 $\text{BPB} = 1.9/(0.693 \times 3.8) \approx 0.721$，B 的 $\text{BPB} = 2.2/(0.693 \times 4.5) \approx 0.705$。B 更好，尽管它每 token 的损失更高。

1.3 进阶：换 tokenizer 后，原有配比为什么会失效

题目 原配比按 token 计，中文 20%、英文 80%。新 tokenizer 对中文的压缩提高 30%，对英文不变。如果仍按“20% 中文 token”采样，模型实际看到的中文内容占比怎样变化?

解题思路 设旧 tokenizer 下中英文每 token 的字节数相同，新 tokenizer 下每个中文 token 承载的内容是原来的 1/0.7 倍。仍按 20% 的 token 采样时，中文的内容份额变为 $\frac{0.2/0.7}{0.2/0.7+0.8} \approx 26.3\%$，相对原意图上调了约 31%；如果中文的唯一数据量固定，重复次数也会增加到原来的约 1.43 倍。所以配比应在字节或文档层面定义，或者在更换 tokenizer 后按压缩率重新标定 π_b。

第二章　原始数据获取与内容抽取

训练数据的上限，在抽取那一步就已经定了。公式被抽坏，后面的分类器只会把文档判成低质量；表格被压平，后面的精炼也补不回结构。本章按来源依次讨论网页、PDF、代码、书籍与垂直领域、多模态与交互数据的获取和抽取，最后讨论贯穿全流程的元信息。下一章讨论怎样从抽取结果中筛出高价值的部分。

2.1　网页语料：抓取、快照与漏斗

网页是预训练最大的来源。决定它价值的，是抓取到的范围与抽取后的质量，而不是名义上的“期数”或页数。

Common Crawl 的三种文件

Common Crawl（CC）大约每一两个月发布一期快照，每期有数十亿网页、数百 TiB 未压缩内容，提供三种文件。WARC 保存原始的 HTTP 响应，包含完整 HTML，主流做法是从 WARC 自行抽取正文。WAT 保存元数据与链接图，可以用来做链接分析，或者采集图像和视频的 URL（2.6 节）。WET 是 CC 官方抽取的纯文本，很便宜，但混有大量导航栏、菜单和页脚。

各期快照之间有大量重叠，“用了多少期”和“有多少唯一数据”是两回事。FineWeb 用了 96 期 [24]，Nemotron-CC-Math 从 98 期中取回 HTML [25]，Olmo 3 用了截至 2024 年 12 月的 104 期 [13]。第四章会看到，全局去重之后，老快照里能留下来的内容少得多。

自建爬虫：覆盖、新鲜度与抓取规范

CC 偏向链接密集的站点，抓取深度有限，头部模型通常同时运行自建爬虫。调度时，抓取优先级可以写成：

$$\text{priority}(u) \propto \hat{q}(u) \cdot \text{freshness}(u) \cdot \text{quota}(\text{domain}),$$

其中 $\hat{q}(u)$ 是 URL 级别的质量预估，来自域名的历史分数与链接结构；$\text{freshness}(u)$ 反映内容多久没更新；quota 是每个域名的配额，防止单个站点主导数据。

抓取要遵守 robots.txt 与 crawl-delay，不绕过付费墙；Olmo 3 的 PDF 爬虫 AI2Bot 公开声明了这一点 [13]。重度依赖 JavaScript 的页面要用无头浏览器渲染，单页成本比直接取 HTML 高一到两个数量级，通常只对高价值域名开启。

一个 2026 年的完整漏斗

MAI-Thinking-1 公开了从抓取到可训练数据的各级规模 [21]：

阶段	规模
抓取	约 1.2T 个网页
过滤后	7940 亿个
精确去重后	4230 亿个
模糊去重后	约 1900 亿个

这些网页最终支撑了约 30T token 的预训练。从抓取到可用，保留率约 16%，其中模糊去重一步就去掉了一半以上。这条漏斗也说明了“数据墙”的具体含义：即便抓取了万亿级网页，真正独特、可用的部分也只有它的零头。



图 2.1　网页语料的漏斗。读图：条形长度与网页数成正比。从约 1.2T 个抓取网页到约 1900 亿个去重后网页，保留率约 16%，其中模糊去重一步去掉了一半以上。数据来自 MAI-Thinking-1。[21]

时间截止与新鲜度

模型的知识截止，由最新数据的时间戳决定，而不同来源的截止时间往往不同。以 Olmo 3 为例，网页截至 2024 年 12 月，arXiv 部分沿用 Proof-Pile-2、截至 2023 年 4 月，Wikipedia 截至 2023 年 3 月。代码同样如此：Nemotron 3 Ultra 的代码刷新新增了 173B token，截至 2025 年 9 月 30 日 [17]。

刷新数据时，要同步更新去污染集，因为新快照更可能包含新发布的评测题。

2.2　HTML 正文抽取

抽取器决定哪些字符进入训练集，是整个管线中最早、也最难事后弥补的一步。

抽取器本身会改变下游分数

常用的抽取器有三类。trafilatura 基于 DOM 启发式与正文密度，FineWeb 的消融显示，从 WARC 用 trafilatura 重新抽取，明显优于直接用 WET [24]。resiliparse 的效果接近 trafilatura，速度快得多，DCLM 与 Olmo 3 都选用了它 [13, 26]。jusText 按段落分类，Nemotron-CC 比较后选用了它，理由是同样的 URL 下，它保留下来的高质量 token 更多 [20]。

抽取器与下游分类器是耦合的。质量分类器是在某个抽取器的输出上训练的，换了抽取器，就等于换了分类器的输入分布，阈值需要重新标定。只比较“分类器打分后的保留率”，会把两种效应混在一起。

公式与代码：Nemotron-CC-Math 的做法

通用抽取器为“去模板”而设计，常常删除或打乱 MathJax 渲染的公式、丢掉行内 LaTeX、压平代码缩进。OpenWebMath、FineMath、MegaMath 都继承了这些问题。

Nemotron-CC-Math 换了一条路线 [25]：

1. 从这些数据集收集数学相关的 URL，到 98 期 CC 中取回 2.3 亿个页面的原始 HTML；
2. 不解析 DOM，而是用文本浏览器 lynx 按布局规则渲染页面，输出接近“人看到的页面”，公式与缩进得以保留；
3. 用 Phi-4（14B）去掉导航与模板，把数学统一成 LaTeX，修正错字；
4. 用 FineMath 分类器打分，用 MinHash 去重（Jaccard 约 0.8）；
5. 用 embedding 对评测集去污染。

选择 Phi-4，是因为在与 DeepSeek-V3、Qwen2.5-32B/72B 的比较中，它效果相当、体积更小。得到的 3+ 子集有 133B token，4+ 子集有 52B token，后者是 FineMath-4+ 的 5.5 倍。在 8B 模型上做 100B token 的退火，MATH 达到 40.6，FineMath-4+ 为 35.8；在 300B token 设置下，MBPP+ 比 MegaMath-Web 高 4.6。公式与代码的完整保存，同时带来了代码能力的提升。

MAI-Thinking-1 走的是相近的思路：把 MathML 与 LaTeX 统一规范化成 Markdown；抽取后的清洗允许 LLM 参与，但只允许它保留或删除文本片段，不允许写入任何新内容（3.7 节会讨论这条约束）[21]。

抽取失败一旦发生，下游任何过滤都无法补救；领域数据需要领域专用的抽取路线。

结构的保留

把 HTML 抽成 Markdown 比抽成纯文本好，因为标题、列表、表格保留了文档的层级，模型能从中学到结构化输出；Qwen3-VL 的文档解析数据甚至定义了带元素坐标的 HTML 格式（11.4 节）。常见的结构问题有三类：多栏版面被串行化后阅读顺序错乱，嵌套表格被展平，链接的锚文字和正文混在一起。

怎样做抽取器消融

抽取器消融要固定同一批 URL 和后续的过滤流程，只改变抽取器。每组训练 1B 级模型约 30–50B token，用 BPB 与早期信号强的任务比较（第十五章）。同时要记录每种抽取器的产出 token 量与吞吐：某个抽取器分数略高、但 token 少了 20%，在数据受限的场景下未必划算。

2.3 PDF 与文档解析

网页之外，论文、技术报告、教材、手册与政府文件大多以 PDF 形式存在。它们篇幅长、信息密度高，是网页覆盖不到的知识。

PDF 的价值

FinePDFs 从 4.75 亿个 PDF 构建了约 3T token、覆盖 1733 种语言的语料 [27]。在 1.67B 模型上，单独使用 FinePDFs 的效果已经接近当时最好的 HTML 数据集，与 HTML 数据混合后效果明显更

好：PDF 与网页提供的知识是互补的。

产业配比印证了这一点。Olmo 3 预训练中 PDF 占 13.6%，即 805B token [13]；Nemotron 3 Ultra 在第二阶段把高质量 PDF 提到 10.7% [17]；Qwen3 用 Qwen2.5-VL 从大量 PDF 中抽取文本、再用 Qwen2.5 精炼，获得了万亿级的 token [28]。

文本层抽取与视觉解析

PDF 解析有两条路线。文本层抽取（pdftotext、PyMuPDF、Docling）只在 CPU 上运行，很便宜，适合原生的数字 PDF；视觉解析（olmOCR、RolmOCR、MinerU、Qwen-VL）需要 GPU，成本高，但能处理扫描件、复杂版面与公式。FinePDFs 用 Docling 处理有文本层的文档，用 RolmOCR 处理需要 OCR 的文档 [27]，路由的信号是文本层的字符数与页面面积之比、乱码率，以及页面是否以整页图像为主。

视觉解析模型在 2025–2026 年演化出三种结构。olmOCR 是端到端 VLM 加文本锚点：微调 Qwen2-VL-7B，输入页面图像，同时把 PDF 原有的文本与坐标作为“锚点”一起提供，以减少幻觉，官方给出的成本约为每百万页 200 美元以内 [29]。MinerU 2.5 是两段解耦：用 1.2B 的 VLM 先在缩略图上分析版面，再对原分辨率的区域裁剪逐块识别，兼顾全局版面与局部细节 [30]。Qwen3-VL 则是版面模型加大模型逐区域识别：先由版面模型预测阅读顺序和区域，再由 Qwen2.5-VL-72B 识别每个区域 [19]。

Olmo 3 的 PDF 漏斗

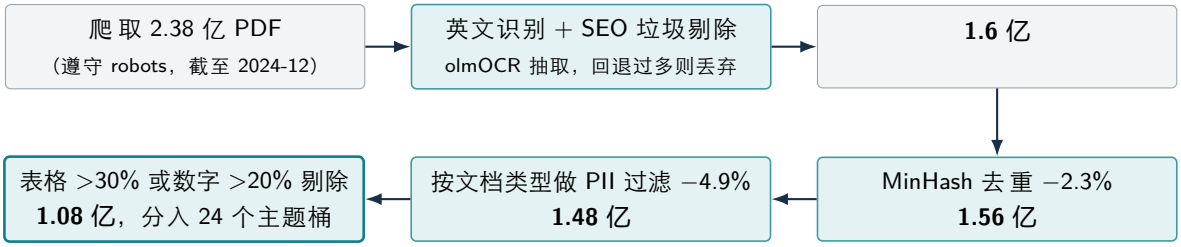


图 2.2　Olmo 3 的 PDF 数据漏斗。读图：从左到右看每一步去掉了什么。比例最大的一步发生在抽取阶段：olmOCR 失败时回退到 pdftotext，回退页超过 1/250 的文档整篇剔除，因为回退意味着版面复杂、文本层不可靠。[13]

PDF 还有一些网页没有的清洗项。每页重复出现的页眉、页脚、页码，靠跨页统计重复行去除；行末用连字符断开的单词需要合并；双栏版面的阅读顺序依赖版面模型，可以用段落连贯性来检查；文本层中退化成乱码的公式，用非 ASCII 符号的密度检测；参考文献在篇幅中占比过大的文档，需要截断或降权。

转换成本估算

$$\text{cost} = \#\text{pages} \cdot \frac{\bar{n}_{\text{out}}}{\text{throughput}} \cdot \text{price}_{\text{GPU·h}},$$

其中 $\bar{n}_{\text{out}}$ 是每页的平均输出 token 数，throughput 是单卡的聚合生成吞吐。

教学假设：每页输出 800 token，单卡聚合吞吐 3000 token/s，单价 2 美元/GPU 时。每卡每小时约处理 13,500 页，每百万页约 148 美元，1 亿页约 1.5 万美元，与 olmOCR 公布的量级一致。成本主要由输出 token 决定，表格与公式密集的页面贵得多；先做文本层路由，能把 VLM 的用量压到总页数的一小部分。

2.4 代码与交互数据

网页和 PDF 记录的是“写下来的知识”，代码仓库和开发过程记录的是“怎样做事”，后者正是 agent 能力的来源。

代码快照与刷新

主要来源是 GitHub 快照与 Software Heritage 归档，The Stack v2 基于 Software Heritage 构建，覆盖 600 多种语言 [31]。刷新的收益可以量化：Nemotron 3 Ultra 新增了 173B token 的新 GitHub 代码 [17]；GLM-5 刷新快照、纳入更多含代码的网页之后，模糊去重后的唯一 token 增加了 28% [16]。GLM-5 还修复了 Software Heritage 文件的元数据对齐问题，并为 Scala、Swift、Lua 等低资源语言单独训练语言分类器，原因是扩展名不可靠：`.h` 可能是 C，也可能是 C++。

从规则过滤到模型打分

代码的低质量来源很具体：压缩或混淆过的 JS（最大行长超过 1000）、lock 文件与数据文件、头部写着“auto-generated”的生成代码，以及超大文件。这些都可以用规则过滤，但“可读性好不好”这类判断，规则无能为力。

Seed-Coder 用模型代替规则 [32]。它先用 DeepSeek-V2-Chat 给 22.2 万个代码文件打 0–10 分，维度是可读性、模块化、清晰度与可复用性；再蒸馏到一个 1.3B 的回归模型，用它剔除 GitHub 文件中分数最低的约 10%，保留约 1T 唯一 token。选择回归而不是二分类，是为了得到更细粒度的分数，便于按分数上采样，而不只是截断。Stack-Edu 则在 The Stack v2 上训练教育价值分类器，筛出 125B token，被 SmolLM2 与 Olmo 3 采用 [13, 33]。

从文件到仓库

文件级样本缺少跨文件的依赖关系。仓库级的构造方法是：解析 import 关系，建立依赖图；按拓扑序拼接文件，被依赖的文件放在前面；以仓库为单位做近似去重，避免 fork 带来的大量重复 [34]。仓库级数据再加上 FIM 变换、commit diff、issue 与 PR，就构成了第七章的中期训练数据，FIM 的格式与参数在 7.4 节讨论。

开发过程本身成为数据

2026 年出现了一类新来源：真实开发过程的轨迹。据第三方整理（非官方），xAI 在 Grok 4.5 的中期训练中使用了匿名化的 Cursor 开发者会话轨迹，即工程师在真实代码库中工作的记录 [35]。

这类数据记录了读代码、改代码、运行测试、根据报错修正的完整过程，比静态的仓库快照更接近 agent 实际要做的事。它的问题同样明显：授权与匿名化的范围难以界定，会话中混有密钥与私有代码，不同开发者的工作习惯差异巨大。它与第十章“把内部工作流转成 RL 环境”的思路方向一致：数据越来越多地来自“做事的过程”，而不只是“做完的结果”。

2.5 书籍、论文与垂直领域

垂直领域的数据很少能直接用。原文要经过清洗，再转写成模型能调用的形式。

长文档与学术来源

DeepSeek-V4 在数据构造中“特别强调长文档的整理”，优先收录科学论文与技术报告 [7]。这类数据既提供知识密度，又提供真实的长程依赖，供长上下文训练使用（第七章）。arXiv 有两种形态：LaTeX 源码保留公式语义，PDF 解析结果保留版面。二者可以互补，但必须按论文 ID 去重。

垂直领域的四种形态

Nemotron-Pretraining-Legal-v1 给出了一个可复用的配方 [17]，同一份领域知识被组织成四种形态：

1. **原文**：直接从 HTML 抽取法规与司法意见，包括加州法规、纽约司法伦理意见、eCFR；
2. **清洗**：用 LLM 清洗 540 万条判例摘要；
3. **任务改写**：把 CaseHOLD 改写成选择题，把 ContractNLI 改写成推理任务；
4. **合成任务**：判例结果预测、定义分类、条款理解等。

在 Nano 规模模型上做 100B token 消融，LegalBench 平均从 64.6 升到 74.7。四种形态各有分工：原文提供知识，清洗提高信噪比，任务改写与合成把知识转成可以被调用的形式。这个配方可以直接迁移到医学、金融、专利等领域，迁移时变化的只是原文来源与任务模板。

多语言长尾

多语言数据的价值，主要在长尾知识。DeepSeek-V4 扩大多语言语料，目的是“捕捉不同文化的长尾知识”[7]；Qwen3.5 的语言覆盖从 119 种扩到 201 种 [8]。语种越多，管线越不能一刀切：FineWeb2 把网页管线扩展到一千多种语言，并为每种语言单独设定 LID 阈值与过滤参数 [36]。

原因在于低资源语言最大的风险是 LID 误判。短文本的置信度天然偏低，近缘语言容易互相误判，混写文本则会被归到占比更大的那种语言。全局统一的阈值，会让高资源语言保留过多噪声，同时把低资源语言的大量正常文本删掉。

2.6 多模态原始数据

图像、视频、语音与交互数据的原始量都很大，难点是把它们和文本描述对上。

图文与交错网页

图像 URL 来自 HTML 的 `<img>` 标签与 WAT 的链接图，文本来自 alt-text 或周边段落。alt-text 噪声大，常常只是文件名或 SEO 关键词，所以主流做法是重写 caption（11.2 节）。交错文档在 HTML 中保留了图文的相对位置（11.3 节）；图像链接会随时间失效，快照越老，下载成功率越低。

视频：从 URL 到千万小时

LAION-BVD 展示了开放视频数据的完整获取链路 [37]：

1. 从 CC 的 WAT 文件中收集 13 亿个视频平台 URL；
2. 尝试下载 1.3 亿个，成功约 60%，得到 8000 万个视频、约 1000 万小时；
3. 剔除短于 10 秒或长于 30 分钟的视频，再按场景切分并去掉静止片段；

4. 用 Qwen3-VL-2B 为视频写 caption，用 Audio Flamingo 3 为音频写 caption。

最终产出 5500 万个带描述的音视频片段，以及 3 亿个“帧–描述”对。产业界的规模与之同量级：HunyuanVideo 1.5 的原始素材经基础去重后超过 1000 万小时，最终得到 8 亿个预训练片段（12.4 节）[38]。

教学估算：1080p H.264 约 5 Mbps，每小时约 2.25 GB，1000 万小时约 22 PB。视频管线的第一原则是尽早切片、转码、降采样，只保留后续阶段需要的分辨率（第十四章）。

语音

MiMo-Audio 的预训练语音超过 1 亿小时 [39]，来源包括播客、有声书与视频音轨；Kandinsky 6.0 的 4000 万条音频轨，就是从其视频库中抽取的 [40]。语音的原始量与可用量差距很大，要经过人声分离、说话人切分、ASR 转写与质量过滤，才能进入训练（第十三章）。

交互与具身

交互数据可以按“动作标签从哪来”分成四种来源。第一视角人类视频成本低、多样性高，但动作需要伪标注。真机遥操作的动作精确，但昂贵，目前最大的数据集在数千小时量级。屏幕录制与 GUI 轨迹的动作要和界面状态对齐。游戏录像加键鼠日志则天然带有动作标签。第十三章会讨论怎样把这些廉价信号变成可训练的动作标签。

2.7 元信息与溯源

前六节产出的每一份数据，都需要带着足够的元信息进入后续流程，否则第三章改阈值、第四章追溯重复、第十五章做归因时，都要重算一遍。

最小元信息集

每篇文档至少应记录下表中的字段：

类别	字段
标识与来源	`doc_id`（内容哈希）、`url`、`domain`、`crawl_date`、`source`
抽取	抽取器及其版本
语言	语言及其置信度
质量	每个分类器的原始分数，而不只是“通过 / 不通过”
分类	主题与格式
去重	MinHash 签名与去重簇 ID
规模	在某个 tokenizer 下的 token 数
版本	处理管线的版本号

保存所有中间分数，是因为阈值会反复调整，而重算一遍分类器的代价远高于多存一列浮点数。`crawl_date` 也有实际用途：Olmo 3 去重时，在每个重复簇中保留抓取时间最新的副本 [13]。

版本与回溯

每个处理阶段的输出版本，由输入数据的版本、代码版本与配置三者共同决定。数据集版本不可变，只能追加新版本，这带来三个好处：任一训练 shard 都能追溯到原始 URL 与处理路径；发现问题文档时，可以精确定位哪些训练运行受到了影响；增量处理新快照时，只需计算新文档与已有签名之间的去重。第十四章会讨论这套机制的工程实现。

时间外评测集

`crawl_date` 的另一个用途，是构造时间外评测集：用截止日期之后的文档测量模型的困惑度与知识覆盖。它既能检验时效性，又能避开评测集早已被训练数据覆盖的问题。

本章思考与进阶练习

2.1 抽取器消融

题目 你想把网页抽取器从 WET 换成 resiliparse。怎样设计实验，才能把收益归因到抽取器本身？

解题思路 固定 URL 集合、去重方式与后续过滤流程。质量分类器如果是在 WET 输出上训练的，需要在新输出上重新标定阈值；也可以让两组都只用不依赖分类器的规则过滤。结果要同时报告产出 token 数、BPB、早期信号强的任务与吞吐；两组的 token 数差距大时，再加一组按 token 数对齐的对照。

2.2 PDF 回退阈值

题目 Olmo 3 规定，回退页超过 1/250 的文档整篇剔除。一本 500 页的书有 3 页需要回退，一篇 10 页的论文有 1 页需要回退，分别怎样处理？这条规则会带来什么偏差？

解题思路 书的回退比例为 $3/500 = 0.6\%$，论文为 10%，都超过 0.4%，都会被剔除。按比例设阈值，意味着 500 页的书最多容忍 2 页回退、10 页的论文一页都不能回退：规则对所有长度一视同仁地要求文本层几乎完全可靠，宁可多删，也不让解析失败的页面混进训练集。它的偏差是系统性的：版面复杂、图表多的文档更容易触发回退，表格与图表密集的领域覆盖会下降。应统计被剔除文档的主题分布，必要时让这些主题改走更强的 VLM 解析路线，而不是直接放宽阈值。

2.3 进阶：时间截止与评测泄漏

题目 刷新网页快照后，基准分数明显上升。怎样判断这是知识新鲜度带来的真实收益，还是评测题泄漏？

解题思路 先用新增文档对评测集做 n-gram 与 embedding 相似度检查（第四章）。再把评测题分成“在新数据中有命中”和“无命中”两组，分别计算提升：提升集中在有命中的一组，就是泄漏；两组提升相近，才更可能是泛化收益。最后用新快照之后才发布的题做外部检验，并比较时间外文档上的 BPB 变化。

第三章　过滤、打分与生成式精炼

抽取得到的是原料池，本章讨论怎样从中取出对目标能力贡献最大的部分。手段按处理深度排列：先是便宜的规则，再是质量分类器与 LLM 标注，然后用分桶与上采样把分数变成配比，再处理 2026 年新出现的 AI 生成文本，最后是让模型改写原文的生成式精炼。本章结尾讨论目标导向的选择与多模态过滤。每一种手段都在改变训练分布，所以本章始终同时关注两件事：留下了什么，删掉了什么。

3.1　数据处理深度的分层视图

过滤与精炼的区别，在于模型是否改写了内容；改写的深度，决定了数据还算不算“原始数据”。

Data Darwinism：L0–L9

daVinci-LLM 按模型介入的深度，把数据处理分成十级 [41]：

级别	做什么	是否改写内容
L0–L1	获取；格式规范化（HTML 或 PDF 转文本、统一编码）	否
L2	规则过滤：语言识别、长度、重复、MinHash	否
L3	轻量模型过滤：fastText 或 embedding 质量分类器	否
L4	生成式精炼：LLM 清洗噪声、修复格式，但不新增知识	是
L5	认知补全：LLM 把原文隐含的推理步骤写出来	是，并加入信息
L6–L9	上下文补全、环境合成、生态合成、世界合成	大部分尚属构想

本章覆盖 L0–L4。L5 及以上属于合成数据，在第六章讨论。

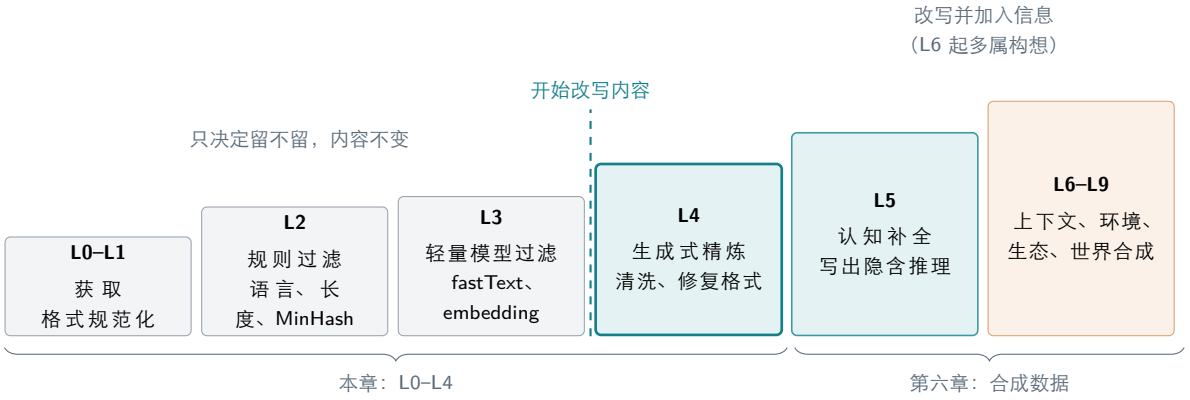


图 3.1　按模型介入深度划分的数据处理层级。读图：L0–L3 只决定一篇文档留不留，内容不变；从 L4 开始模型改写内容，L5 起还会加入原文没有的信息。本章讨论 L0–L4，L5 及以上属于第六章的合成数据。[41]

处理深度与收益

在 3B 模型、8T token 的设置下，L3 过滤让 MBPP 提升了 3.40，L4 精炼让 MATH 提升了 7.00，GSM8K 却只提升 1.37 [41]。推理密集的领域从更深的处理中获益更大：数学文本里的噪声（公式断裂、步骤缺失）会直接破坏推理链，只筛不改无法修复这类问题。

过滤是在改变分布

硬过滤与软加权分别把原始分布 p_{raw} 变成：

$$p_{\text{hard}}(x) \propto p_{\text{raw}}(x)\mathbb{1}[q(x) > \tau], \qquad p_{\text{soft}}(x) \propto p_{\text{raw}}(x)\, w(q(x)),$$

其中 $q(x)$ 是质量分，τ 是阈值，w 是按质量分加权的函数。

质量分与主题、格式、语言都相关，所以过滤必然改变主题分布。Nemotron-CC 的测量是：常规启发式过滤会误删 18.1% 的高质量 token（按 FineWeb-Edu 分类器判定）；对高质量部分跳过启发式过滤后，MMLU 从 55.5 升到 57.5 [20]。

规则是为“去垃圾”设计的；把它作用在高质量数据上，删掉的往往是最有价值的长尾格式。

3.2 启发式过滤

规则过滤便宜、可解释，适合先去掉大头的垃圾。它的风险集中在误删的长尾上。

语言识别

常用工具有 fastText LID、GlotLID 与 Lingua。FineWeb 保留英语置信度不低于 0.65 的文档 [24]；Olmo 3 的网页用 fastText 做英语过滤，PDF 改用 Lingua [13]。

语言识别有三个常见陷阱：短文本的置信度天然偏低，近缘语言容易互相误判，代码与公式则会拉低语言置信度，让技术文档被误删。对含大量代码或公式的文档，应先去掉代码块与公式，再做语言识别。

三套经典规则

Gopher 的质量规则针对“不像正常文章”的文档 [42]：词数要在 50 到 10 万之间，平均词长在 3 到 10 个字符之间，必须包含至少两个常见停用词；符号（# 或…）与词数之比超过 0.1、九成以上的行以项目符号开头、三成以上的行以省略号结尾，或者含字母的词不到 80% 的文档都被剔除。Gopher 的重复规则检查重复行与重复段落的占比、最高频 2/3/4-gram 的字符占比，以及重复 5–10-gram 的字符占比，阈值随 n 的增大逐级收紧。

C4 的规则更激进 [43]：只保留以终止标点结尾、至少有 5 个词的行，少于 3 句的页面剔除，含有“lorem ipsum”、花括号或不良词表中词汇的页面也剔除。

FineWeb 逐条测量了这些规则的代价 [24]。C4 的终止标点规则单独就删掉了约 30% 的 token，所以 FineWeb 没有采用它，而是换成三条自定义规则：以标点结尾的行不超过 12% 的文档剔除（去掉约 10% 的 token），重复行字符占比不低于 10% 的剔除（约 12.5%），短于 30 字符的行不少于 67% 的剔

除（约 3.7%）。三条合计去掉约 22% 的 token，在 28B token 的消融中把综合分数提高了约 1%。这组测量说明，每条规则都要单独记录“删掉了多少”与“换来了多少”。

产业量级

Olmo 3 的启发式过滤包括 URL 黑名单、长度与符号检查、重复与垃圾过滤、fastText 英语过滤，以及 Madlad-400 的句级规则：先给每个句子打可疑标记，再看文档中可疑句子的比例 [13, 44]。网页文档从 2526 亿篇降到 388 亿篇，剔除 84.6%。后面的分类器与去重，是在剩下的 15% 中继续精挑。

误删要按切片审计

教学假设：1000 篇文档中有 200 篇垃圾，“平均词长 3–10”这条规则剔除了 150 篇垃圾和 30 篇正常文档。它的精确率为 83%、召回率为 75%，正常文档的损失只有 3.75%。

但如果这 30 篇正常文档集中在“化学式密集的论文”这个子域（共 50 篇），这个子域就损失了 60%，只剩 40%。整体保留率会掩盖子域的塌陷，规则必须按主题、格式、语言切片审计误删率。

3.3 质量分类器

规则能识别“明显的垃圾”，却分不出“平庸”与“优秀”。分类器把“像好文档”学成一个分数。

fastText：DCLM 的配方

DCLM 用词袋加 bigram 特征训练 fastText 二分类器 [26]：正例是 OpenHermes-2.5 指令数据加上 ELI5 社区的高分帖子，负例是随机网页，保留分数最高的 10%，得到 DCLM-Baseline。在 DCLM 的对比中，这个分类器优于困惑度过滤等方案，7B 模型训练 2.6T token 后，MMLU 5-shot 达到 64%。Olmo 3 在此基础上扩充了样本 [13]：正例加入 UltraChat-200k 与 WildChat-1M，负例用 30GB 的 DCLM-RefinedWeb。

fastText 的优势是吞吐：纯 CPU 运行，单核每秒可以处理数千篇，能直接扫过百亿级文档。

Embedding 分类器

另一类分类器在句向量上加一个回归头。FineWeb-Edu 用 Snowflake-arctic-embed-m 拟合 LLM 打出的教育分 [24]；Nemotron-CC 用同样的结构，分别拟合 Mistral 8x22B 与 Nemotron-340B 的打分 [20]；GLM-5 也训练了一个基于句向量的 DCLM 式分类器 [16]。

embedding 分类器能捕捉语义，而不只是词汇特征，但需要 GPU。常见做法是两级串联：先用 fastText 粗筛，再对剩下的文档做 embedding 精排。

正例决定“质量”的含义

分类器学到的是“像正例”，而不是抽象的“好”。以指令数据为正例（DCLM），留下的文本偏向问答式、解释性的写法；以教育价值为正例（FineWeb-Edu），留下的文本偏向教材与科普，在 MMLU、ARC 等知识推理任务上提升明显，但叙事、对话、常识类文本的覆盖会下降。这两种偏好是互补的，所以 Nemotron-CC 把三个分类器集成起来，取三者的最大分（3.5 节）。

选择正例之前，先想清楚目标能力需要什么样的文本，再找能代表它的样本。

3.4 LLM 标注与蒸馏打分

分类器的上限取决于标签。用大模型打标签，再蒸馏到小模型全量推广，是 2024 年以来的主流做法。

FineWeb-Edu：大模型打分，小模型推广

FineWeb-Edu 的流程分三步 [24]：先用 Llama-3-70B-Instruct 给 45 万篇文档打 0–5 分，再在 embedding 上训练回归头拟合这些分数，最后对全量文档打分、按阈值筛选。打分用的是累加式 prompt：每满足一条标准（与教育相关、连贯、适合中小学等）就加 1 分。累加式让每一分都有明确的语义，比直接要一个 1–5 的总分更稳定。

阈值决定规模：取 3 分以上得到 1.3T token（FineWeb-Edu），取 2 分以上得到 5.4T token。回归头的训练目标是

$$\min_{\phi} \ \mathbb{E}_x \left[\left(f_\phi(e(x)) - s_{\text{LLM}}(x) \right)^2 \right],$$

其中 $e(x)$ 是冻结的文本 embedding，$s_{\text{LLM}}(x)$ 是大模型给的分数。用回归而不是分类，是为了保留分数的序关系，便于事后调整阈值、按分数加权上采样。

多维评分与按分数采样

QuRating 把“质量”拆成写作风格、所需专业度、事实与知识、教育价值四个维度 [45]。它让 LLM 做成对比较，再用 Bradley–Terry 模型拟合出每个维度的标量分。它的关键发现在于用法：按分数带温度采样

$$P(x) \propto \exp\left(q(x)/T\right)$$

优于直接取分数最高的 top-k。纯 top-k 会让主题塌缩到少数高分领域，温度采样保留了多样性。

长文档：分块打分再聚合

LLM 打分的窗口有限，长文档必须分块。GLM-5 对数学与科学数据采用“分块打分、再聚合”的方法 [16]，而聚合方式会改变结果。

教学假设：一本教材切成 10 块，正文 7 块各得 4 分，目录、索引、版权页 3 块各得 1 分。取最大值得 4 分，但同样会放过“一段好内容加大量垃圾”的文档；取平均值只有 3.1 分，被附属页拉低。更好的做法是先在块级别剔除低分块，再对剩下的块在文档级别聚合。

为知识单独设一个分类器

教育分类器会把大量“中低质量但含知识”的网页判成低分，例如产品说明、地方新闻、百科式条目。GLM-5 为此额外训练了一个 World Knowledge 分类器，标签来自 Wikipedia 条目与 LLM 标注，专门从中低质量网页中回收知识价值 [16]。背后的原则是：每个质量维度配一个分类器，分别设阈值，再取并集，因为一个分数代表不了所有维度。

标注质量的校准

保留几百篇人工标注的锚点文档，每次更换标注模型或 prompt，先在锚点集上检查与人工打分的 Spearman 相关性。不同标注模型的分数分布也不同，所以 Nemotron-CC 先把每个分类器的输出映射到分位桶，再做集成 [20]。

3.5 分桶、集成与按质量上采样

分类器给出的是分数，配比需要的是“每个桶采多少”。分桶把连续的分数变成可以配比的离散单元。

Nemotron-CC：取最大值，分成 20 个桶

Nemotron-CC 把三个分类器的输出各自映射到 0–19 的整数桶，每桶约含 5% 的文档，文档的最终分数取三者的最大值 [20]。再用继续预训练的实验把 20 个桶合并成 5 个质量档：在训练到 70% 的 8B 模型上，用 34% 的比例混入待测的桶，训练 50B token，比较下游表现。最终 High 档（第 19 桶）占 12.63% 的 token，Medium-High 档（第 18 桶）占 11.52%，中间档合计占 46%，低档合计占约 30%。

不同质量档有不同的去向：高质量档做多样化的 QA 合成（第六章），并提高重复次数；低质量档只做维基风格的改写；中间档保留原文。

Olmo 3：主题 × 质量

Olmo 3 先用 WebOrganizer [46] 把网页分到 24 个主题（为提高吞吐，蒸馏到 fastText），再在每个主题内按质量分切成 20 个分位，共 480 个子集 [13]。配比在两个层面分别优化：主题之间的比例，以及每个主题内部各分位的采样强度（第五章）。

用采样曲线代替硬阈值

硬阈值在两头都有浪费：阈值以上的文档都只用一次，最好的 5% 没有得到更多权重；阈值以下的全部丢掉，次优数据被完全浪费。Olmo 3 为每个主题 k 定义一条上采样曲线 $u_k(s)$，其中 $s \in [0,1]$ 为质量分位 [13]：

$$\int_0^1 u_k(s)\,ds = \frac{T_k}{P_k}, \qquad 0 \le u_k(s) \le 7, \qquad u_k \text{ 单调不减},$$

其中 T_k 是该主题的目标 token 数，P_k 是池中的 token 数，7 是最大重复次数。

教学例子：某主题池有 1T token，目标 500B，即平均采样强度 0.5。

质量分位	硬阈值（前 50%）的重复次数	上采样曲线的重复次数
0–50%	0	0
50–60%	1	0
60–85%	1	1
85–97%	1	1.5
97–100%	1	2.33

两种方案的总量都是 500B。上采样曲线只用了 400B 唯一 token，但前 3% 的数据在训练中的占比从 6% 提高到 14%：用 100B 唯一 token，换来顶部数据两倍多的曝光。重复次数控制在 4 次以内时，这个代价很小（5.8 节）。GLM-4.5 与 GLM-5 在代码数据上也采用按质量分采样，而不是截断 [16]。

3.6 反模板与反 AI 文本过滤

前面几节都假设“低质量”来自人写得差。2026 年出现了一个新的低质量来源：机器批量生成的文本。它往往看起来干净、流畅，恰好能通过质量分类器。

规模：网页中的 AI 文本

一项 2026 年 9 月的研究，用检测器测量了经过 FineWeb 质量过滤的 Common Crawl 中 AI 生成 token 的占比：2024 年 6 月为 10.1%，2026 年 6 月为 27.5%，2026 年 8 月为 31.1% [23]。过滤后的占比高于原始数据，因为 AI 文本比人写的文本更容易通过质量过滤。

研究者从头训练了 800 个模型（1990 万到 9.73 亿参数），按不同比例混合人类文本与 AI 文本。结论是：AI 文本只在模型严重缺少人类数据时（每参数少于约 10 个人类 token）有帮助，到了计算最优的约 20 个 token 每参数，帮助就消失了，再往上反而会抬高在人类文本上的损失。按他们拟合的规模律，以 2026 年 8 月的 AI 占比直接训练未过滤的网页，要达到同样的人类文本损失，需要只训练人类文本时 1.6 倍的计算。

这项研究的局限也要一起看：最大模型不到 1B，只测量了损失、没有测量下游能力，检测标签来自作者所属公司的检测器。但它给出了一个可以操作的结论：在人类文本与 AI 文本上分别报告验证损失。该研究中，在出现损害的实验里，有 95.5% 的损害被混合验证集掩盖了。

产业做法

2026 年的几份报告都把这一步单列出来。DeepSeek-V4 的网页过滤专门“去除批量自动生成和模板化的内容，以降低模型坍缩的风险”[7]；GLM-5 在数学与科学数据中加入了排除合成、AI 生成和模板化文本的过滤器 [16]。MAI-Thinking-1 走得最远：预训练完全不用语言模型生成的合成数据，并尽量识别、删除抓取数据中的 AI 生成内容，还对模板化页面做“骨架化”处理后再去重（4.1 节）[21]。

识别方法

识别分三个层面。模板页（商品页、天气页、SEO 农场）的特征是同一域名下 DOM 骨架相同、跨文档的 n-gram 重合度高；批量生成的内容表现为同一域名下文档长度的分布异常集中、句式模板反复出现；单篇 LLM 生成的文本，则在参考语言模型下困惑度异常低，带有典型的套话，句长方差小。

单篇文档的分类器容易误判低困惑度的正式文本，例如法规与标准文档，所以域名级与批次级的统计信号，比单篇文档的分类器更可靠。

与有意合成的边界

被过滤的是来源不明的机器文本。有意生成的合成数据要带来源标记（`source=synthetic`、`generator@version`、`prompt_id`），单独成桶，比例受控（第六章）。两者在管线中必须能区分，否则既无法审计，也无法做消融。

3.7 生成式精炼（L4）

过滤只能选择或删除，生成式精炼让模型改写原文。它的边界，是不新增知识。

定义与约束

L4 精炼用 LLM 逐篇改写文档，只允许做三件事：去除模板残留、广告与乱码，修复断行、表格与 LaTeX 的格式，修正明显的 OCR 与拼写错误。它不允许新增知识，也不允许补出原文没有的推理步骤，那是 L5。Nemotron-CC-Math 用 Phi-4 做的清洗，就是典型的 L4（2.2 节）。

MAI-Thinking-1 给出了更严格的工程化约束：LLM 在清洗中只能保留或删除文本片段，不能写入任何新文本 [21]。这样一来，“是否越过 L4”在机制上就不可能发生，不再需要事后审计；代价是错字、断裂的公式无法被修复，只能整段删掉。

DataEvolve：自动进化清洗策略

为每类数据人工写清洗 prompt，成本高，也难以迭代。DataEvolve 用多个 agent 组成闭环，自动进化清洗策略 [47]：

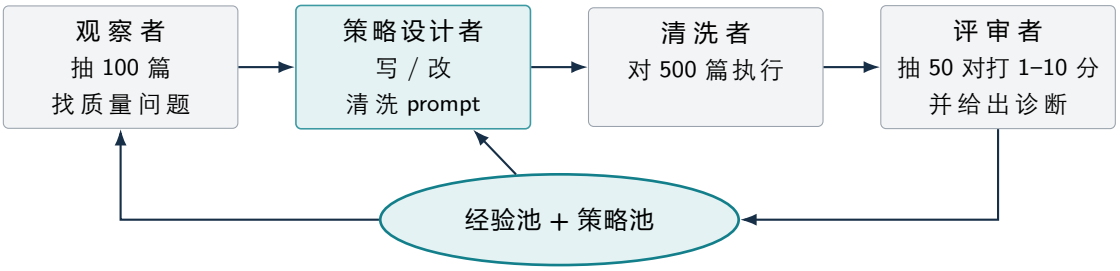


图 3.2 DataEvolve 的策略进化闭环。读图：从观察者出发，沿箭头完成一轮“发现问题 → 写策略 → 执行 → 评审”。评审结果写入经验池，供下一轮的观察者与设计者使用。整个闭环只在样本上评估，不训练模型，所以每轮的成本很低。[47]

每个数据类别迭代 30 轮。用它处理 Nemotron-CC 中 8 个学术 STEM 类别（672B token），得到 504B token 的 Darwin-CC。3B 模型训练 500B token 后，18 项基准的平均分为 44.13，原始数据为 40.17，次优策略为 41.20，DCLM 为 42.42；其中 MMLU 提升了 18.64。需要注意，Darwin-CC 只覆盖学术 STEM，与通用语料的比较并不完全公平。

成本与忠实度校验

教学假设：精炼 500B 输出 token，使用 8B 级生成器，单卡吞吐每秒 1 万 token。共需约 1.39 万 GPU 时，按每 GPU 时 2 美元计约 2.8 万美元，输入侧的 prefill 另计。这与动辄几十万 GPU 时的预训练相比很小；第六章还会看到，生成器超过 1–3B 后，改写质量的收益很快就饱和了。

忠实度校验看四个信号：数字与实体的保留率；新实体率，即输出中有、原文中没有的实体，出现即需要复核；长度比，输出与原文的长度之比超出 [0.5, 1.2] 视为异常；以及输出与原文的 embedding 相似度。

3.8 目标导向的数据选择

前面几节问的都是“这篇文档好不好”。目标导向的选择换了一个问题：这篇文档对我要的能力有没有用。

参考模型困惑度

CCNet 为每种语言在 Wikipedia 上训练 KenLM 5-gram 模型，用困惑度把网页分成 head、middle、tail 三等份 [48]。问题在于，参考语料定义了什么是“干净”：口语、方言与小众领域的困惑度偏高，会被系统性地下调。困惑度适合作为一个特征，不宜单独作为过滤标准。

DSIR：重要性重采样

给定一批代表目标分布的样本（例如高质量书籍），DSIR 用哈希化的 n-gram 特征，分别拟合目标分布与原始分布的词袋模型，再按重要性权重重采样 [49]：

$$w(x) = \frac{\hat{p}_{\text{tgt}}(x)}{\hat{p}_{\text{raw}}(x)}.$$

教学假设：某个 bigram 特征在目标分布中的频率是原始分布的 5 倍，另一个特征只有原始分布的 0.2 倍。一篇文档同时含有这两个特征各一次时，词袋模型下两者的贡献相乘，权重为 $5 \times 0.2 = 1$，既不被偏好也不被排斥。实现上用 Gumbel top-k 做不放回采样。DSIR 代价低、可以扩展到全量数据，但特征只停留在 n-gram 层面。

LESS：梯度相似度

LESS 先用 LoRA 在少量数据上预热，然后计算每个训练样本经过 Adam 预条件的梯度，随机投影到 8192 维，再按它与目标任务样本梯度的余弦相似度打分 [50]。只选 5% 的指令数据，效果常常超过使用全量数据。在预训练规模上计算每个样本的梯度太贵，这类方法主要用于 SFT 与中期训练的数据选择。

让模型自己参与选择

选择也可以细到 token。RHO-1 用一个在高质量数据上训练的参考模型，计算每个 token 的“超额损失”，即当前模型的损失减去参考模型的损失，只在超额损失高的 token 上计算梯度：参考模型也学不会的噪声 token 与当前模型已经学会的 token 都被跳过 [51]。到文件级，Seed-Coder 用 LLM 给代码打分，再蒸馏成过滤器（2.4 节）[32]。

两者的共同点是：数据的价值取决于当前模型学到了哪里。这条思路在 RL 数据中发展为按通过率筛选 prompt（10.2 节）。

3.9 多模态质量过滤

图像、视频、音频的过滤信号不同，但原则与文本一致：便宜的规则先行，阈值随训练阶段收紧。

图像

Z-Image 为每张图计算一组信号 [52]：分辨率、文件大小、pHash 等元信息；压缩伪影、偏色、模糊、水印、噪声等技术质量；边缘像素方差、JPEG 每像素字节数等低信息量指标；美学分、AIGC 检测、VLM 语义标签与 NSFW 分数等内容信号；以及 CN-CLIP 图文相关性，低于阈值的丢弃。

Qwen-Image-2.0 把这些过滤器按训练阶段逐级启用 [53]。256p 预训练只启用坏文件、分辨率、去重、NSFW、旋转、熵、CLIP 对齐与 caption 长度这些基础过滤器；到 512p 与 1024p 阶段，再加入画质、美学与压缩质量；多分辨率阶段提高分辨率门槛；SFT 阶段复用前面的过滤器，并收紧阈值。

低分辨率阶段重覆盖，高分辨率阶段重质量。同一组过滤器随阶段收紧阈值，比为每个阶段单独设计过滤器更可控。

视频与音频

视频在图像信号之上多了两个维度：镜头边界与运动。镜头切分之后，要剔除残留的转场、裁掉字幕与水印，并去掉静态或低运动的片段；Wan 甚至把运动质量单独分成六档。音频则多了音画同步：Kandinsky 6.0 对检测到人物的片段要求唇形同步分不低于 3（满分 5）[40]。这两类过滤与生成模型的数据管线紧密耦合，具体流程与阈值的取舍放在 12.4 节与 12.7 节讨论。

本章思考与进阶练习

3.1 分类器的正例

题目 分类器 A 以指令数据为正例，分类器 B 以教科书为正例，两者各取前 10%。训练出的模型在知识问答、常识推理、对话流畅度上会有什么差异？怎样组合两者？

解题思路 A 偏向问答与解释，B 偏向系统性知识，两者都可能削弱叙事与日常对话类文本。组合有两种方法：各自映射到分位桶后取最大值，即 Nemotron-CC 的做法，保留“在任一维度上优秀”的文档；或者取并集后，再按主题配比回填。无论哪种，都要用切片评测验证，并检查被两者同时排除的主题。

3.2 硬阈值与上采样

题目 某主题池有 1T token，目标 400B。方案一：取前 40%，各用一次。方案二：前 10% 重复 2 次，70–90% 分位用一次，其余丢弃。比较两者的唯一 token 数，以及前 10% 数据在训练中的占比。

解题思路 方案一唯一 400B，前 10% 占 $100/400 = 25\%$；方案二唯一 300B，前 10% 占 $200/400 = 50\%$。方案二用 100B 唯一 token 换来顶部数据加倍的曝光，重复 2 次远低于 4 次的经验上限。但若该主题的唯一数据本来就稀缺，或者模型很大、对多样性更敏感，方案一更稳妥。

3.3 进阶：生成式精炼的信息增量

题目 L4 精炼之后，下游分数上升了。怎样确认收益来自去噪，而不是精炼模型偷偷加入了知识？

解题思路 先统计输出中的新实体率与新数字率，抽样核对新增内容在原文中是否有依据。再设一组“只删不写”的对照（MAI 式约束），按 token 数对齐训练：L4 组的收益如果远超对照组，说明改写本身贡献了信息。还可以挑一些只在精炼模型的知识中、不在原文中的事实，测试训练后的模型是否学会了它们。若确实有知识注入，就把这批数据归为合成数据，按第六章的合成比例单独管理。

第四章　去重、去污染与数据审计

过滤回答“要不要这篇”，去重回答“这篇是不是已经有了”。重复数据会浪费计算、放大记忆，并让少数模板在梯度中占据过高的权重；评测题混进训练集，则会让所有的比较失效。本章先讨论精确、模糊、子串、语义四类去重，再区分有意与无意的重复，然后讨论去污染，最后讨论怎样审计一个 30T 级的语料。下一章讨论去重之后的数据怎样组织成配比与阶段。

4.1　三级去重流水线

去重按代价从低到高串联：每一级处理上一级的输出，前一级越便宜、去掉得越多，后一级的成本越低。

精确、模糊与子串

精确去重对规范化后的 URL、文档或行计算哈希，去掉镜像、转载与完全相同的页面，只需要一次 shuffle。模糊去重用 MinHash 与 LSH 找出只改了日期、广告或少量措辞的近似副本，需要计算签名并分桶。子串去重用后缀数组找出跨文档反复出现的长片段，例如免责声明、页脚、模板段落，它的成本最高，需要分片构建索引。

Olmo 3 的三级流水线给出了每一级的作用 [13]：

1. 精确哈希去重：把 387 亿篇文档降到 128 亿篇，去掉 67%；
2. MinHash 模糊去重（分 32 个分片执行）：降到 98 亿篇，再去掉 23%；
3. 后缀数组子串去重（分 57 个分片，针对 500 字节以上的片段）：又去掉 14% 的文本字节，最终剩下 97 亿篇文档、36.5T 字节。

整体文档数下降了 75%。有两条实施细节值得留意：模糊去重的每个簇中，保留抓取时间最新的副本；子串去重时，每个重复片段保留一次出现，而不是全部删掉。



图 4.1　Olmo 3 的三级去重流水线。读图：条形长度与文档数成正比。每一级处理上一级的输出，单位代价逐级升高、处理量逐级减少；精确去重去掉的最多，子串去重按字节删除片段，文档数几乎不变。[13]

2026 年的 MAI-Thinking-1 在这三级之外又加了两步 [21]。第一步是对模板化页面做“骨架化”：同一站点的商品页、目录页共用一套导航与页脚，按全文比较时，两页的相似度会被模板抬高；先把固定的页面结构抽出来，只比较可变的部分，模糊去重（阈值 0.8）比较的才是内容本身。第二步是用 Qwen3-Embedding-0.6B 做语义去重（4.3 节）。

子串去重的价值

Lee 等人用后缀数组在 C4 等语料中发现了大量跨文档重复的长片段，其中一句 60 多词的模板句出现了数万次 [54]。去掉这些片段之后，模型逐字复现训练数据的频率下降了约一个数量级，困惑度与下游表现持平或更好。文档级去重看不到“文档整体不重复、内部片段重复”的情况，子串去重正是为此准备的。

4.2 MinHash 与 LSH 的参数

模糊去重的效果，几乎完全由两个参数决定：签名怎样分带，以及每个带有多宽。

MinHash：用签名估计 Jaccard

把文档表示成 shingle 集合 S（例如词级 5-gram），两篇文档的相似度定义为：

$$J(A,B)=\frac{|S_A\cap S_B|}{|S_A\cup S_B|}.$$

对一个随机哈希函数 h，两个集合的最小哈希值相同的概率，恰好等于它们的 Jaccard 相似度：

$$\Pr\left[\min h(S_A)=\min h(S_B)\right]=J(A,B).$$

原因是：在 $S_A\cup S_B$ 中，哈希值最小的那个元素等可能地是其中任何一个；它同时属于两个集合的概率，就是交集在并集中所占的比例。用 k 个独立的哈希函数，得到一个 k 维签名，签名中相同分量的比例就是 J 的无偏估计。签名长度固定，与文档长度无关，所以这个方法能扩展到百亿级文档。

LSH 分带与 S 曲线

把 $k=b\times r$ 个分量分成 b 个带，每带 r 个。只要两篇文档在任意一个带里的 r 个分量全部相同，就成为候选重复：

$$P_{\text{cand}}(s)=1-\left(1-s^{r}\right)^{b},$$

其中 s 是两篇文档的真实相似度。这条曲线关于 s 呈 S 形，拐点约在 $(1/b)^{1/r}$。

FineWeb 用词级 5-gram、112 个哈希，$b=14$，$r=8$ [24]，拐点约在 0.72：相似度 0.6 的文档对成为候选的概率约 0.21，0.75 时约 0.77，0.85 时约 0.988。调参的规则很简单：增大 r，曲线变陡，误报变少；增大 b，召回率提高；二者的乘积决定签名的存储量。

候选确定之后，可以选择是否再做两两精确的 Jaccard 校验。Olmo 3 的 PDF 去重沿用了 FineWeb 的参数，没有做两两校验 [13]。

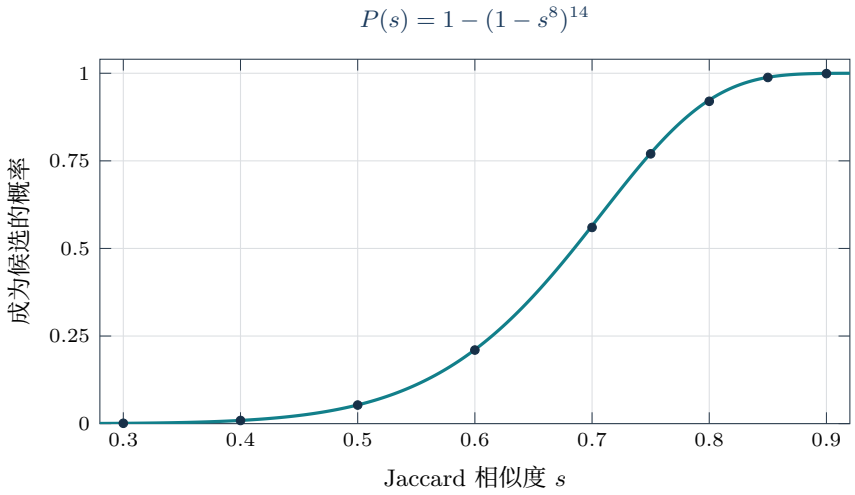


图 4.2 LSH 的 S 曲线。读图：横轴是两篇文档的真实相似度，纵轴是它们被判为候选重复的概率。拐点附近的曲线越陡，阈值越“硬”；左侧尾部的高度就是误报率。数值按 $P = 1 - (1 - s^8)^{14}$ 计算。[24]

逐快照去重，还是全局去重

全局去重听上去更彻底，FineWeb 的消融却得出了相反的结论 [24]。96 期快照全局去重（从新到旧处理）之后，最老快照的数据被删掉近 90%，只剩约 4T token，用它训练 350B token，只比不去重略好，明显不如 RefinedWeb；每期快照独立去重，留下约 20T token，与 RefinedWeb 持平；URL 级、行级等更轻量的全局去重，也都不如逐快照去重。

原因藏在最老的 2013-48 快照里：全局去重后保留下来的约 31B token，质量反而低于被删掉的约 171B token。被删掉的，是在多期快照中反复出现的稳定、高质量页面；只出现一次因而被留下的，更多是广告、关键词列表与格式混乱的页面。

重复次数本身就是一种质量信号。全局去重在删除重复的同时，系统性地留下了“没人转载”的长尾垃圾，所以去重策略必须和质量过滤一起评估。Nemotron-CC 在质量分桶之后再做全局去重，并按质量档决定重复次数（第三、五章）。

跨来源去重的顺序

同一内容可能同时出现在网页、PDF、arXiv 与书籍中，跨来源去重要先定好保留哪个来源。MAI-Thinking-1 为所有数据集规定了一个全局的“丢弃顺序”，保证一篇文档不会在不同数据集中被重复计数 [21]。一般的原则是优先保留抽取质量最高的版本，例如 arXiv 的 LaTeX 源码优先于网页转载，并在被删副本的元信息中记录 `dup_of`，便于回溯。

4.3 语义去重

MinHash 只能识别字面上相近的文档。改写、翻译、换模板之后的同一内容，要在 embedding 空间里识别。

近邻与聚类

语义去重把每篇文档映射成单位向量 e_i，两篇文档的相似度取余弦：

$$\cos(i,j) = e_i^\top e_j, \qquad \text{判为重复 iff } \cos(i,j) > 1-\varepsilon,$$

其中 ε 是允许的距离，ε 越小，只有几乎相同的内容才被删除。SemDeDup 先对 embedding 做 k-means 聚类，再只在簇内做两两比较，删除超过阈值的近邻 [55]，聚类的作用是把复杂度从全量降到簇内。

数值例子（教学假设：$N = 10^9$ 篇文档，$K = 10^5$ 个簇，簇大小均匀）：全量两两比较约 $N^2/2 = 5\times10^{17}$ 次，簇内比较为 $K \times (N/K)^2/2 = 5 \times 10^{12}$ 次，少了五个数量级。代价是跨簇的近邻会被漏掉，簇越多越快，漏得也越多。SemDeDup 在 LAION 子集上删掉一半数据，性能损失很小，训练时间约减半；在 C4 上训练语言模型时，也优于此前的去重方法 [55]。D4 在 SemDeDup 之后，再用原型距离挑选多样化的样本 [56]：前者处理“太像”，后者处理“太集中”。

十亿级：图上的社区发现

Z-Image 把语义去重建模为 k-NN 近邻图（k = 100）上的社区发现，每个社区只保留代表样本，8 张 H800 处理十亿条数据约需 8 小时 [52]。图方法比阈值法更能处理“链式相似”：A 像 B，B 像 C，但 A 不像 C。阈值法遇到这种情况，要么漏删，要么把整条链都删掉。

视频与长文档

视频中最常见的重复，是同一素材被不同账号转发、加字幕、裁剪、调色后再上传。这类重复在像素层面差异很大，只能靠语义 embedding 识别。NeMo Curator 的视频管线先用 Cosmos-Embed1 计算片段级 embedding，再在 k-means 聚类的簇内按两两相似度去重 [57]；Kimi K3 整理长上下文数据时，对视频帧做感知哈希，用来去除近重复的视频与长文档 [14]。

保护长尾

语义去重的阈值过低时，会把“主题相同但内容不同”的文档当成重复删掉，长尾主题首当其冲。对策是按簇的大小设定不同的阈值：大簇用严格的阈值，小簇用宽松的阈值，甚至不去重；去重之后再检查一次主题分布。

4.4 有意重复与无意重复

去重之后，高质量数据又会被有意地重复多次（第三、五章）。区别重复的好坏，要看是谁在重复、重复了多少次。

无意重复的代价

Hernandez 等人的实验表明，只把 0.1% 的数据重复 100 次，800M 模型的表现就退化到相当于 400M 模型 [58]。少量高频重复会抢占模型容量去记忆这些片段，还会损伤归纳头等泛化机制。

记忆与重复次数

可抽取记忆的定义是：给定训练文档中的一段前缀，模型能逐字生成其后的内容。Carlini 等人发现，可抽取记忆的比例随模型规模、重复次数与前缀长度近似对数线性增长 [59]。所以去重有两层作用：提高数据效率，并降低训练数据被逐字复现的概率。

有意重复的边界

两种重复在对象、次数与分布上都不同。无意重复的对象通常是模板与转载，多为低质量，次数不受控，单个片段可达数万次，并且集中在少数片段上。有意重复的对象是经过筛选的高质量数据，次数受控（Olmo 3 的配比优化把每个域的重复上限设在约 4–7 次 [13]，MAI-Thinking-1 设为 8 次 [21]），并按质量曲线平滑分布。

MAI-Thinking-1 还提出了一种按“记忆”设定重复上限的方法 [21]：统计验证损失的改善中，有多大比例来自负对数似然已低于 0.01 的 token，也就是模型几乎已经背下来的 token。设 $\Delta\ell_t$ 是 token t 在两个 checkpoint 之间的损失下降，则

$$m = \frac{\sum_{t:\ \ell_t < 0.01} \Delta\ell_t}{\sum_t \Delta\ell_t}$$

m 越高，说明这一段训练的收益越多地来自“把已经会的背得更熟”。某个来源的 m 随 epoch 快速上升，就说明它已被重复得太多，应当在那里设上限。这个判据比固定的“4 次”更细：多样性高的来源可以多重复几次，模板化的来源则更早触顶。

改写替代重复

Kimi K2 在 SimpleQA 上，用同一份维基文本做了对照 [60]：原文训练 10 遍得 23.76，改写 1 次、训练 10 遍得 27.39，改写 10 次、每个版本训练 1 遍得 28.94。同样 10 遍的曝光，用 10 个不同的版本各训练一次，优于把原文训练 10 遍。改写为同一份知识提供了多种表述，让模型学到知识本身，而不是一种固定的措辞。这正是第六章合成数据的出发点之一。

4.5 去污染

评测题一旦混入训练集，所有比较都会失真。去污染的难点在于，污染不只以原文的形式出现。

n-gram 重叠

最常用的方法，是把评测集的问题与答案切成 n-gram，在训练语料中检索命中。对一道题 q，它的重合率是

$$o(q) = \frac{|\{g \in G_n(q) :\ g \in G_n(D)\}|}{|G_n(q)|}$$

其中 $G_n(q)$ 是题目的 n-gram 集合，$G_n(D)$ 是训练语料的 n-gram 集合；$o(q)$ 超过阈值，就判为污染。

数值例子（教学假设）：一道 60 个 token 的题，13-gram 共 48 个，其中 30 个在训练语料中出现，$o = 0.63$。若阈值设为 0.5，这道题判为污染；若只有题干的前半段被转载、答案没有出现，o 可能只有 0.3，就会漏判。所以题干与答案最好分开计算，答案部分的命中更说明问题。

工程上可以用 Bloom filter 或后缀数组索引加速。infini-gram 能对万亿 token 级的语料做任意长度的 n-gram 计数 [61]，把“这道题在训练集中出现过几次”变成一次查询。

MAI-Thinking-1 用的是更宽松的规则 [21]：对全部语料做 20-gram 的模糊匹配，相似度阈值 80%；直接删除 huggingface.co 及其镜像站的全部内容，因为这些站点大量托管评测集；并且承认这些方法都不完美，另外使用私有基准做评测。

改写污染：n-gram 抓不到的部分

评测题被改写、翻译、换个数字之后进入训练集，n-gram 检测就完全失效了。Yang 等人的实验显示，在改写后的 MMLU 等基准上训练，13B 模型的分数可以被抬到接近 GPT-4 的水平，而 n-gram 去污染检测不出来 [62]。

对策是“embedding 检索 + LLM 判别”：先用 embedding 召回相似的训练样本，再让 LLM 判断两者是不是同一道题。产业上，Nemotron-CC-Math 用 Qwen2.5-32B 的 embedding 与 MMLU、MATH、GSM8K 的题目和答案比较，余弦相似度大于 0.9 的文档剔除 [25]；Nemotron-Pretraining-Specialized 只从公开数据集的训练集合成数据，并显式排除测试集 [17]。

合成数据与 RL 题库的去污染

污染风险最高的，恰恰是从公开数据集派生出来的数据。合成数据的生成器可能记得测试题，从种子题出发生成的变体也可能落在测试题附近，所以生成之后要再对评测集做一轮 embedding 检索。RL 题库同样如此：Olmo 3 的 RL-Zero 数据对预训练与中期训练数据都做了去污染 [13]，因为 RL-Zero 实验想回答的是“纯 RL 能学到什么”，如果预训练里已经见过答案，结论就不成立了。中期训练的思考轨迹中也可能包含基准题的完整解答，需要单独检测。

统计检测与报告口径

无法访问训练数据时，可以对模型做成员推断。Min-K% Prob 取一段文本中概率最低的 $k\%$ 个 token，计算它们的平均对数概率：见过的文本，即使最难的 token 概率也不会太低 [63]。它适合审计第三方模型或历史 checkpoint，但对单个样本的误判率不低，不能代替数据侧的检索。

去污染报告应写清楚五件事：检测方法与阈值，受污染题目的比例，在“干净子集”上的分数，去污染覆盖了哪些训练阶段，以及是否使用了私有评测集。任何一个阶段漏检，前面的去污染就都白做了。

4.6 数据审计

规则与模型都会出错。30T 级的语料，需要一套固定的方法来发现它们出了什么错。

分布面板

对 30T 级的语料，不能只看总量，至少要按主题与格式（WebOrganizer 各有 24 类 [46]）、语言、来源、时间与质量分位统计 token 分布与质量分分布。每次修改过滤或去重规则，都对比前后两版面板，确认改动只影响了预期的切片。

抽样人审

自动指标看不到的问题，要靠固定协议的抽样人审：每个“主题 × 质量档”抽一批文档，按清单检查抽取是否完整、是否模板化、是否为机器生成、是否含有个人信息，记录每类问题的比例，作为下一轮规则迭代的输入。

每格抽多少，取决于想把问题比例估得多准。设真实问题率为 p，希望估计值的误差不超过 e（95% 置信），所需样本数约为

$$n \approx \frac{1.96^2\, p(1-p)}{e^2}$$

数值例子（教学假设）：问题率约 5%、要求误差 ±2 个百分点时，$n \approx 456$；每格只抽 30 篇时，误差约为 ±7.8 个百分点。所以每格 20–50 篇的抽样，只能发现“这一格有严重问题”（例如问题率 30% 以上），不能用来比较两版规则之间几个百分点的差别。后者要把抽样集中在改动影响到的切片上，或者用模型做全量判别、人工只校准判别器。DataEvolve 的观察者 agent，就是把这一步自动化了（3.7 节）。

从模型输出回溯训练数据

模型出现异常输出（复读模板、泄漏个人信息、某类知识出错）时，需要找到对应的训练数据。OLMoTrace 基于 infini-gram，能实时把模型输出中的长片段匹配回训练文档 [64]；语义相近的来源，则用 embedding 检索。回溯依赖第二章的元信息与版本管理：没有 `doc_id` 与处理版本，就无法判断问题出在哪一步。

本章思考与进阶练习

4.1 LSH 参数

题目 签名长度固定为 120。你希望相似度 0.8 的文档有 95% 以上的概率成为候选，同时相似度 0.5 的文档成为候选的概率低于 5%。比较 $(b, r) = (20, 6)$ 与 $(10, 12)$ 两组参数。

解题思路 $(20, 6)$ 在 $s = 0.8$ 时 $P \approx 0.998$，$s = 0.5$ 时 $P \approx 0.27$，误报过高；$(10, 12)$ 在 $s = 0.8$ 时 $P \approx 0.51$，召回不足，$s = 0.5$ 时 $P \approx 0.0024$。两组都不满足要求。改选中间的 $(15, 8)$：$s = 0.8$ 时 $P \approx 0.94$，$s = 0.5$ 时 $P \approx 0.057$，两项都接近目标。也可以放宽召回，再对候选做两两精确校验。

4.2 改写污染

题目 你的合成数学数据是从 GSM8K 训练集改写来的，GSM8K 测试集的分数大涨。怎样判断这是不是污染?

解题思路 对每道测试题，在合成数据中做 embedding 检索，再由 LLM 判别是否为同一道题，统计命中率；然后比较命中题与未命中题的分数提升。最后用同难度、不同来源的新题集做外部检验：如果只有 GSM8K 上涨，说明学到的主要是题型记忆。

4.3 进阶：去重与上采样的冲突

题目 你先做了全局去重，又按质量曲线把高质量数据上采样 3 次。有人认为这“等于没去重”。这种说法对吗?

解题思路 不对。去重去掉的是不受控的重复：模板与转载，单个片段可能重复成千上万次，多为低质量；上采样是对筛选过的数据做受控、平滑的重复，次数在经验上限以内。但有两点要注意：若去重时保留的是低质量的副本，上采样会把错误的版本放大；上采样之后还要再做一次子串级检查，避免某个高质量文档内部的长模板被成倍放大。

第五章　预训练配比、阶段与规模律

前几章产出的，是一个个按来源、主题、质量分好的数据桶。本章回答怎样把它们组织成一次 20–36T token 的训练：每个桶采多少（配比 π_b），每个桶重复几次（ρ_b），配比是否随训练阶段变化，以及唯一数据不够时怎么办。配比方法已经从人工经验发展到“代理模型群 + 回归”；2026 年的报告则开始追问，小规模上找到的配比，在大规模上是否依然最优，这是本章最后一节的主题。

5.1　配比问题的形式化

配比是梯度上的权重；重复次数是配比与唯一数据量相除的结果。两者必须一起设计。

配比、重复与总预算

设有 B 个数据桶，桶 b 的唯一 token 数为 $|D_b|$，训练总预算为 T：

$$T_b = \pi_b T, \qquad \rho_b = \frac{\pi_b T}{|D_b|}, \qquad \mathcal{L}(\theta) = \sum_b \pi_b \, \mathbb{E}_{x \sim D_b} \left[\ell(x; \theta) \right].$$

其中 T_b 是分给桶 b 的训练 token，ρ_b 是它的重复次数，ℓ 是单个样本的损失。

改变 π_b，就是改变各类数据在梯度中的权重，而 ρ_b 由 π_b 与 $|D_b|$ 共同决定。所以任何配比方案都同时受两个约束：目标能力需要足够大的 π_b，而唯一数据有限，ρ_b 又不能超过上限（5.8 节）。

温度采样与重复上限

按自然比例采样，小语种几乎看不到。温度采样把比例拉平：

$$\pi_b \propto |D_b|^{1/\tau},$$

其中温度 τ 越大，各桶的比例越接近均匀。

教学假设：英语 1000B，中文 100B，斯瓦希里语 1B，总预算 1T。按自然比例（$\tau = 1$），三者分别占 90.8%、9.1%、0.09%；取 $\tau = 3$，变为 63.9%、29.7%、6.4%。此时斯瓦希里语分到 64B 训练 token，而它只有 1B 唯一 token，要重复 64 次，远超有效区间。

所以实际做法是“温度 + 上限”：先给每个桶设定重复上限，超出的预算按比例返还给其他桶；或者对小语种做翻译、改写等扩充（第六章）。5.4 节会看到，Olmo 3 把同样的上限直接写进了配比优化的约束。

配比要在什么单位上定义

第一章的练习 1.3 已经说明，按 token 定义的配比会随 tokenizer 改变。跨 tokenizer 迁移配比时，要按字节或文档重新标定；对比不同报告的配比时，也要先确认它们的口径。

5.2 多阶段预训练

配比不是一个常数。2026 年的报告都把预训练分成几个阶段，每个阶段的数据目标不同。

两阶段：先求广，再求精

Nemotron 3 Ultra 的 20T token 预训练使用 WSD（warmup-stable-decay）学习率，分为两个阶段 [17]：第一阶段约 15T token，重多样性与领域覆盖；第二阶段约 5T token，重高质量数据，切换点在训练的约 75% 处。

学习率在最后 5T token 内从 2.5e-4 衰减到 2.5e-6，所以第二阶段与学习率的衰减段重合；网页类数据的占比则从约 49% 降到约 38%。把高质量数据放在衰减段，是第七章“退火”逻辑的放大版。

三阶段：通用 → 推理 → 长上下文

Qwen3 的预训练分为三段 [28]：先用 30T 以上的 token、4K 上下文，覆盖 119 种语言的通用语言与知识；再用约 5T token 提高 STEM、代码、推理与合成数据的比例；最后用数千亿 token、32K 上下文的长上下文数据收尾。GLM-5 同样在早期就优先使用代码与推理数据，之后接中期训练与长上下文扩展，共 28.5T token [16]。

两种划分的共同点，是把“质量更高、量更少”的数据留到后面。原因有两个：一是后段学习率更低，高质量数据的边际收益更大（7.1 节）；二是高质量数据的唯一量有限，放在后段的较短窗口里，重复次数更容易控制在上限以内（5.8 节）。

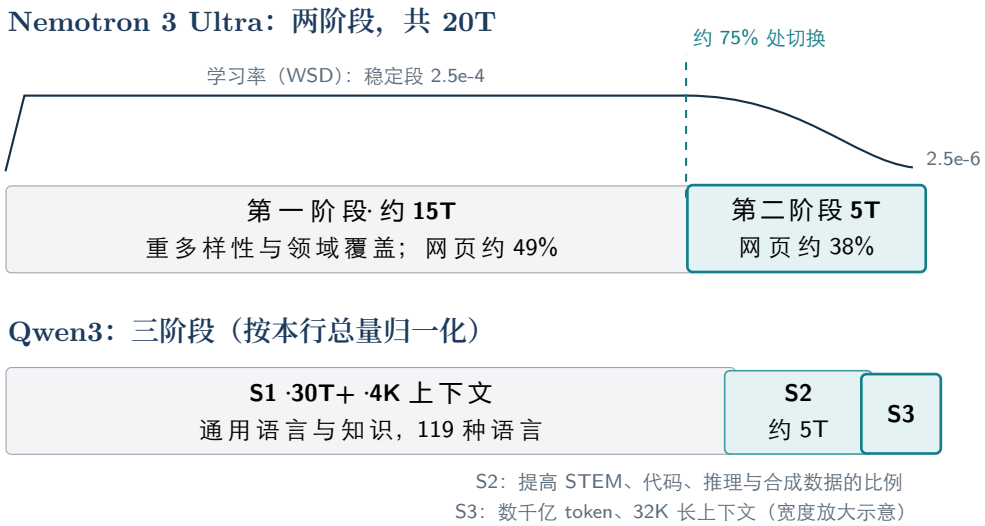


图 5.1　多阶段预训练的两种划分。读图：每行按各自的 token 总量归一化，Qwen3 的第三阶段只有数千亿 token，图中加宽示意。两种划分都把质量更高、量更少的数据放在后段；Nemotron 3 Ultra 的第二阶段与学习率衰减段重合。[17, 28]

阶段切换要逐步过渡

daVinci-LLM 的第二阶段（2T token）专门测试了结构化 QA 数据的比例 [41]：先用 30% 的 QA、再升到 70%，效果最好；第一步就把 QA 提到 100%，会损伤通用知识与代码能力。先用中等比例让模型适应新格式，再提高比例，可以避免塌缩。阶段切换时，数据分布应逐步过渡，而不是突然改变。

5.3 各家的配比与取舍

公开的配比表，只有和“唯一量”“重复次数”放在一起看，才能读出背后的取舍。

Nemotron 3 Ultra：合成数据成为第一大类

Nemotron 3 Ultra 第一阶段配比中占比最大的六类，依次是高质量网页的合成改写 22.4%、代码 14.0%、中等质量网页的合成改写 10.3%、SFT 风格的 STEM 数据 9.2%、高质量原始网页 6.5%、数学 6.4% [17]。第二阶段的主要变化，是用高质量 PDF（10.7%）替换中等质量的网页与改写。

这份配比有三个值得注意的信号。合成改写合计约 33%，是第一大类（第六章）；SFT 风格的数据合计约 15%，从预训练第一天起就在配比中（6.6 节）；原始网页只占剩下的一部分。

MAI-Thinking-1：把唯一量与重复次数一起公开

MAI-Thinking-1 的 30T 配方，是目前公开得最完整的一份 [21]：

来源	唯一 token	训练 token	占比	平均重复
代码	7.4T	16.4T	54.6%	2.22
STEM	2.2T	4.7T	15.8%	2.17
网页文本	8.1T	4.5T	14.9%	0.55
数学	0.3T	1.6T	5.4%	5.28
PDF	2.7T	1.4T	4.7%	0.53
书籍与期刊	0.6T	0.9T	3.1%	1.65
其他多语言	8.1T	0.5T	1.6%	0.06

表中各项的唯一量相加约 29.4T，与报告给出的全部唯一量 29.2T 略有出入，来自分项的舍入；平均重复次数是报告按未舍入的数据算出的，与表中两列直接相除会有小的差别。

这张表可以从两个方向读。一是重复：代码占了一半以上的训练 token，每个代码 token 平均被看到两次多；数学的唯一量只有 0.3T，却被重复了 5 次多，已经接近该报告设定的 8 次上限。二是未用尽：网页、PDF 与多语言数据有大量唯一 token 根本没被用完，多语言 8.1T 只用了 0.5T。

报告另外给出一组目标权重：代码 50%、STEM 17.5%、数学 17.5%、通用知识 10%、多语言 5%。从数字上看，它与上表按来源统计的 token 占比不是同一个口径，不能逐项对应：数学能力的训练信号不只来自“数学”这一个来源，STEM 与代码数据中也有大量数学内容，而数学来源本身的唯一量只有 0.3T，按 17.5% 的 token 占比去采会远远超出 8 次上限。两组数字指向同一个结论：这是一个为推理与代码能力服务的配比，在其他能力上主动做了让步。

Olmo 3 与 Llama 3

Olmo 3 的 Dolma 3 Mix 共 5.93T token，其中 Common Crawl 占 76.1%、olmOCR 科学 PDF 占 13.6%、Stack-Edu 代码占 6.89%、FineMath 占 2.56% [13]。整个数据池有 9.31T，配比只用了其中的 5.93T：哪些唯一数据被用上，由配比与重复上限决定。

Llama 3 先用分类器给网页打上知识类别，再对过度代表的类别（如娱乐、艺术）降采样，最终按通

用知识 50%、数学与推理 25%、代码 17%、多语言 8% 配比 [6]。从 Llama 3 到 MAI-Thinking-1，代码的比例从 17% 升到了 55%。两年间，配比的重心从“通用知识”整体移向了“推理与代码”。

5.4 代理模型群与回归配比

人工设定的配比依赖直觉，而 RegMix 发现，直觉在这里并不可靠：网页语料与下游表现的相关性，比 Wikipedia 等“公认高质量”的来源更强 [65]。

从 DoReMi 到 Data Mixing Laws

DoReMi 用 280M 的代理模型，以 Group DRO 最小化各域相对参考模型的超额损失，学出域权重；迁移到 8B 模型后，达到基线精度所需的步数减少 2.6 倍，few-shot 平均高约 6.5 个点 [66]。

RegMix 换了一个思路：训练 512 个 1M 参数的代理模型，每个 1B token，直接回归“配比 $\rightarrow$ 损失”的关系，再取预测最优的配比。在 1B 模型、25B token 上，它胜过了 64 个候选配比；在 7B、100B token 规模上胜过人工配比，而计算量只有 DoReMi 的约 10% [65]。

Data Mixing Laws 则给出了一个显式的函数形式：拟合 $L(\mathbf{r}) = c + k\exp\left(\sum_j t_j r_j\right)$，其中 $\mathbf{r}$ 是各域的配比，c、k、t_j 是拟合参数，再与步数、规模的规模律嵌套使用。在 1B 模型、100B token 上，优化后的配比相当于默认配比多训练 48% 的步数 [67]。

Olmo 3 的约束配比

Olmo 3 把这条路线工程化了 [13]：

1. 训练一群 30M 参数的代理模型，每个 3B token（约 5 倍 Chinchilla）。配比从以自然分布为中心的 Dirichlet 分布中采样，代理数约为域数的 5 倍。
2. 对每个评测任务，拟合一个“配比 $\rightarrow$ BPB”的广义线性模型。
3. 在约束下最小化预测的平均 BPB：任何域的重复都不能超过约 4–7 次。

在网页主题配比上，1B 模型、5 倍 Chinchilla 的设置下，平均 BPB 改善了 0.056，最大改善 0.209；但 54 个任务中有 13 个变差，只是没有一个超过 0.035。配比优化是在任务之间做取舍，而不是全面提升，报告中写出变差的任务，与写出提升同样重要。

Olmix：把配置定下来

Olmix 系统检验了离线配比流程中的七个设计选择 [68]，推荐的配置是：30M 参数的代理模型，约 $3(m+1)$ 个代理（m 为域数），每个任务一个 log-linear 回归，在优化中直接施加重复约束，并做带 KL 正则的精确求解。学到的配比只用约三分之一的训练步数，就达到了自然分布配比的最终表现，相当于 3.05 倍的数据效率。

一个完整的算例

教学假设：只有网页与代码两个域，配比记为 $(r, 1-r)$，r 是网页的比例。用 6 个代理模型（$r = 0.5$、0.6、0.7、0.8、0.9、0.95）测得平均 BPB，拟合出

$$\mathrm{BPB}(r) = 0.80 + 0.30\,(r - 0.72)^2.$$

无约束的最优点是 $r = 0.72$。如果代码只有 200B 唯一 token、总预算为 2T、重复上限为 4 次，代码最多能分到 800B，即 $1 - r \le 0.4$，这个约束不起作用，最优仍是 $r = 0.72$。若总预算增加到 4T，同样的上限要求 $1 - r \le 0.2$，最优点就被推到边界 $r = 0.8$，预测的 BPB 为 $0.80 + 0.30{\times}0.0064 \approx 0.802$，代价很小。

这说明重复约束在大预算下才开始起作用。代理阶段训练的 token 太少，看不到这一点，所以必须由优化器显式施加。

5.5 配比在开发过程中的复用

真实开发中，数据集会不断增加、删除、拆分、修订。每次变化都重跑完整的代理群，成本太高。

分层冻结

Olmo 3 分三轮优化配比，每一轮都把上一轮已优化好的部分冻结成一个“虚拟域”[13]：第一轮优化网页内部的主题配比，同时决定各来源的比例；第二轮把网页固定在 75%、Stack-Edu 固定在 25%，优化代码语言之间的配比；第三轮优化 PDF 各主题之间的配比。每一轮只有少数几个自由变量，需要的代理数随之减少。

Olmix 的配比复用

数据域变化时，Olmix 保持不受影响的域之间的原有比例，把它们合并成一个整体，只重新计算这个整体的权重以及受影响的域 [68]。在 5 次模拟的域更新之后（最终 64 个域，1B 模型，100B token），完全复用达到了完全重算 95% 的收益（比自然分布高 11.6%），代理运行次数从 832 次降到 216 次，少了 74%；部分复用，即额外重算那些与变化域影响同一批任务的域，达到 98% 的收益，代理运行次数少 67%。

复用成立的前提，是未受影响的域之间的相对比例确实不受新域的影响。新加入的域如果与已有域高度重叠，例如一份新的 STEM PDF 与原有的 STEM 网页覆盖同一批主题，就应把那几个域一起放回优化，而不是冻结。

5.6 多模态联合预训练的配比

当视觉数据从“后接阶段”变成预训练配比中常驻的一个桶，问题就变成：什么时候加，加多少。

Kimi K2.5：早注入、低比例

Kimi K2.5 在 K2 接近训练结束的 checkpoint 上，继续训练约 15T 的视觉-文本混合 token [18]。消融实验固定视觉与文本的总预算，只改变注入的时机与比例：

方案	视觉: 文本	视觉推理	OCR	文本知识	文本推理
早注入（训练 0% 处）	10:90	43.8	65.7	45.5	58.5
中注入（50% 处）	20:80	40.7	64.1	43.9	58.6
晚注入（80% 处）	50:50	39.0	61.5	43.1	57.8

早注入、低视觉比例的方案，在视觉指标与文本指标上同时最好。一种可能的解释是：晚注入时，模型要在短时间内吸收大量视觉数据，文本能力被挤压。

读这张表时要注意，实验固定了视觉总预算，所以“晚注入”必然对应“高比例”，时机与比例的效应并没有被分开。要单独回答“时机”的问题，需要补做固定比例、只改时机的实验。

Qwen3.5 从预训练一开始就采用早融合，在混合数据上的训练吞吐接近纯文本 [8]。Kimi K3 延续了这一路线，并在视觉数据中加入了代码与渲染结果配对的程序化数据（11.4 节）[14]。

5.7 计算最优与过训练

Chinchilla 给出的最优点约为每参数 20 个 token，而当前模型普遍远超这个点。

推理成本改变最优点

Llama 3 8B 训练了 15T token，约每参数 1900 个；Qwen3 各尺寸都用约 36T token 预训练 [28]，0.6B 模型约每参数 6 万个，即便是 235B-A22B 的 MoE，按 22B 激活参数算也有约每参数 1600 个。原因是部署阶段的推理成本：小模型多训练一些，换来更低的推理开销，在“训练 + 推理”的总成本上更优 [69]。

教学假设：达到同一损失，有两个方案。

项	A：14B 模型训练 3T token	B：7B 模型训练 9T token
训练计算	2.52×10^{23}	3.78×10^{23}
每 token 推理计算	2.8×10^{10}	1.4×10^{10}

两者的训练计算差 1.26×10^{23}，每 token 的推理计算差 1.4×10^{10}。部署的推理量超过约 9.0×10^{12} token 之后，B 方案的总成本更低。推理量估得不准时，这个结论也会随之改变。

过训练的前提

1.6 节列出的 2026 年前沿模型，预训练量都在 20–36T token 之间，远超各自参数规模的计算最优点。过训练的前提是唯一数据足够，否则多出来的 token 只能靠重复补齐，这就引出了 5.8 节。

5.8 数据受限：重复与有效数据量

唯一数据不够时，重复不可避免。问题是：重复多少次之后，收益就不值得了。

有效数据量的衰减

Muennighoff 等人在 400 次训练（最大 9B 参数、900B token）中，拟合出重复数据价值的衰减形式 [70]：

$$D' = U + U\,R^*\left(1 - e^{-R/R^*}\right), \qquad R^* \approx 15.4,$$

其中 U 为唯一 token 数，R 为额外的重复次数（epoch $-$ 1），D' 为等效的唯一数据量。

教学算例，$U = 100$B：训练 4 个 epoch（名义 400B），有效 373B，约 93%；训练 16 个 epoch（名义 1.6T），有效 1.06T，约 66%；训练 40 个 epoch（名义 4T），有效 1.52T，约 38%。有效量的上限为 $U(1 + R^*) = 1.64$T。

论文的结论是：约 4 个 epoch 以内，重复数据与唯一数据几乎等价，之后收益递减，最终趋近于零。Olmo 3 的 4–7 次上限与 MAI 的 8 次上限都落在这一区间附近；MAI 的数学数据重复 5.28 次，正处在收益开始递减的地方（5.3 节）。

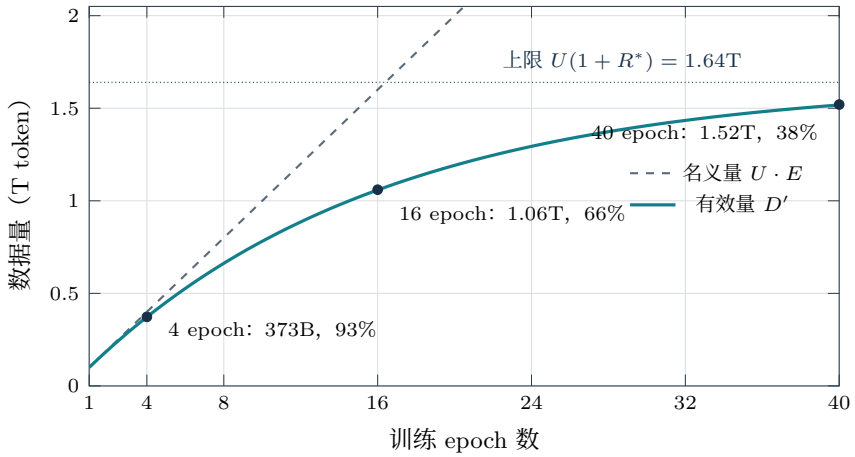


图 5.2　重复数据的有效量。读图：虚线是把重复当作新数据的名义量，实线是拟合的有效量 $D'(U = 100\text{B}, R^* \approx 15.4)$。约 4 个 epoch 以内两条线几乎重合，之后有效量趋近上限 $U(1 + R^*) = 1.64$T。[70]

重复的替代方案

唯一数据不够时，有三条替代路线。第一是改写，为同一知识提供多种表述（4.4 节 Kimi K2 的 SimpleQA 对照）。第二是放宽过滤，把次优数据纳入训练：Muennighoff 等人发现，数据受限时去掉部分常用过滤器反而有帮助 [70]。第三是回收被丢弃的数据：REWIRE 混入的改写数据中，约 82% 来自原本会被过滤掉的文档（6.2 节）。

此外还有一条底线：2026 年关于 AI 生成网页文本的研究建议，人类数据不足时，宁可重复人类文本，也不要先加入来源不明的 AI 文本（3.6 节）[23]。

5.9　领域饱和与规模上的排名反转

到这里，配比方法都默认一个前提：小规模上找到的最优配比，在大规模上依然最优。这个前提并不总是成立。

不同领域的饱和点不同

daVinci-LLM 观察到，通用知识约在 1T token 时趋于饱和，代码与科学一直到约 4T 仍在提升，并且代码与科学的均衡配比（30/30）优于偏重代码（40/20）[41]。

当网页、代码、科学这些格式都饱和之后，继续微调它们的比例就不再有效。daVinci 在第二阶段加入结构化 QA 数据，突破了这个平台期；Nemotron 3 Ultra 加入 SFT 风格数据与合成选择题，用的也是格式这一杠杆：同样的信息，换一种呈现格式，就能提供新的学习信号（第六章）。

排名在大规模上反转

MAI-Thinking-1 报告了一个反例 [21]：在小规模上，偏重 STEM 的配比胜过偏重代码的配比；放大到 23B 激活参数之后，结论反转了。作者把原因归于 STEM 来源的多样性不足、模糊重复较多：小模型学一遍就够了，大模型则很快把它们“吃完”，开始受困于重复。他们的配比在约 2.8 倍于全局配比搜索的计算量上做了验证。

这与 5.4 节算例中“重复约束在大预算下才起作用”是同一个机制：代理规模看不到重复。配比结论外推之前，至少要检查两件事：每个域在目标预算下的重复次数，以及每个域的内部多样性，例如去重簇的数量与大小分布。第十五章会从实验方法的角度再讨论这类外推失效。

本章思考与进阶练习

5.1 温度采样与重复上限

题目 沿用 5.1 节的例子（$\tau = 3$，总预算 1T），给每个语言设 4 次重复上限，超出的预算按 $\tau = 3$ 的比例返还给其他语言。斯瓦希里语最终得到多少 token?

解题思路 斯瓦希里语截断到 4B，返还 60B。按英语与中文 10 : 4.64 的权重分配，英语加 41B、中文加 19B，最终英语 680B，中文 316B（重复 3.16 次），斯瓦希里语 4B。要让小语种得到更多训练，只能扩充它的唯一数据。

5.2 有效数据量

题目 某领域有 200B 唯一 token，计划训练 10 个 epoch。按 Muennighoff 公式，有效数据量是多少？如果改成“5 个改写版本 × 各训 2 遍”，并假设改写版本之间与唯一数据等价，有效数据量又是多少?

解题思路 原方案 $R = 9$，$D' \approx 200 + 3080 \times 0.443 \approx 1.56$T，有效比例 78%。改写方案的唯一量为 1T、$R = 1$，$D' \approx 1.97$T。改写版本之间并不完全独立，真实值介于两者之间。

5.3 进阶：配比外推

题目 你用 30M 的代理群在 3B token 上得到最优配比，其中某个小众高质量域的权重是 8%。为什么这个权重放到 30B 模型、10T token 上可能并不合适?

解题思路 代理阶段该域的重复次数很低，看不到重复的损害；到 10T 时，8% 意味着 800B token，可能远超这个域的唯一量。大模型还会更快吃完低多样性的来源，即 MAI 的排名反转。修正方法有三：按目标预算施加重复上限；用两到三个代理规模观察配比的漂移趋势；在目标规模上用退火实验做最终校验。

第六章　合成预训练数据

第五章的结论是：唯一数据有上限，重复的收益在约 4 个 epoch 后开始递减。对此，2026 年的各家给出了两种相反的回答：Nemotron 3 Ultra 让合成改写占到配比的三分之一，成为第一大类；MAI-Thinking-1 与 MiniMax 的 M2 系列则在预训练中完全不用合成数据。本章先建立合成数据的统一视图，再依次讨论网页改写与它的组织方式、教科书与题目格式、把 SFT 风格的数据前移到预训练，以及合成比例的规模律，最后回到两条路线的分歧与风险。后训练阶段的合成数据（指令、推理轨迹、agent 轨迹）在第八、十章讨论。

6.1　统一视图

合成数据的质量，由“种子、生成器、prompt、过滤器”四个旋钮共同决定。每项研究都是固定其中几个，去调剩下的。

四个旋钮

$$x_{\text{syn}} \sim G_\phi\left(\cdot \mid \text{prompt}_k,\ s\right), \qquad s \sim D_{\text{seed}}, \qquad \text{保留 iff } \mathcal{F}(x_{\text{syn}}, s) = 1.$$

种子 s 来自数据集 D_{seed}，可以是高质量原文，也可以是原本会被丢弃的低质量文档；生成器 G_ϕ 决定改写能力的上限；prompt 集 $\{\text{prompt}_k\}$ 决定风格与格式的多样性；过滤器 $\mathcal{F}$ 把合成结果与种子比较，检查忠实度与质量，并做去污染。

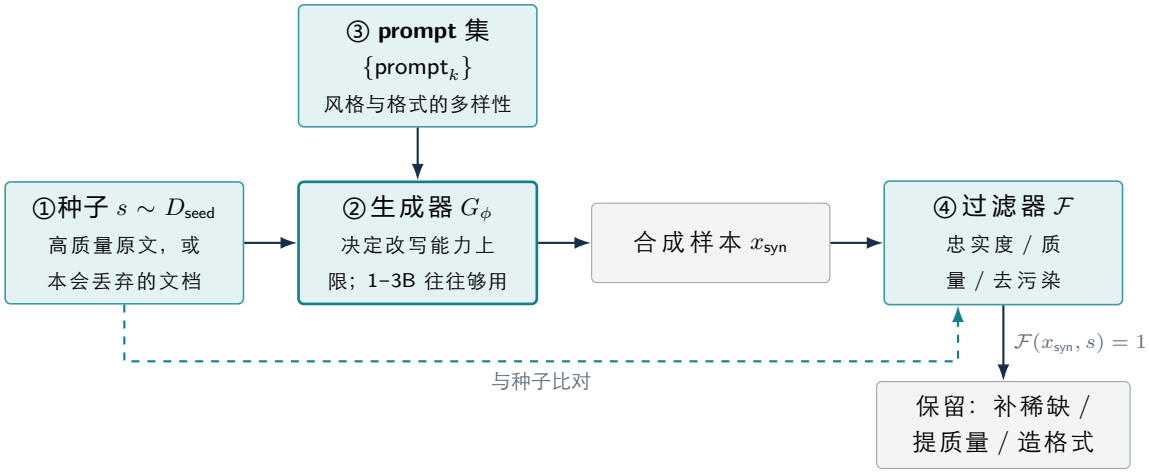


图 6.1　合成数据的四个旋钮。读图：种子提供信息，生成器与 prompt 决定怎样改写，过滤器把合成结果与种子比对后决定是否保留。每项研究都是固定其中几个旋钮，去调剩下的。

合成数据有三种用途。补稀缺：唯一数据不够时，用改写替代重复。提质量：原文噪声大、组织差，改写后更容易学。造格式：把同样的知识写成问答、选择题、表格，提供新的学习信号。

生成器不必很大

三项独立研究给出了一致的结论。FinePhrase 在受控实验中生成了超过 1T token，发现生成器超过 1B 参数后没有额外收益 [71]。BeyondWeb 的改写器从 1B 增大到 3B 提升 1.5 个点，从 3B 到 8B 只提升 0.4 个点，不同模型族之间的差距约 1 个点，而且改写器自身在通用基准上的分数，预测不了它产出数据的质量 [72]。Demystifying Synthetic Data 发现，Llama-3-8B 作为生成器优于 3B，但 70B 并不比 8B 更好，在高质量改写风格上甚至更差 [73]。

原因在于，改写任务本身不难，信息主要来自种子；大生成器的价值在需要推理的生成上（L5 及以上）。这直接决定了成本：用 1–3B 的生成器，万亿 token 级的合成在经济上可行，FinePhrase 报告的生成成本比已有方法低约 30 倍。

6.2 网页改写

改写是最接近原文的合成：信息来自种子，生成器只改变表述。

WRAP：风格改写加原文混合

WRAP 用指令模型把 C4 文档改写成“像维基百科”“问答对”等风格，再与原文混合训练，预训练速度提高约 3 倍，在 Pile 各子集上困惑度平均改善超过 10% [74]。关键在于混合：只用改写数据，会丢掉网页的风格多样性；原文与改写放在一起，二者互补。

Nemotron-CC：按质量档选择合成方式

Nemotron-CC 用 Mistral NeMo 12B，按第三章的质量档采用不同的合成方式 [20]。低质量文档用维基风格改写来降噪、纠错，产出 336B token；高质量文档用四种 prompt（多样化问答对、精炼、知识抽取、知识列表），产出约 1.5T token；中间档不做合成。两部分合计约 1.8T 合成 token。

低质量的先救回来，高质量的再充分利用：同一份高质量内容被写成多种格式，每种格式提供一种学习信号。

REWIRE：回收被丢弃的文档

REWIRE 只处理原本会被质量过滤丢掉的文档 [75]。Llama-3.3-70B-Instruct 先推理出文档的目的，以及实现这个目的需要哪些步骤，再写出改进后的版本；最后用 fastText 分类器保留改写结果中最好的 10%，与高质量原文混合。

在 DCLM 的 22 项 CORE 指标上，相对“只用高质量原文”，1B、3B、7B 模型分别提升 1.0、1.3、2.5 个点；1B 规模下，这种混合（29.9）优于把原文数据量翻倍（29.4）。混入的合成文本中，81.7% 来自原本会被丢弃的文档。REWIRE 把“过滤阈值以下”的数据从浪费变成了资源，直接缓解了数据墙。

Kimi：多风格改写、分块与忠实度

Kimi K2 的知识类改写有三个要点 [60]。它借鉴 WRAP，用多风格、多视角的 prompt，在保持事实的前提下改变表述；长文档切段后逐段改写，改写时把前文作为上下文，再拼回整篇，避免输出长度限制造成的信息丢失；每一段都检查改写结果与原文的语义一致性。数学数据则改写成“学习笔记”的风格（参照 SwallowMath [76]），并把其他语言的高质量数学材料翻译成英文。

2026 年的 Kimi K3 沿用了这套方法，对知识与数学语料做同样的改写 [14]。4.4 节的 SimpleQA 对照说明了它为什么有效：10 个改写版本各训练一遍（28.94），优于原文训练十遍（23.76）。

FinePhrase：结构化格式最有效

FinePhrase 系统比较了 prompt、生成器与源数据 [71]。结构化的输出格式（表格、数学题、FAQ、教程）稳定优于精选网页基线，也优于之前的合成方法；生成器超过 1B 没有额外收益；用于混合的原始数据的选择，显著影响下游结果。最终发布的 FinePhrase 有 486B token，并公开了全部 prompt 与生成框架。

把这几项工作放在一起看，改写的收益主要由两件事决定：选哪些文档做种子（Nemotron-CC 按质量档、REWIRE 专挑被丢弃的），以及把它们写成什么格式（FinePhrase 的结构化格式）。生成器只要过了 1B 左右的门槛，就不再是主要变量。

6.3 改写的组织方式：megadocs

改写得到的多个版本，最常见的用法是各自当作一篇独立文档打乱训练。但对同一篇源文档改写 32 次，这 32 个版本之间的关系就丢失了。

两种 megadoc

Marin 项目在一个数据受限的设置下（300M 模型、200M 真实 token），比较了三种组织方式 [77]。简单改写把每个改写版本当作独立文档，32 次生成时数据效率 1.48 倍，基准平均提升约 5%。拼接改写（stitched）把同一源文档的所有改写拼成一篇长文，原文放在最后，数据效率 1.64 倍，基准平均提升约 6%。潜在思考（latent thoughts）把原文切段，在段与段之间插入生成的推理，拉长成一篇长文，数据效率 1.80 倍，基准平均提升约 9%。

简单改写的收益随生成次数增加而趋于平台，两种 megadoc 的收益仍在增长。在长上下文的 arXiv 论文上，megadoc 把损失改善了 0.14–0.19，比在独立同分布数据上的改善更大。

收益从哪里来

论文对收益做了诚实的拆解：megadoc 的部分优势，来自它允许训练更久，即真实数据可以多训几个 epoch，合成数据的混合比例可以更高。控制这些因素之后，megadoc 仍然有帮助，但不再随生成次数更好地扩展。比较合成方案时，要把“数据本身更好”与“允许训练更久”分开（15.3 节）。

6.4 教科书与书级组织

改写保持原文的信息，教科书式合成则让生成器按主题重新组织知识。后者更自由，也更容易偏离真实分布。

Phi：教科书质量的数据

phi-1 的训练数据由两部分组成：约 6B token 按“教科书质量”过滤的网页代码，加上约 1B token 由 GPT-3.5 生成的教科书与练习 [78]。1.3B 模型在 HumanEval 上达到了 50.6%。它证明了高信息

密度的合成数据可以大幅压缩训练预算，也开启了教科书式合成这条路线。

多样性从哪里来

教科书合成最大的风险，是主题与风格的同质化。扩大多样性有两种思路。Cosmopedia 从种子入手：用 Mixtral-8x7B-Instruct 生成约 25B token、3000 万篇文本，种子来自对网页聚类得到的主题以及精选的教育资源，prompt 中再指定受众（儿童、大学生、研究者）与体裁（教科书、博客、故事）[79]。Persona Hub 从视角入手：从网页中提取约 10 亿个“人物设定”，把“从某个人物的视角生成”作为 prompt 的一个维度，使生成内容的分布接近真实人群的多样性 [80]。

另一条思路是在文档之间做组织。2026 年的一项工作把合成教材按整本书的结构组织起来，作为中期训练数据，报告效果优于逐篇改写 [81]。这与 megadoc 的思路相通：学习信号不只在单篇文档里，也在文档之间的组织结构里。

比例宜低

几项研究都指向同一个方向。小规模下，教科书类数据的最优比例常常低于 5%，并随规模上升，但始终低于改写类数据 [73]；33% 教科书加 67% 网页，在约 20B token 之后优于纯网页，67% 教科书则不如纯网页 [73]；单一教科书风格的合成数据（如 Cosmopedia）随训练延长会进入平台期，多样化的生成则持续提升 [72]。

教科书类数据适合作为“浓缩剂”，不适合作为主体数据。

6.5 题目格式、事实数据与小语料扩充

前面几节都是把文本改写成另一种文本。本节讨论另一种形态：把知识写成题目，或者从很小的语料中扩充出大量训练数据。

从公开训练集生成题目

Nemotron 3 Ultra 从公开数据集的训练集生成选择题与生成式题目，并显式排除测试集 [17]。在 100B token 的消融中，MMLU-Pro 从 64.8 升到 66.6，GPQA 从 30.8 升到 41.9，代码从 73.2 升到 75.1。GPQA 提升 11 个点，说明题目格式本身就是一种学习信号：模型学会了在选项之间做判别推理。

事实型数据：在训练末段注入

Nemotron 用 Qwen3-30B-A3B-Instruct 从 FineWiki 条目生成事实型数据，在 Nano 模型训练的最后 100B token 注入，SimpleQA 从 40.24 升到 50.16 [17]。需要注意，这个评测被转换成了选择题形式，与标准 SimpleQA 不可直接比较。选择在末段注入，是因为事实性知识在学习率衰减段最容易被“定型”（7.1 节）。

EntiGraph：小语料的知识扩充

当某个领域只有很小的语料时（例如一批内部文档），直接继续预训练学不到多少。EntiGraph 先从文档中抽取实体，再让 LLM 生成实体之间关系的描述，合成大量多样化的文本 [82]。它把 130 万

token 的源语料扩充为 4.55 亿 token，继续预训练后，模型回答相关问题的能力随合成 token 数近似对数线性增长。实体组合提供了单纯改写无法提供的多样性。

程序化的多模态合成

合成数据不只限于文本。Kimi K3 把“代码与它的渲染结果”配对，作为多模态合成数据 [14]：代码就是图像的精确描述，图文对齐天然正确。这类数据在 11.4 节详细讨论。

6.6 SFT 风格数据进入预训练

前面讨论的合成数据都还是“知识”。把对话、推理这类 SFT 风格的数据前移到预训练，改变的是模型学习的格式。

推理数据要早放

NVIDIA 在一个 8B 混合架构模型上从头预训练 1T token，比较推理数据放在预训练还是 SFT [83]。预训练中加入推理数据，平均提升约 19%；同样的数据只在 SFT 阶段使用，无法追平，SFT 之后预训练组仍领先 9.3%，经过 RL 后领先约 18.6%，AIME 高 39.3%。

两个阶段对数据的偏好也不同。预训练阶段更看重多样性，大而多样的推理数据比小而精的高约 9%；SFT 阶段更看重质量，高质量数据带来约 15% 的平均提升，而用混合质量的数据扩大 SFT 规模，平均没有收益，数学还下降约 5%。推理能力的容量在预训练阶段打下基础，后训练只能在已有的容量上调整。

产业落地与风险

Nemotron 3 Ultra 第一阶段配比中，SFT 风格的 STEM、代码与通用数据合计约 14.7% [17]，这些数据从预训练第一天起就在配比中。

风险有三类。第一是格式塌缩：base 模型过早变成问答机，续写能力下降，缓解的办法是控制比例，并在阶段之间逐步提高（daVinci 的 30% 到 70%，5.2 节）。第二是评测污染：SFT 数据常常从公开基准的训练集派生，需要对测试集做 embedding 去污染（4.5 节）。第三是 base 模型的评测失真：few-shot 评测的提升，部分来自对格式的熟悉，应同时报告 BPB 与生成式评测。

6.7 合成比例与规模律

合成数据加多少，没有普适的答案。最系统的两项研究，都把答案落在“约三分之一”附近，并都指出它会随规模变化。

1000 多个模型的系统研究

Demystifying Synthetic Data 训练了 1000 多个模型（最大 3B 参数、最多 200B token），比较了维基风格改写、问答风格改写、教科书三类合成数据与 CC 原文 [73]。纯改写数据并不比原文学得更快；约 1/3 改写加 2/3 原文，在较大的数据预算下，达到同样的验证损失快 5–10 倍。改写类的最优比例收敛到约 30%，问答类的最优比例则从小规模的约 50% 降到大规模的约 30%。模型越大，合成数据

的优势越小，纯改写数据在大模型上落后于原文。多数含合成数据的混合，预测的不可约损失低于纯原文，其中 33% 维基风格改写加 67% 原文最低。

为什么最优比例会随规模下降？一种与这些结果一致的解释是：改写数据的信息全部来自种子，表述却更规整，unigram 多样性略低于原文（6.8 节）。小模型容量有限，更受益于规整、高密度的文本；大模型能从嘈杂但多样的原文中学到更多，合成数据带来的“易学”优势就被摊薄了，留下的只有多样性上的损失。所以合成比例应在两三个规模上分别验证，而不是从小模型直接外推。

BeyondWeb：多样性决定能否持续扩展

在 14 项基准上，8B 模型的平均分 [72]：

训练数据	平均分	达到 RedPajama 训练 180B token 水平所需的 token
BeyondWeb	63.7	23.2B（7.7 倍加速）
Nemotron-Synth	61.1	66.2B（BeyondWeb 相对它 2.7 倍加速）
Cosmopedia	58.6	—
RedPajama 原文	56.6	180B

3B 的 BeyondWeb 模型（60.8）胜过 8B 的 Cosmopedia 模型（58.6）。研究还有几个发现：单纯的摘要就几乎能追平 Cosmopedia，说明“信息密度”是机制之一，但不是全部；简单续写只比重复数据略好；1B 模型训练 1T token（约 50 倍 Chinchilla）时，其他基线开始过拟合，BeyondWeb 仍在提升。作者的总结是没有银弹，需要同时调好源数据、风格、多样性与生成器。

产业中的比例

产业配方落在同一区间。Nemotron-CC 的合成数据约 1.8T（6.2 节两部分之和），约占其 6.3T 总量的 30%；Nemotron 3 Ultra 第一阶段的合成改写合计约 33%（5.3 节）。另一端是 MAI-Thinking-1，比例为零（6.8 节）。

“约三分之一”在多个独立来源中反复出现，但它依赖合成类型与模型规模，不是一个常数。

6.8 两条路线与风险诊断

合成数据的支持者看到的是数据墙，反对者看到的是分布漂移。2026 年，两种立场都有了大规模的产业实践。

不用合成数据的一方

MAI-Thinking-1 的预训练完全不使用语言模型生成的文本，并主动识别、删除抓取数据中的 AI 生成内容；在后训练阶段，它也拒绝从第三方模型蒸馏，只做自蒸馏 [21]。据第三方整理（非官方），MiniMax 的 M2 系列同样在预训练中避免使用合成数据 [35]。

这条路线的论据，来自 3.6 节那项关于 AI 生成网页文本的研究 [23]：AI 文本只在每参数少于约 10 个人类 token 时有帮助，到了计算最优的约 20 个之后帮助消失，再往上会损害在人类文本上的表现。

但两项研究的对象不同。那项研究测量的是来源不明、混在网页中的“野生”AI 文本；本章前几节讨论的，是以真实文档为种子、受控生成、带忠实度校验的合成数据。前者没有种子锚定，也没有质量控制；后者的信息主要来自种子。两个结论并不矛盾，它们一起划出了边界：没有种子锚定、来源不明的机器文本应当过滤掉；有种子锚定、经过校验的改写可以使用，但比例要控制。

模型坍缩

Shumailov 等人证明，反复用模型生成的数据替换真实数据，分布的尾部会逐代消失，造成不可逆的退化 [84]。Gerstgrasser 等人补充说，只要把合成数据累积在真实数据之上而不是替换，测试误差就有上界 [85]。在预训练实践中，改写数据在已测试的规模下没有出现退化，教科书类的混合却出现了与坍缩预测一致的模式 [73]：改写锚定在真实文本上，教科书则更多依赖生成器自身的分布。

一张诊断清单

每批合成数据入库前，至少检查五项：

1. **忠实度**：数字与实体的保留率、新实体率、长度比；
2. **多样性**：风格分布、n-gram 重复率、embedding 聚类的熵，因为合成数据会让 unigram 多样性略微下降 [73]；
3. **污染**：生成器可能记得评测题，生成之后要对评测集做 embedding 检索；
4. **元信息**：生成器版本、`prompt_id` 与种子文档 ID 齐全；
5. **验证损失分开报告**：在人类文本与合成文本上分别报告验证损失，这是 AI 文本研究给出的最直接的建议 [23]。

本章思考与进阶练习

6.1 生成器选型

题目 要合成 1T token 的改写数据，候选生成器为 1.7B、8B、70B，单卡生成吞吐分别为每秒 4 万、1 万、1500 token。按每 GPU 时 2 美元，计算三者的生成成本，并给出选择。

解题思路 三者分别需要约 6.9 千、2.8 万、18.5 万 GPU 时，成本约 1.4 万、5.6 万、37 万美元。现有证据显示，生成器超过 1–3B 后收益很小，70B 甚至可能更差。应在 1.7B 与 8B 之间做小规模消融，选定后用小模型生成，把省下的预算用在更多样的 prompt 与更多的源文档上。

6.2 拆解 megadoc 的收益

题目 你的实验显示 megadoc 比简单改写低 0.05 的损失。怎样确定收益来自“组织方式”本身?

解题思路 先控制三个混杂因素：总训练 token 与真实数据的 epoch 数相同，合成数据的混合比例相同，序列长度与打包方式相同。再加一组“同源改写放在同一序列、但不拼接”的对照，检验收益是否来自同源内容的共现。

6.3 进阶：两条路线如何取舍

题目 你的团队面临选择：A 路线预训练完全不用合成数据，B 路线用 30% 的种子锚定改写数据。哪些信息能帮你做出决定？

解题思路 先看唯一数据是否充足：按目标预算计算各来源的重复次数，人类数据已经需要重复 4 次以上时，B 路线的价值更大。再看模型规模：模型越大，合成数据的优势越小 [73]。还要看可审计性：B 路线需要完整的生成元信息与忠实度校验，否则会退化成“来源不明的机器文本”。最终用实验决定：在目标规模的代理上，对比两条路线在人类文本验证集与下游任务上的表现，而不是只看混合验证集。

第七章　中期训练、仓库级代码与长上下文数据

中期训练（mid-training）位于大规模预训练与后训练之间，规模在 0.1–3T token 之间，通常与学习率的衰减段重合，数据面向明确的目标能力。本章先讨论它的定位，以及怎样用退火实验评估数据的价值；再依次讨论推理轨迹、仓库级代码、agent 行为与长上下文四类目标数据。下一章进入后训练的 SFT 数据。

7.1　中期训练的定位

学习率的衰减段对数据最敏感，所以它既是放高质量数据的地方，也是测量数据价值最便宜的地方。

规模与目标

各家中期训练的规模差异很大。Olmo 3 的 Dolmino 只有 100B token [13]；GLM-5 按长度分三段，共约 1.55T [16]；MAI-Thinking-1 分两段，3.4T（64K 上下文）加 150B（256K 上下文）[21]；AgentFounder 的 agentic 中期训练约 300B [86]；OctoThinker 则先以恒定学习率训练 200B，再在三个分支上各衰减 20B [87]。

目标同样各异。Olmo 3 侧重数学、代码与思考轨迹，GLM-5 侧重仓库级代码与长上下文，MAI 的配比在中期训练中转向 35% STEM 与数学、55% 代码、10% 背景数据 [21]。2.4 节提到的开发者会话轨迹，据第三方整理（非官方），也是用在中期训练阶段 [35]。

为什么放在衰减段

WSD 学习率把训练分为预热、稳定与衰减三段，衰减段有两个性质。第一，学习率小，每个 token 被更“定型”地吸收，高质量数据的边际收益最大。第二，可以分支：从同一个稳定段的 checkpoint 出发，用不同的数据跑多个衰减分支，比较后再选。

OctoThinker 正是这样做的。它的结论是：中期训练的规模越大，后续 RL 的效果越好；中期训练在很大程度上决定了一个模型“适不适合做 RL”[87]。

用退火评估数据价值

完整的训练太贵，无法对每份候选数据做消融。退火实验把评估成本降到了完整训练的百分之一左右。Llama 3 取训练到 50% 的 8B 模型，在 40B token 上把学习率线性降到 0，候选数据占 30%、默认配比占 70% [6]；Olmo 3 的 microanneal 用 5B 候选数据加 5B 网页数据退火，与 10B 纯网页数据的退火对比 [13]。数据的价值定义为：

$$\Delta = \mathrm{score}\left(\mathrm{anneal}(D_{\mathrm{cand}} \oplus D_{\mathrm{base}})\right) - \mathrm{score}\left(\mathrm{anneal}(D'_{\mathrm{base}})\right),$$

其中两边的总 token 数相同，D'_{base} 用基线数据补足了候选数据那部分的 token。退火之所以有效，是因为衰减段对数据最敏感，小规模就能看到信号。

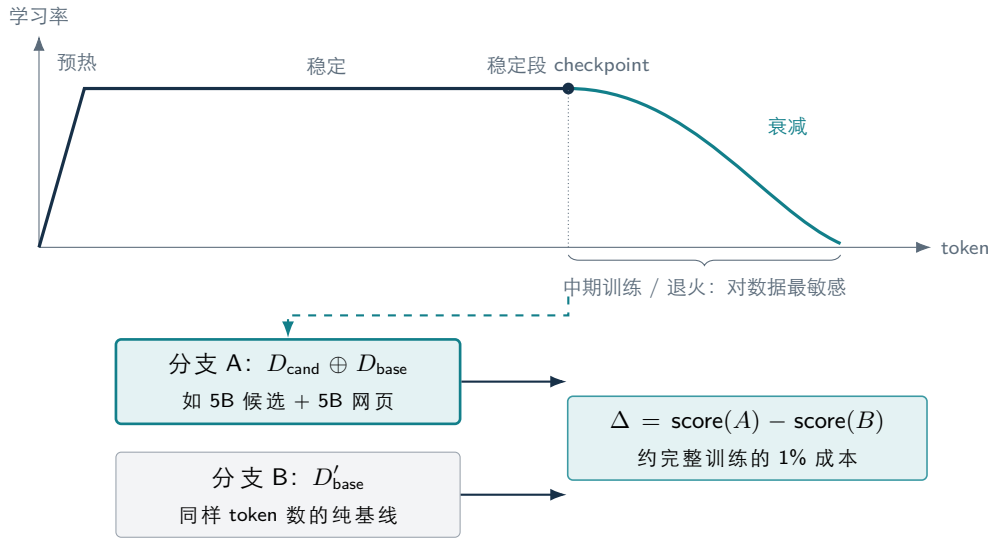


图 7.1　WSD 学习率与退火分支。读图：稳定段结束后，从同一个 checkpoint 出发跑多个衰减分支，分支之间只有数据不同，两者的分数差就是候选数据的价值 Δ。中期训练与 microanneal 利用的都是衰减段对数据最敏感这一点。[13, 87]

Olmo 3 还做了五轮完整的集成测试：用整份 100B 候选配比退火，再做 SFT 后评测 [13]。之所以评测 SFT 之后的模型，是因为有的中期训练数据对 base 模型的分数帮助不大，SFT 之后才显现，这与 6.6 节 front-loading 的“潜在效应”一致。

7.2　中期训练的数据构成

退火实验确定了每份数据的价值，下一步是把它们组合成一份配比。Olmo 3 的 Dolmino 是目前公开得最完整的一份。

Dolmino：100B 的完整配比

Olmo 3 从 2.19T token 的候选池中选出 99.95B token [13]：

类别	占比	主要来源
网页	27.4%	CC 高质量子集 22.4B，STEM 网页 5.0B
代码	20.0%	StackEdu FIM 10B，CraneCode 10B
合成数学	19.2%	Dolmino Math、CraneMath、MegaMatt、TinyMATH
合成 QA	13.9%	Reddit 转闪卡、Nemotron Synth QA、维基转阅读理解
思考轨迹	8.3%	QwQ、Llama-Nemotron、OpenThoughts2、Gemini 等
指令	6.1%	Flan，Tulu 3 SFT
PDF	5.0%	olmOCR 科学 PDF 高质量子集

这张表有三处值得读。网页仍是最大的单类：中期训练不是“只用精选数据”，保留足够的通用数据，才能避免能力遗忘。合成数据（数学、QA、思考轨迹）合计约 41%，中期训练是合成数据的主要用武之地（第六章）。指令与思考轨迹合计约 14%，base 模型在中期训练中就已经见过后训练的格式，SFT 阶段只需对齐，不必从头学。

MAI-Thinking-1：在中期训练中收紧来源

MAI-Thinking-1 在中期训练中对两类来源做了专门筛选 [21]。PDF 用基于 Bloom 认知分类法“分析”层级的启发式规则，只保留技术上正确、推理深度至少达到中等水平的文档；代码按仓库质量分为三档，每档使用不同的文件扩展名白名单，并做文件级的格式检查。它的 epoch 上限也按记忆率设定（4.4 节）。预训练阶段侧重覆盖，中期训练阶段侧重“推理深度”，筛选标准随阶段变严。

7.3 推理轨迹与元推理数据

中期训练的一个目标，是让后续的 RL 更容易找到奖励信号。推理轨迹与元推理数据就是为此准备的。

多教师的思考轨迹

Dolmino 的思考轨迹来自多个教师：QwQ、Llama-Nemotron、OpenThoughts2、Gemini 等 [13]。多教师一方面避免单一教师的风格被固化，另一方面让不同教师在各自擅长的领域发挥作用。

OctoThinker：什么样的数据让 RL 更有效

OctoThinker 研究的问题是：为什么 Llama 系模型的 RL 效果不如 Qwen，中期训练能否弥补 [87]。它的第一个发现是，数学语料的质量是关键：MegaMath-Web-Pro 同时提升了 base 模型与 RL 之后的表现，FineMath-4plus 则不行。第二，加入 QA 风格与长 CoT 数据能改善 RL 的结果，再加上指令数据，效果进一步放大；但长 CoT 有副作用，推理更深的同时回答冗长、RL 训练不稳定，数据格式需要谨慎处理。第三，中期训练越多，下游 RL 的表现越好。最终发布的 MegaMath-Web-Pro-Max 有 700 亿以上 token。

元推理与程序可验证数据

Dolmino 还包含两类小而专的数据 [13]：围绕推理过程本身的元推理数据，包括检查、规划、纠错；以及答案能被程序验证的问题。两者都是为 RL 铺路：模型在中期训练中先见过“自我检查”的模式与可验证任务的格式，RL 阶段才更容易找到奖励信号。

中期训练怎样改变 RL 的有效题比例

“更容易找到奖励信号”可以量化。10.2 节会推导，组大小为 G 时，一道通过率为 p 的题产生非零梯度的概率是 $P_{\text{info}}(p) = 1 - p^G - (1-p)^G$。中期训练不改变题库，改变的是 base 模型在这些题上的初始通过率。

数值例子（教学假设：$G = 8$，题库中 60% 的题起初通过率约 0.01）：不做推理类中期训练时，这 60% 的题 $P_{\text{info}} \approx 0.077$，几乎都在空转；中期训练把它们的通过率提到约 0.05 之后，$P_{\text{info}} \approx 0.337$，有效题数增加约 4.4 倍。

同一份 RL 题库、同样的 rollout 预算，起点模型不同，能用上的题就差了几倍。这就是 OctoThinker 观察到“中期训练越多，RL 越好”的一种机制解释：中期训练把大量题从“全错”区拉进了“有对有错”的区间。

7.4 仓库级代码数据

推理轨迹训练的是思考，仓库级代码训练的是“在真实工程中定位问题、写出修改”。

GLM-5 的构造

GLM-5 的中期训练，把 issue 与 PR 组织成仓库级序列 [16]：

```
[仓库相关源文件] → [GitHub issue 描述] → [commit diff] → [PR 描述与讨论] → [修改后的文件]
```

构造上有三点改进：放宽仓库过滤，纳入更多仓库，得到约 1000 万个 issue-PR 对；加强 issue 级的质量过滤，剔除描述不清或与改动无关的 issue；为每个 issue-PR 对检索更多相关的源文件，放进上下文。这部分数据共约 160B 唯一 token。序列的结构直接对应软件工程任务：读 issue、定位文件、写 patch，模型在中期训练中就已经见过成千上万次“问题 → 修复”的完整过程。

从 PR 到 RL 任务

同样的原料，也是第十章 RL 环境的来源。MAI-Thinking-1 从约 1.02 亿个 GitHub PR 中，筛出约 487 万个带关联 issue 的 PR，构建成可执行的软件工程任务 [21]。中期训练用它们的文本，RL 用它们的可执行版本，两处要使用同一份去污染与防泄漏规则。

FIM 与依赖排序

FIM（fill-in-the-middle）把一个文件切成前缀、中段、后缀三段，再重排成“先给两头、最后生成中间”的顺序。常见的 PSM 格式是：

```
<fim_prefix> 前缀 <fim_suffix> 后缀 <fim_middle> 中段
```

模型在看到前后文之后生成中段，学到的正是 IDE 里“在光标处插入”的能力；另一种 SPM 格式把后缀放在前面。数据侧有两个参数要定：多大比例的文件做 FIM 变换（常用 0.5），以及切分点按字符、按行还是按语法结构选。按语法结构切（例如整段函数体），中段更接近真实的补全请求。

Dolmino 包含 10B token 的 StackEdu FIM 数据，训练补全与插入能力 [13]。仓库级序列内部的文件按依赖的拓扑顺序排列，被调用的文件放在前面（2.4 节）。

泄漏风险

SWE 类评测（如 SWE-bench）的题目，就是真实仓库中的 issue 与 PR。所以仓库级数据必须按评测集的仓库与 issue 编号排除对应数据；对评测仓库只使用评测题创建时间之前的提交；并用 diff 内容的 n-gram 检查，确认测试补丁没有混进训练数据。

7.5 Agentic 中期训练

仓库级代码教会模型“改代码”。agentic 中期训练则更进一步：在后训练之前，就让模型学会多步决策与工具调用。

为什么要前移 agent 行为

只在后训练阶段教 agent 行为，模型要同时完成两件事：学会多步决策、工具调用这些新行为，以及对齐专家演示的具体格式。两件事互相干扰。Agentic CPT 的思路，是把第一件事前移到中期训练，让后训练只负责对齐 [86]。产业上，DeepSeek-V4 在中期训练加入 agentic 数据来增强代码能力 [7]，GLM-5 在后期阶段上采样合成的 agent 轨迹 [16]。

一阶动作合成

AgentFounder 的一阶动作合成（FAS）不调用任何工具，完全离线生成 [86]：

1. **知识转问题**：把网页文本转成以实体为键的陈述，组成一个开放世界的“记忆”；再采样实体与陈述，生成事实检索、数值计算、多跳推理、综合等类型的问题。
2. **规划动作**：让 LLM 写出问题分析与第一步动作（调用工具或直接回答）。同一份记忆会生成风格不同的多道问题以增加多样性，再由 LLM 评审剔除那些不太可能检索到所需知识的样本。
3. **推理动作**：把问题拆成子问题，先用模型的内部知识起草答案，再用映射过来的知识修正，只保留最终答案正确的样本。

FAS 只合成第一步，理由是第一步推理的质量与最终任务的完成率高度相关。

高阶动作合成

高阶动作合成（HAS）把已有的轨迹改造成“决策”训练数据：对轨迹中的每一步，生成 N 个替代的“思考 + 调用”，与原步骤混合打乱，得到 $N+1$ 个选项；训练文本写成“我选择选项 k”，后面接真实的环境返回；最后根据整条轨迹的结果，标注这次选择是否正确。这样就绕开了噪声很大的步级奖励，用整条轨迹的结果为每一步的选择打标签。

两阶段训练与规模

AgentFounder 分两阶段训练：先用约 200B token、32K 上下文，使用 FAS 与较短的 HAS 数据；再用约 100B token、128K 上下文，使用高质量的 HAS 数据。在 50B token 的消融中，加入第二阶段让平均 Pass@1 提升约 3.3%，原因是单阶段训练中长序列被截断，丢失了数据。

数据量与效果近似呈对数关系：前 15B token 的收益最大，约 3.8%；到 315B token 时，Pass@3 从 54.2% 升到 62.2%。AgentFounder-30B 在 BrowseComp-en 上达到 39.9%，在 HLE 上达到 31.5%。这些都是作者在自己的基线与评测设置下报告的数字，跨报告比较时要先对齐评测协议。另一个间接信号是，后续 SFT 的最终损失（0.7953）低于基线（0.8656），说明经过 agentic 中期训练的模型更容易被对齐。

7.6 自然长文档

agent 轨迹动辄几十万 token。要训练这样的长度，先要有足够多、真正长的文档。

来源与稀缺

ProLong 的系统实验表明，代码仓库与书籍是最好的长文档来源，但要与高质量的短文本混合使用 [88]。网页中超过 32K token 的文档极少，长文档必须专门收集；DeepSeek-V4 把重点放在科学论文与技术报告上 [7]。

Olmo 3 的长上下文数据池给出了这种稀缺的具体量级 [13]：整个池有 640B token，其中超过 8K token 的文档 2230 万篇，超过 32K token 的文档只有 450 万篇、共 380B token。7B 模型用 50B token、32B 模型用 100B token 完成扩展，支持 65K 上下文。

清洗与上采样

长文档的清洗有两处特殊之处：困惑度要分段计算，避免一段乱码拉低整篇；去重要包括文档内部的长片段重复（4.1 节的子串去重）。GLM-5 用困惑度、去重、长度三类过滤，并上采样知识密集的领域 [16]。Kimi K3 的长上下文数据经过了完整的清洗，包括精确与模糊去重、视频帧的感知哈希、启发式与分类器过滤，以及结构有效性检查；它在冷却阶段专门上采样真正长的文档与视频，避免短序列在批次中占据主导 [14]。

打包与注意力 mask

ProLong 发现，屏蔽跨文档的注意力，能同时改善短上下文与长上下文的表现 [88]。DeepSeek-V4 把来自不同来源的文档打包以减少截断，同时使用样本级注意力 mask（1.4 节）[7]。

7.7 合成长上下文数据

自然长文档的问题不只是少：在多数长文档中，后文真正需要依赖远处前文的位置很少，模型只看局部就能预测下一个 token。合成数据的目标，是制造“必须看远处”的依赖。

NExtLong：用硬负例拉开距离

NExtLong 用短文档构造长序列 [89]。它先把文档按段落切成最长 2048 token 的“元块”，再在去重后的语料上检索每个元块最相似的块作为干扰项，最后让元块在前、干扰项在后交错排列，把相关的内容推远。

在 Llama-3-8B、128K 上下文、约 4B token 的设置下，NExtLong 在 HELMET 与 RULER 上的平均分为 62.58，同样设置下 Quest 为 55.25，直接拼接为 52.85；512K 版本平均 65.76，高于用自然长文档训练的 ProLong-512K（60.34）；短文本任务基本不受影响。

EntropyLong：用不确定性定位依赖

EntropyLong 用模型自身的不确定性，找出需要远程信息的位置 [90]：

1. base 模型计算每个 token 的预测熵，超过“文档内均值加 2 倍标准差”的位置标为不确定；
2. 以不确定位置前后各 16 个词为查询，在 10 亿级文档中检索 32 个候选；
3. 把候选放到文档前面，重新计算该位置的熵，相对熵下降超过 0.4 才保留；
4. 把通过验证的上下文打乱顺序，放在原文之前。

用 10 万篇源文档生成约 4B token 的 128K 序列，RULER 平均 87.37，NExtLong 为 85.22，Quest 为 80.53；在 128K 长度上差距更大，分别为 81.26、77.99、60.11。去掉验证步骤之后，RULER 从 87.37 降到 85.82：只保留“确实降低了熵”的上下文，是收益的关键。

产业落地

GLM-5 受这两项工作启发，把相似的文本交错打包，以减轻“中间遗忘”，并在 200K 阶段加入少量多轮对话回忆数据 [16]。Kimi K3 把多模态文档与子任务打乱顺序后拼接，使每个任务只有在看完整个 1M token 上下文后才能解出 [14]。Nemotron 3 Ultra 的长上下文扩展阶段不用 RULER 风格的数据，SFT 阶段则使用 512K 的合成数据，覆盖多文档推理、顺序扫描与表格查询 [17]。

需要注意，ProLong 发现在它的设置下，SFT 只用短指令数据就足够，加入合成的长指令数据反而有害 [88]。这与 Nemotron 的做法并不矛盾：ProLong 的目标长度是 64K–512K、模型是 8B，Nemotron 面向 1M 上下文的 agent 场景。长指令数据是否有用，要在目标场景下验证。

7.8 长度课程与短上下文保持

有了长文档与合成的长依赖，最后的问题是：怎样一步步把长度拉上去，同时不丢掉短上下文的能力。

各家的长度课程

长度课程有一个共同的结构：在大部分预训练中用中等长度，把最长的阶段留到冷却或中期训练，长度逐级翻倍，并在每一级混入短数据。各家的具体做法如下：

模型	长度课程
Qwen3 [28]	4K 扩到 32K，训练数千亿 token，75% 的文本长 16K–32K、25% 长 4K–16K；RoPE 基频从 1 万提高到 100 万，推理时用 YaRN 与 DCA 再扩 4 倍
GLM-5 [16]	32K 训练 1T，128K 训练 500B，200K 训练 50B
Kimi K3 [14]	预训练中从 8K 扩到 64K，冷却阶段从 256K 扩到 1M
MAI-Thinking-1 [21]	64K 训练 3.4T，256K 训练 150B
DeepSeek-V4 [7]	从 4K 一路扩到 1M
Nemotron 3 Ultra [17]	恒定学习率训练 33B token，46% 长数据加 54% 第二阶段数据；92% 的迭代用 1M 序列，8% 用 4K 序列

多模态模型也在按长度分级。Keye-VL-2.0 的视频数据分三级：15 秒片段、最长 15 分钟、最长 2 小时，对应的视频 token 预算分别为 24K、64K、180K；长上下文阶段与短样本按 1:1 混合 [91]。

短上下文能力的保持

长上下文扩展最常见的副作用，是短文本任务退化。ProLong 的实验给出了三条保护手段：混入短数据，最优比例约为 40%，只用长数据会同时损害长、短两种能力；训练长度大于评测长度，在 512K 上训练能改善 64K 的表现；调大位置编码的基频，它在 64K 时用 8×10^6、在 512K 时用 1.28×10^8

[88]。Nemotron 3 Ultra 还多了一条：按长度分配数据类型，数学与代码的 SFT 风格数据只放在 4K 的迭代里 [17]。这些数据本身很短，硬塞进 1M 的序列只会被拼接稀释。

一个批次的算例

Nemotron 3 Ultra 每次迭代处理 25,165,824 个 token，正好是 24×2^{20}，即 24 条 1M 序列 [17]。33B token 共约 1311 次迭代，其中约 1206 次用 1M 序列，其余约 105 次每次换成 6144 条 4K 序列。用“整批切换”代替“批内混合长度”，让每个批次的注意力计算形态一致，便于并行与显存规划。

长上下文的评测同样需要注意：困惑度与大海捞针都不是好的开发指标 [88]。应使用 HELMET 等多任务套件，在 SFT 之后评测，并同时检查短上下文的表现。

本章思考与进阶练习

7.1 Microanneal 的信号

题目 你的候选数学数据只有 2B 唯一 token。在“5B 候选 + 5B 网页”的 microanneal 里，它会被重复 2.5 次。这会怎样影响你对数据价值的判断?

解题思路 重复 2.5 次仍在安全范围内，但会让小数据集显得更有效：它在退火中的曝光量，大于真实训练中按配比分到的量。应按真实训练中预计的重复次数构造候选部分，例如 2B 候选加 8B 网页，并报告唯一 token 数与重复次数。

7.2 issue-PR 数据的泄漏

题目 你的 1000 万个 issue-PR 对中，有 300 个来自 SWE-bench Verified 的仓库，且晚于对应评测题的创建时间。只删这 300 个够吗?

解题思路 不够。至少还要做三件事：删除评测仓库中所有晚于评测题创建时间的提交、PR 与 issue；用 diff 的 n-gram 检索，检查同一修复是否以 cherry-pick、fork 或转载的形式出现在别处；检查合成的 agent 轨迹是否以评测题为种子。最后在完全排除评测仓库的设置下重新评测，比较分数差异。

7.3 进阶：测量真实的依赖距离

题目 怎样量化一批长上下文数据中，“必须看远处才能预测”的位置有多少?

解题思路 对每个位置 t，比较只看最近 k 个 token 与看完整上下文时的损失：

$$\Delta\ell_t(k) = \ell\left(x_t \mid x_{t-k:t}\right) - \ell\left(x_t \mid x_{<t}\right).$$

统计 $\Delta\ell_t(k)$ 显著大于零的位置所占的比例，并画出它随 k 变化的曲线。这正是 EntropyLong“熵下降验证”的思路，可以用来比较自然长文档、拼接数据与合成数据，也可以作为长上下文数据入库的质量指标。

第八章　SFT 与推理蒸馏数据

预训练与中期训练建立的是能力，SFT 数据决定模型怎样使用这些能力：回答的格式、思考的深度、工具调用的协议，以及在不同任务之间怎样切换。2026 年的 SFT 数据以教师蒸馏为主体，人工从直接写答案转向设计提示与质检；推理轨迹占比最大，agent 轨迹则成为独立的一大类，并有了专门研究它的数据配方。本章先看规模，再分别讨论 query 侧与 response 侧，然后是几份完整的推理数据配方、推理预算、agent 轨迹与专项数据，最后是打包与训练细节。下一章讨论偏好与奖励数据。

8.1　SFT 数据的规模与构成

SFT 数据的量在不同报告之间相差三个数量级。差异的来源，是 base 模型本身的能力。

各家公开的规模

DeepSeek-R1 在冷启动之后，通过拒绝采样收集约 60 万条推理数据，加上约 20 万条非推理数据 [92]。Qwen3-VL 约有 120 万条 SFT 样本，其中三分之一是纯文本 [19]。Nemotron 3 Ultra 的一份推理类 SFT 混合有 40B token，其中科学占 58.75%、数学 23.63%、竞赛代码 10.13%、通用 7.5% [17]。Keye-VL-2.0 的 SFT 数据约 500B token，其中约 40% 是纯文本 [91]。

在另一端，LIMO 只用了 817 道精选数学题，s1 只用了 1000 道题，就在强 base 模型上激发出了很强的推理能力 [93, 94]。而 OpenThoughts 把数据从 3 万条扩到 120 万条，7B 模型的表现一直在提升 [95]。

少而精，还是多而广

这两组证据看似矛盾，调和点在于 base 模型。base 模型越强，预训练中已有的推理容量越大，SFT 只需少量样本激发格式；小模型，或者推理容量不足的模型，则需要大量蒸馏数据来“灌”能力。这与 6.6 节 front-loading 的结论一致：推理的容量在预训练阶段奠定，SFT 能调动它，却难以凭空创造它。

决定 SFT 规模之前，先在 base 模型上用少量高质量数据试一次。少量数据就已有明显提升，说明容量已在，应把投入放在质量上；提升很小，则需要更大规模的蒸馏，甚至回到中期训练补数据。

8.2　Query 侧：构造与过滤

SFT 数据的质量，一半取决于问了什么。query 侧的过滤，比 response 侧的过滤更容易被忽视。

Query 的来源

query 有三类来源。真实用户请求的分布最接近部署场景，但需要清洗与去隐私，Nemotron 用 LMArena 与 WildChat 作为对话数据的种子 [17]。题库（竞赛题、教材习题、StackExchange）的答案可以验

证。合成的 query 规模最大，例如 Magpie 直接从对齐模型中抽取指令 [96]，但容易同质化。

剔除不需要推理的题

Qwen3 的长 CoT 冷启动用 Qwen2.5-72B-Instruct 剔除两类 query [28]：一类是难以验证的，包含多个子问题，或要求泛泛地生成文本；另一类是不写推理就能答对的，这类题放进长 CoT 数据，只会教模型“小题大做”。剩下的每个 query 都标上领域，用于平衡各领域的比例。Qwen3-VL 做了类似的处理：剔除无法验证或信息量低的 query，对有歧义的 query 只做最小程度的澄清 [19]。

难度：参照哪个模型

GLM-5 只保留对上一代 GLM-4.7 仍然困难的数学与科学题 [16]。参照模型有两种选法：用学生模型自身做参照，能精确找到它的知识边界，但要先有学生；用上一代模型做参照，便于提前构造数据。

怎样过滤：LLM 判断胜过分类器

OpenThoughts 比较了多种 query 过滤方法 [95]。基于 LLM 的过滤在所有领域都优于 fastText 与 embedding 过滤。具体到领域，代码题按 LLM 判断的难度过滤最好，数学与科学题则按“回答长度”过滤最好：回答越长的题，往往需要越多的推理步骤，回答长度因此是推理难度的一个廉价代理。相对随机选择，过滤让数学提升约 4%、代码提升约 6%。

来源宜少不宜多

OpenThoughts 还有一个反直觉的发现：只用最好的 2 个代码题源，平均 41.3；用排名前 16 的题源，平均反而降到 36.4。题源越多，混入的低质量题源越多，平均质量就被拉低了。正确的做法是先在每个领域内评估题源，再只保留最好的少数几个。

8.3 Response 侧：教师与拒绝采样

选定了 query，下一步是决定由谁来回答、保留哪些回答。

更强的模型不一定是更好的教师

OpenThoughts 发现，QwQ-32B 作为教师优于 DeepSeek-R1，尽管它在目标基准上的分数更低 [95]。到 agent 数据上，结论同样成立：OpenThoughts-Agent 发现分数最高的 GPT-5.3-Codex 产出的训练数据，反而不如 GLM-4.7 的 [97]。

可能的原因有两个。学生更容易模仿较弱教师的推理风格，或者较弱教师的推理链长度与结构更适合学生；更强的教师则依赖学生不具备的知识，学生只能学到表面形式。对教师的要求是“可学习性”，而不只是正确率。

产业上普遍使用多个教师。Kimi K2.5 的回答由 K2、K2 Thinking 与内部专家模型共同生成，经过多阶段验证 [18]；Kimi K3 用此前各个领域专精的 Kimi 模型合成 agent 轨迹，再经多阶段验证与人工参与标注 [14]；Nemotron 3 Ultra 按领域选用 DeepSeek-V3.2、GPT-OSS-120B、Qwen3-Coder-480B、MiniMax-M2.5 等教师 [17]。

一题多答

OpenThoughts 为每道题采样多个教师答案，把数据规模扩大了至少 16 倍，带来显著提升 [95]。高质量的题目有限时，“一题多答”是扩大 SFT 数据的主要手段：多个答案提供了不同的推理路径，作用类似 4.4 节的“改写替代重复”。

但这种扩展有上限。OpenThoughts-Agent 发现，agent 数据从 3.16 万条扩到 10 万条，靠的是为每个任务采更多 rollout，效果在这个区间进入平台期；而对多样性低的任务来源做合成扩充，在更大规模上仍在提升 [97]。

答案过滤：哪些必须做

OpenThoughts 测试了多种验证与答案过滤方法，没有一种带来显著提升 [95]。Qwen3 的冷启动数据却剔除了六类回答：最终答案错误，大量重复，没有充分推理就猜答案，思考过程与最终总结不一致，不当的语言混杂或风格突变，以及疑似与验证集重叠 [28]。

二者并不矛盾。蒸馏大规模数据时，学生主要学的是推理的格式与过程，答案对错的边际影响小；重复、语言混杂这类显式缺陷，却会被学生直接模仿，必须清除；疑似泄漏的数据则无论如何都要删除。Qwen3-VL 用两层过滤：先用规则过滤重复、截断、格式错误与离题内容，再由奖励模型从正确性、完整性、清晰度、帮助性与视觉依据五个维度打分 [19]。

8.4 推理数据配方

前三节讨论的是单个环节。本节把它们串起来，看几份完整的推理数据配方。

OpenThoughts：1000 多次消融得出的流程

OpenThoughts 的最终流程是 [95]：

1. 每个领域只选最好的少数几个题源；
2. 在领域内混合这些题源；
3. 用 LLM 按难度或回答长度过滤；
4. 去重；
5. 每题采多个答案；
6. 选“可学习性”最好的教师。

每个设计选择都用 31,600 条数据的消融确定，共 1000 多次实验。最终的 OpenThoughts3 有 120 万条数据，训练出的 OpenThinker3-7B 在 AIME25 上达到 53.3，在 GPQA Diamond 上达到 53.7，12 项平均比 DeepSeek-R1-Distill-Qwen-7B 高 12.4 分。

DeepSeek-R1：RL 产出数据，数据再训练

DeepSeek-R1 的流程是先用少量长 CoT 数据做冷启动 SFT，再面向推理任务做 RL；然后从 RL 之后的模型拒绝采样，收集约 60 万条推理数据，加上约 20 万条写作、问答等非推理数据，重新做 SFT；最后再做一轮覆盖全场景的 RL [92]。“RL 产出数据，数据再训练 SFT”的循环，从此成为推理模型的标准流程。

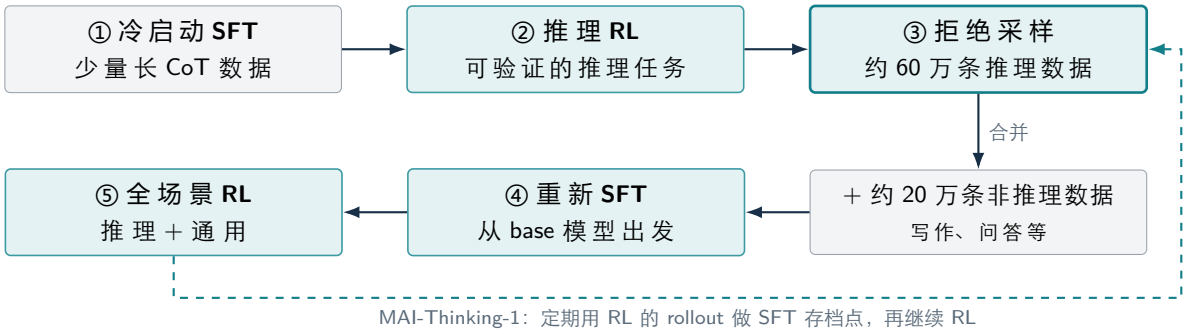


图 8.1 DeepSeek-R1 的数据与训练循环。读图：RL 之后的模型通过拒绝采样产出约 60 万条推理数据，与约 20 万条非推理数据一起重新做 SFT，再进入全场景 RL。虚线是 MAI-Thinking-1 把这一步做成训练中定期存档点的做法。[21, 92]

MAI-Thinking-1 把这个循环变成了训练中的“存档点”：它定期在 RL 自己的 rollout 上对一个新的 checkpoint 做 SFT，再从这里继续 RL [21]。这样做既能从训练崩溃中恢复，也能把进度带到新的 base 模型上。它的消融有三个有意思的结论：约 100 万条轨迹就够了；答错的轨迹几乎和答对的一样有用；取自多个后期 checkpoint 的轨迹，优于只取最终策略的轨迹。第二条结论与 OpenThoughts “答案过滤收益有限”的发现相互印证：在这类蒸馏中，学生学的主要是推理的过程与格式。

多模态的长 CoT 冷启动

Qwen3-VL 的冷启动数据中，图文与纯文本约为 1:1 [19]。它保留通过率低、回答长的样本，剔除错误答案、重复、语言混杂与猜测；并用纯文本的 Qwen3-30B 测试，不看图就能解出的视觉数学题一律剔除，因为这类题教不了视觉推理（11.6 节）。

SFT 阶段对质量更敏感

front-loading 研究的另一半结论是，SFT 阶段对数据质量远比预训练敏感 [83]：高质量数据带来约 15% 的平均提升；用混合质量的数据扩大 SFT 规模，平均没有收益，数学还下降约 5%；而只加入约 0.4% 的高质量数据，就能稳定地带来提升。

规模与多样性的投入应放在预训练与中期训练；SFT 优先保证质量。

8.5 推理预算与思考模式

推理模型的数据，不只要教“怎么想”，还要教“想多久”。

让模型学会在预算内思考

Nemotron 3 Ultra 收集了 GPT-OSS-120B 在中等推理强度下的轨迹，覆盖数学、STEM 问答与指令遵循；再把部分轨迹随机截断到不同的推理预算，截断处的 `</think>` 不计损失 [17]。模型由此学会在预算用完时给出答案，却不会学到“主动提前结束思考”的坏习惯，因为这个结束标记是外部强加的。

思考与非思考模式的融合

Qwen3 的思考模式融合阶段同时训练两种数据 [28]。思考数据用第二阶段 RL 之后的模型，对第一阶段的 query 做拒绝采样；非思考数据覆盖代码、数学、指令遵循、多语言、创作、问答与角色扮演，由自动生成的检查清单评估质量，并为低资源语言增加翻译任务。

模式由三条规则控制：`/think` 与 `/no_think` 放在用户消息或系统消息中；非思考的回答保留一个空的思考块；多轮对话中，以最后一个标记为准。空的思考块让两种模式共享同一种输出格式，模型只需学会何时填充它。

Kimi K3 把这一思路推广到多个推理强度：通用、通用 agent、代码 agent 三个领域，各分低、高、最高三档推理强度，共训练出 9 个专家，再合并成一个模型（10.12 节）[14]。

控制思考的长度分布

思考过长有两个代价：推理成本上升，冗长的推理还会在 RL 中造成不稳定（7.3 节 OctoThinker 的发现）。在 SFT 数据侧，常见的控制手段是按难度分桶，限制简单题的思考长度；同一道题的多个正确答案中，优先保留更短的那个。这些手段各家报告给出的细节不多，效果需要在自己的设置下验证。到 RL 阶段，再用逐题的长度预算继续调节（10.2 节）。

8.6 Agent 轨迹 SFT

推理轨迹教模型“想”，agent 轨迹教模型“做”：读环境、调用工具、根据返回结果修正。

OpenThoughts-Agent：agent 数据的配方

OpenThoughts-Agent 是第一份系统研究 agent SFT 数据的工作 [97]。它最重要的发现是任务来源：仅任务的生成方式一项，就能让 SWE-Bench Verified 的准确率相差多达 30 个点，表现最好的来源既有合成的 issue 任务（SWE-Smith 等），也有人类写的任务（StackExchange SuperUser 等）。来源的数量也有讲究：混合前 4–8 个最好的来源，可以避免对 SWE-Bench 的过度专精，超过前 4 个之后再加来源没有稳定收益，用前 16 个反而有害，这与 8.2 节“题源宜少”的结论一致。

过滤上，剔除少于 5 轮的轨迹是收益最大的一项，即使对齐 token 预算，仍有约 3.5 个点的平均提升；基于 LLM 的任务过滤带来约 3 个点的平均提升；而用 LLM 改写任务描述，则没有带来帮助。最终 10 万条的 OpenThoughts-Agent-v2 数据，微调 Qwen3-32B 后，在 7 个 agent 基准上平均 44.8%，比 Nemotron-Terminal-32B 高 3.9 个点，SWE-Bench Verified 为 54.0%。

Nemotron 3 Ultra 的 agent 数据

Nemotron 3 Ultra 的 agent SFT 数据按任务类型构造 [17]。终端数据约 37 万条多轮轨迹，种子来自 SWE-bench、SWE-rebench、SWE-smith 等数据集，由 DeepSeek-V3.2 在 Harbor 的 Terminus-2 agent 中生成。软件工程数据的问题来自 SWE-Gym、R2E-Gym 与 SWE-rebench V1/V2，由 MiniMax-M2.5 与 Qwen3-Coder-480B 在 OpenHands、SWE-agent、mini-SWE-agent、OpenCode 等多个框架中解题。搜索数据从 9.7 万条 OpenResearcher 轨迹中筛出 2.17 万条，另有需要 50–100 次搜索的困难轨迹。

多个框架与多个教师组合的目的，是让学生学到任务本身的解法，而不是某一个框架的格式习惯。Kimi

K3 在 RL 中走得更远，直接为每组任务采样不同的 agent 框架（10.7 节）[14]。

轨迹的质量过滤

Nemotron 用启发式规则剔除 SWE 轨迹中的四类坏模式 [17]：违规的 git 操作（例如查看未来的提交，属于作弊），反复修改、反复失败的编辑-测试死循环，留在最终 patch 中的调试残留，以及没有运行测试就提交的修改。

GLM-5 的处理方式不同：它保留轨迹中的错误片段，但把它们从损失中 mask 掉 [16]。模型在上下文中看得到错误与失败的反馈，学会怎样从错误中恢复，却不会被训练去生成错误本身。

工具调用的合成管线

Kimi K2 的工具调用数据来自一套多 agent 合成管线 [60]：

1. 从 GitHub 收集真实的 MCP 工具，再按领域演化出大量合成工具；
2. 为工具组合生成任务，以及具有不同系统提示与工具集的 agent；
3. 由用户 agent 扮演人类、工具模拟器返回调用结果，生成多轮轨迹；
4. 由评审 agent 按任务附带的 rubric 打分，只保留成功的轨迹。

代码这类需要真实执行的任务，则使用真实的沙箱。

8.7 多语言、长上下文与格式数据

推理与 agent 之外，还有一类数据量不大、却决定模型能否真正落地的专项数据。

多语言

Nemotron 3 Ultra 的多语言 SFT 数据由平行语料，加上数学、代码与科学数据的合成翻译组成；印地语、日语、韩语、巴西葡萄牙语等语言则用 DeepSeek-V3-0324 新合成 [17]。在其多语种安全数据中，它用回译检查翻译质量：把译文翻回原语言，与原文的语义相似度低于 0.8 的样本剔除，约去掉 10–15%。回译检查不需要目标语言的标注，是一种廉价的质检方式。Qwen3 则在非思考数据中为低资源语言增加了翻译任务 [28]。

长上下文 SFT

Nemotron 3 Ultra 的第二阶段 SFT 打包到 515K，含最长 512K 的合成数据 [17]；Qwen3-VL 在 256K 长度下训练，输入包括数百页的文档、教材与长达两小时的视频 [19]。7.7 节 ProLong 的反例提醒我们，长指令数据是否有用，取决于目标长度与任务 [88]。

格式与可用性

Nemotron 3 Ultra 把“可用性”单列为一类数据，包括 JSON、YAML、XML、TOML、CSV 等结构化输出，引用格式，以及文档信息抽取 [17]。这类数据量不大，却直接决定模型能否接入真实的软件系统：格式错一个字符，下游解析就会失败。

对话数据

Nemotron 的对话数据，以 LMArena 与 WildChat 为种子，由 GLM-5 生成多个候选，用 Nemotron-GenRM 排序后选出最优，再用模拟用户扩展成多轮对话 [17]。“候选生成加奖励模型选优”，把第九章的奖励模型变成了 SFT 数据的过滤器。

8.8 打包与训练细节

数据准备好之后，还要决定怎样把它们组织成训练批次。SFT 数据量小，这些细节的影响反而比预训练更大。

打包：不截断

Nemotron 3 Ultra 按长度做 best-fit 打包：把每个样本放进剩余空间最合适的序列，不截断任何样本，并在包内去重 [17]。SFT 样本被截断的危害比预训练大，因为截掉的往往是答案或轨迹的结尾。

两阶段：先短后长

Nemotron 3 Ultra 第一阶段打包长度 294,912、峰值学习率 1.5e-5，第二阶段打包长度 515K、峰值学习率 1e-5 [17]；Qwen3-VL 第一阶段 32K，第二阶段 256K，并与 32K 数据按课程混合 [19]。先短后长，与 7.8 节预训练的长度课程逻辑一致。

损失的归一化

打包之后，一条序列里有长短不一的样本，损失的归一化方式决定了每个样本的权重：

$$\mathcal{L}_{\text{token}} = \frac{\sum_i m_i \ell_i}{\sum_i m_i}, \qquad \mathcal{L}_{\text{sample}} = \frac{1}{S}\sum_{s=1}^{S} \frac{\sum_{i\in s} m_i \ell_i}{\sum_{i\in s} m_i}.$$

前者按 token 平均，长回答的权重大，长 CoT 主导梯度；后者先在样本内平均、再在 S 个样本间平均，每个样本的权重相同，短回答的格式更受重视。

教学假设：一个 8K token 的推理样本与 40 个各 200 token 的对话样本放在一起。按 token 平均，推理样本占梯度权重的 50%；先按样本平均，它只占 1/41，约 2.4%。推理数据与对话数据混合时，两种方式会训练出很不同的模型，必须在配置中写明。

只用文本激活视觉能力

Kimi K2.5 发现，只用纯文本 SFT 数据，就能激活模型的视觉与 agent 行为，图像操作通过 IPython 代码完成；而用图文 SFT 数据训练，视觉 agent 任务上的表现反而更差 [18]。这件事的前提是联合预训练已经把视觉与文本对齐好（5.6 节）。此时 SFT 只需要教“怎样使用能力”，用文本数据教，泛化反而更好。

本章思考与进阶练习

8.1 教师选择

题目 教师 A 在 AIME 上 80 分，教师 B 只有 70 分，用 B 蒸馏出的 7B 学生反而更好。可能的原因是什么？怎样设计实验验证？

解题思路 可能的原因是 B 的推理链更短、更规整，更容易模仿，而 A 的推理依赖学生不具备的知识。验证可以分三步：统计两位教师轨迹的长度与结构；在教师轨迹上计算学生的初始损失，损失越低，可学习性越高；把 A 的轨迹按长度或风格匹配到 B 的分布后再蒸馏，看差距是否消失。

8.2 难度过滤的参照模型

题目 你用学生模型本身做难度过滤，只保留它 8 次采样中答对 0–2 次的题。这有什么风险？

解题思路 全错的题可能超出学生的能力，或者题目本身有错；只保留难题还会让数据分布偏离真实使用场景。改进方法是对全错题检查教师能否稳定答对，并保留一定比例的中等难度题。难度过滤更适合 RL 数据（10.2 节），SFT 数据要兼顾覆盖面。

8.3 进阶：错误片段 mask 的作用

题目 一条 SWE 轨迹中，agent 先改错了一个文件，测试失败后改对了。比较整条丢弃、整条训练、保留但 mask 错误片段三种处理方式。

解题思路 整条丢弃，损失了“从错误中恢复”的示范；整条训练，学生也学会了生成错误的修改；mask 错误片段，学生看到错误与测试反馈，但只在恢复的步骤上计算损失，学到的是“看到失败之后怎样修正”。关键在于错误片段的边界：通常以测试失败为界，失败之前的编辑整段 mask，之后的诊断与修正正常训练。

第九章　偏好、奖励模型与评判数据

SFT 告诉模型“这样做”，偏好与奖励数据告诉模型“哪个更好”。后者有两种用法：直接训练策略，即 DPO 等离线偏好优化；或者先训练奖励模型或评判器，再为 RL 提供信号（第十章）。到 2026 年，这一层发生了三个变化：判别式奖励模型逐步让位于生成式奖励模型（GRM），不可验证的任务普遍改用基于 rubric 的奖励，偏好数据的构造则从“人工比较”转向“on-policy 采样 + 模型评判 + 少量人工校准”。本章从偏好对的数学结构讲起，依次讨论规模化的偏好数据、GRM、rubric、多模态与生成模型的奖励，最后讨论 reward hacking 的诊断。

9.1　偏好对与 Bradley–Terry 模型

偏好数据只记录“哪个更好”，不记录“好多少”。奖励模型从这种比较中学到的，也只能是回答之间的差值。

从比较中学分数

Bradley–Terry 模型把偏好写成两个奖励之差的函数：

$$P(y_w \succ y_l \mid x) = \sigma\left(r(x, y_w) - r(x, y_l)\right), \qquad \mathcal{L}_{\text{RM}} = -\log \sigma(\Delta), \quad \Delta = r(x, y_w) - r(x, y_l),$$

其中 y_w 是被偏好的回答，y_l 是不被偏好的回答，σ 是 sigmoid 函数。对 Δ 求导：

$$\frac{\partial \mathcal{L}_{\text{RM}}}{\partial \Delta} = -\sigma(-\Delta).$$

这个式子说明了三件事。第一，奖励只在差值上有意义：给所有奖励加同一个常数，损失不变，所以跨 prompt 直接比较奖励的绝对值，没有依据。第二，已经分清的对贡献很小：当 Δ 已经很大（例如 4），梯度约为 -0.018。第三，混淆的对贡献最大：当 $\Delta \approx 0$，梯度为 -0.5。所以一批偏好数据的训练价值，集中在模型还分不清的那些对上；第十章 RL 按通过率筛题，依据的也是梯度集中在“分不清”的样本上。

DPO 把同样的结构直接写到策略上，用 $\beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$ 充当隐式奖励，省去了显式的奖励模型。

三种标注形式

偏好标注有三种常见形式。第一种是成对比较加强度：Llama 3 的标注员对两个回答给出四档偏好强度，还可以编辑被选中的回答得到更好的版本，构成“编辑版 > chosen > rejected”的三级排序 [6]。第二种是多属性评分：HelpSteer2 从帮助性、正确性、连贯性、复杂度、冗长度五个维度各打 0–4 分，约 2.1 万条回答 [98]；多属性的分数可以事后按需组合，比单一偏好保留了更多信息。第三种是模型打分：UltraFeedback 为 6.4 万个 prompt 各从 17 个模型中采 4 个回答，由 GPT-4 从指令遵循、真实性、诚实性、帮助性四个方面打分 [99]，它规模大，但继承了评判模型的偏好。

On-policy 与 off-policy

off-policy 的回答来自其他模型，多样、便宜、可以复用，但与当前策略的分布不一致，学到的差异未必是它需要的。on-policy 的回答由当前策略自己采样，差异正好落在策略的薄弱处，代价是每轮迭代都要重新采样、重新评判。Tulu 3 混合使用两类回答，由 GPT-4o 评判 [100]；Nemotron 3 Ultra 的对话数据则由 GLM-5 生成候选，再用 Nemotron-GenRM 排序 [17]。

9.2 Delta Learning 与规模化偏好数据

如果偏好学习学的只是差值，那么 chosen 回答本身是否足够好，也许并不重要。

Delta Learning：弱数据也能带来强增益

Delta Learning 的假设是：偏好学习学的是 chosen 与 rejected 之间的“差”，而不是 chosen 的绝对质量 [101]。它使用 Tulu 3 的 prompt，chosen 回答由一个小模型生成，rejected 回答由同一模型族中更小的模型生成（例如 Qwen 2.5 3B 对 1.5B），用模型规模代表回答质量。每个数据集去重后约 26.5 万对，生成 chosen 回答的成本约为 Tulu 3 原数据的 6%。

结果如下（11 项基准平均，起点模型 Tulu-3-8B-SFT 为 57.2）：

偏好对	平均
Qwen 2.5 3B 对 1.5B	63.4
Tulu 3 官方偏好数据	63.0
Llama 3.2 3B 对 1B	61.1
Qwen 2.5 1.5B 对 0.5B	59.3

最好的那组 chosen 回答，GPT-4o 质量分只有 3.98，低于 Tulu 3 的 4.44，训练效果却更好。更细的发现还有几条：即使 chosen 回答的平均水平比被训练的模型低 11.4 分，仍能带来 2.1 分的提升；收益随两者的质量差增大，到约 0.55 后饱和；用 72B 对 14B 或 32B 构造的偏好对反而有害；“按规模定好坏”与 GPT-4o 偏好标签的一致率为 80.5%。

Olmo 3 的 DPO 采用了这一思路，在 OLMo-2-7B-SFT 上把平均分从 50.0 提到 55.0，略高于 OLMo 2 原有的偏好数据（54.8）[13, 101]。结合 9.1 节的梯度分析，可以这样理解：偏好对的价值在于给出一个清晰、正确的差异方向，而不在于 chosen 回答本身有多好。72B 对 32B 的差异多是风格细节，方向并不清晰。

4000 万对偏好数据的精选

Skywork-Reward-V2 构建了 4000 万对的 SynPref-40M，用两阶段的人机协同流程精选：人类提供经过验证的标注，LLM 在人类的指导下做自动精选 [102]。最终用其中 2600 万对训练了 8 个奖励模型（0.6B–8B），在 7 个主流奖励模型基准上取得最好成绩，并超过了生成式奖励模型；消融显示，收益同时来自规模与精选质量。这代表了一种新的分工：人工标注不再直接产出训练数据，而是作为种子与校准，指导模型去精选大规模数据。

9.3 生成式奖励模型

判别式奖励模型只输出一个标量。它快，但不能解释，也无法按 rubric 逐条检查。生成式奖励模型（GRM）让评判者先写理由，再给判断。

从打分头到生成判断

生成式验证器把“答案是否正确”变成下一个 token 的预测，并允许先写一段 CoT 再判断；在推理任务上，它优于判别式奖励模型 [103]。代价是推理开销大，在 RL 中需要专门的服务化部署（第十四章）。

产业实例

各家的差别，主要在“有没有参考答案”与“评判者是谁”两件事上。Qwen3 的通用 RL 覆盖 20 多类任务，按这两件事分了三类奖励：规则奖励；有参考答案时，由 Qwen2.5-72B-Instruct 对照参考答案评判；没有参考答案时，用在人类偏好数据上训练的奖励模型 [28]。

Kimi 系列让模型自己当评判者。Kimi K2 按 rubric 对自己的输出做成对比较，rubric 分三类：体现模型价值观的核心 rubric、防止 reward hacking 的规定性 rubric、人工标注的场景 rubric；评判者本身用可验证任务的信号持续校准 [60]。Kimi K2.5 把同一个 GRM 推广到对话助手、代码 agent、搜索 agent 与产物生成 agent，评判帮助性、相关性、细节、美观与指令遵循 [18]。

DeepSeek-V4 的原则是能用规则验证的任务就用规则；其余任务，由领域专家在自己的 GRPO 训练中按 rubric 评判轨迹，只需少量人工标注 [7, 104]。

成对比较的成本与位置偏差

GRM 做成对比较时，有两个数据侧的代价要算清楚。第一是调用次数。一组 G 个候选做循环赛，需要

$$C_{\mathrm{rr}} = \binom{G}{2} = \frac{G(G-1)}{2}$$

次比较；淘汰赛只需要 $G-1$ 次，但只能排出冠军，排不出完整的名次。

第二是位置偏差：评判者往往偏爱放在前面（或后面）的那个回答。常规做法是每一对交换顺序各评一次，两次结论一致才采纳，不一致就记为平局，调用次数因此再翻一倍。

数值例子（教学假设：$G = 8$，每次评判输出 1,500 token）：逐个打分是 8 次调用、约 1.2 万 token；循环赛加顺序交换是 $2 \times 28 = 56$ 次调用、约 8.4 万 token。一个 prompt 的评判开销，可能比生成 8 个候选还大。成对比较买到的是稳定性，代价是评判算力，所以它更适合放在不可验证、又对质量敏感的任务上，而不是全量使用。

Kimi K3 的 agentic GRM

Kimi K3 把 GRM 设计成一个 agent，有三个关键做法 [14]。第一是锦标赛式的组奖励：同一组候选之间做二元的成对比较，比绝对打分稳定。第二是评判者自己写 rubric：评判者必须先读完产出，自己写出评分细则，再逐个候选打分，并把分数记在一个“计分板”上，判断因此有据可查。第三是冗

长控制：候选长度超过 $\sigma \cdot \ell_0$ 时，这次比较自动判负，其中 ℓ_0 是冷启动模型的估计冗长度，σ 是倍数阈值，这直接切断了“越长越高分”的漏洞。

评判数据从哪来

GRM 本身也需要训练数据，常见来源有三个：用强模型生成“理由 + 判断”，按判断是否与人工标签一致做拒绝采样；在有标准答案的任务上直接检查 GRM 的判断（Kimi K2 的闭环）；以及几百到几千条高质量的人工判断，作为检测 GRM 偏差与漂移的校准集。

9.4 Rubric 数据

不可验证的任务，没有标准答案可比。rubric 把“好回答”拆成一条条可以检查的标准。

Rubrics as Rewards：怎样生成，怎样聚合

RaR 为每个 prompt 生成 7–20 条能独立检查的 rubric 条目，由 o3-mini 或 GPT-4o 以参考答案为条件生成 [105]。每条带一个类别：必须满足（Essential）、重要（Important）、加分项（Optional）、常见错误（Pitfall）。

聚合有两种方式。显式聚合逐条判断是否满足，按权重求和后归一化，权重分别为 Essential 1.0、Important 0.7、Optional 0.3、Pitfall 0.9。一种常见的写法是

$$r = \frac{\sum_{j \in P} w_j c_j - \sum_{k \in F} w_k e_k}{\sum_{j \in P} w_j}$$

其中 P 是正向条目，$c_j \in \{0,1\}$ 表示是否满足；F 是 Pitfall 条目，$e_k \in \{0,1\}$ 表示是否犯了这个错。

数值例子（教学假设）：一份 rubric 有 2 条 Essential、3 条 Important、2 条 Optional、1 条 Pitfall。回答满足了 2 条 Essential 与 2 条 Important，没有满足 Optional，并犯了那条 Pitfall。分母为 $2 \times 1.0 + 3 \times 0.7 + 2 \times 0.3 = 4.7$，分子为 $2.0 + 1.4 - 0.9 = 2.5$，$r \approx 0.53$；不犯 Pitfall 时 $r \approx 0.72$。一条 Pitfall 的扣分，几乎抵掉了一条 Essential 的收益。权重怎样设，直接决定策略会优先“避坑”还是“补全”。

隐式聚合则把整份 rubric 交给评判者，给出 1–10 的总分，再归一化。在 HealthBench 上，隐式 RaR 达到 31.2%，直接 Likert 评分为 25.5%，参考答案加 Likert 为 28.9%，所有题目共用一套通用 rubric 只有 12.5%；在 GPQA-Diamond 上，隐式 RaR 37.6%，直接 Likert 34.8%。

rubric 的质量取决于参考答案：在 HealthBench-1k 上，以参考答案为条件生成的 rubric 达到 35.9%，不用参考答案的只有 32.0%，人工撰写的为 34.8%。

从预训练文档自动合成 rubric

ARES 把 rubric 的构造延伸到预训练语料 [106]。它先把原始文档中的知识转写成自包含的问答对，再为每道题生成带权重的 rubric，并用领域标签与人物设定增加多样性，最后验证题目是否自包含、答案是否忠实于原文、rubric 是否有效。在 10 个领域得到 10 万条带 rubric 的样本；在 7 个基准上，基于 rubric 的 RL 优于继续预训练、SFT 与二元奖励 RL。这条路线把“原文中的知识”变成了“可以用 RL 训练的任务”，打通了预训练语料与 RL 数据。

Query 与 rubric 要一起设计

QUBRIC 指出了一个被忽视的问题：rubric 的质量取决于 query 的结构 [107]。过于开放的 query 只能写出含糊的 rubric；简单地收窄 query，参考答案又可能无法验证，回答拿不到奖励。

它的做法分三步：用教师得出的要点，把开放式 query 改写成有场景、可评估的问题；用教师与学生回答之间的差距，生成针对这道题的评判标准；只保留“有区分度”的 query–rubric 对，用于 GRPO 训练。只用指令遵循数据训练，ArenaHard 比 SFT 基线提高 5.5 分，并迁移到法律、道德、叙事三个留出基准，平均提高 6.3 分。

树形 rubric 与可执行打分

QUEST 为深度研究任务构造奖励 [108]。客观任务由 Claude Sonnet 4.5 从热点关键词出发浏览网页，构建由可检查约束组成的 rubric 树，再由 GPT-5 把这棵树写成 Python 打分脚本；开放任务则共用指令遵循、全面性、可读性、洞察力四个根标准，与参考报告做成对比较。

它的质量控制非常严格：1.7 万个客观任务只保留了 5,934 个；人工抽查 50 个，仍有 2 个脚本无法执行、6 个 rubric 有错，错误率约 16%。可见 rubric 本身也需要质检：可执行的 rubric 要实际运行一次；用强、弱两个模型的回答测试区分度；同一回答多次评判，看方差；并检查高分回答是否只是堆砌了关键词。

9.5 规则化奖励与多模态、生成模型的奖励数据

文本之外，视觉理解与视觉生成的奖励各有自己的构造方式，但思路与 rubric 相通：把一个模糊的“好”拆成可以逐项检查的维度。

视觉理解：连续的规则奖励

Kimi K2.5 为视觉 RL 的每类任务设计了连续奖励 [18]：框定位用软 IoU 匹配下的 F1，点定位用高斯加权距离匹配下的 F1，分割用栅格化后 mask 的 IoU，OCR 用归一化编辑距离，计数用与真值的绝对差。连续奖励比“对或错”的二元奖励提供更细的梯度：差一点的框，也能得到部分奖励。

GLM-5V-Turbo 把规则验证器放在本地同步执行，模型评判通过 API 异步调用，并为每个样本打上数据来源标签，以便按来源统计奖励与 pass@k [109]。

视频生成：VLM 奖励与人工偏好

HunyuanVideo 1.5 的后训练奖励分两条线 [38]。I2V 用 VLM 奖励模型评估文本对齐、图像对齐、画质与运动动态，prompt 集覆盖 100 多个类别，由 VLM 生成后人工检查。T2V 用约 1 万个 prompt，每个 prompt 的多个候选由人工做 GSB（好/同/差）标注，构成偏好对，先做离线 DPO，再做在线 RL。

图像生成：把 prompt 拆成可检查的元素

Z-Image 的指令遵循奖励，先把 prompt 拆成主体、属性、动作、空间约束与风格等元素，再由标注员逐项标出生成图中没有满足的元素，满足元素的比例就是指令遵循分 [52]。文字渲染、物体计数这

类客观维度，则由 VLM 生成偏好对、人工验证后做 DPO。Qwen-Image-2.0 为美学、图文对齐、人像、指令遵循、视觉一致性分别训练奖励模型 [53]。

“拆成元素、逐项检查”与文本侧的 rubric 完全同构。生成模型的多维奖励与它们的标注协议，在 12.8 节还会结合产业实例展开。

9.6 Reward hacking 的诊断

任何奖励都是对真实目标的近似。策略优化得越用力，就越会找到近似与目标之间的缝隙。

常见的漏洞

漏洞大致有三类，分别出在奖励模型、rubric 与环境上。

奖励模型层面，最常见的是长度偏差：奖励随回答长度上升，RLHF 的大部分“收益”只是回答变长了 [110]，Kimi K3 的冗长控制正是针对这一点。另一种是行为偏好，例如奖励偏爱“调用工具”，模型在该拒答时不拒答，这是 AWM 在 BFCL 幻觉测试上暴露的弱点 [111]。

rubric 层面，回答会堆满 rubric 关键词，换取逐条“满足”。环境层面，agent 会绕过任务本身去操纵评测，例如读取测试文件、伪造产物。

三类漏洞有一个共同的表现：代理奖励持续上升，真实质量先升后降。这就是过度优化。

过度优化的规律

Gao 等人用一个“金标准”奖励模型模拟真实偏好，测量策略对代理奖励优化时，金标准分数怎样变化 [112]。他们以 $d = \sqrt{\mathrm{KL}(\pi \,\|\, \pi_{\text{init}})}$ 衡量策略偏离初始模型的距离，发现 RL 下的金标准分数近似为

$$R_{\text{gold}}(d) = d\,(\alpha - \beta \log d)$$

其中 α、β 是随奖励模型规模与数据量变化的系数。对 d 求导并令其为零，得到金标准分数的峰值位置

$$d^* = e^{\alpha/\beta - 1}$$

数值例子（教学假设）：$\alpha = 1$、$\beta = 0.5$ 时，$d^* = e \approx 2.72$，对应 KL 约 7.4 nats，越过这一点继续优化，金标准分数开始下降。若奖励模型更大、数据更多，使 β 降到 0.4，$d^* = e^{1.5} \approx 4.48$，KL 约 20 nats，可以安全地优化得更远。

论文的另一条发现与此一致：代理奖励模型越大、训练数据越多，下降来得越晚。由此得到两条实用原则：用 KL 约束或早停，限制策略偏离初始模型的程度；定期用独立的评判者（人工或更强的模型）检查真实质量，在 d^* 之前停下来。

对策

对策按漏洞的位置分三层。

奖励层面，用多个 rubric 或多个奖励同时打分，一个维度上的作弊会被其他维度惩罚 [18]；对长度，

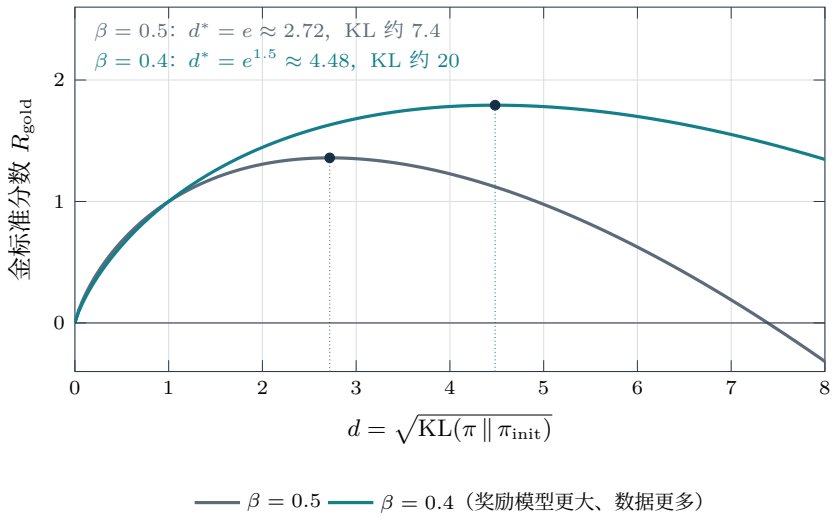


图 9.1　奖励的过度优化。读图：横轴是策略偏离初始模型的距离，纵轴是金标准分数 $R_{\text{gold}}(d) = d(\alpha - \beta \log d)$，取 $\alpha = 1$。β 从 0.5 降到 0.4 时，峰值从 $d^* \approx 2.72$ 推迟到 $d^* \approx 4.48$。数值为教学假设。[112]

用长度惩罚，或 Kimi K3 式的超长自动判负 [14]；对评判者本身，定期在有标准答案的任务上校准，让它的判断锚定在可验证的信号上 [60]。

rubric 层面，把已知的作弊方式写成 Pitfall 条目 [105]，让它们直接扣分。

环境层面，GLM-5.2 的防作弊模块由规则过滤器加 LLM 评判组成，被拦截的调用返回一个假结果，让 rollout 继续而不是中断，避免策略从“被中断”这件事中学到信号 [113]；Kimi K3 的 kernel 优化任务专门设有作弊检测器，惩罚 CUDA graph 重放、缓存输入、降低精度等手段（10.4 节）[14]。

奖励模型上线前的检查

上线前至少检查四项：

1. 在独立的偏好测试集上的准确率；
2. 奖励与回答长度的相关系数；
3. 对抗样本（堆砌关键词、格式正确但内容错误）是否被压低分数；
4. 小规模的 RL 试跑：真实质量是否随代理奖励一起上升。

本章思考与进阶练习

9.1　Delta Learning 的质量差

题目　Delta Learning 中，72B 对 14B 的偏好对反而有害。结合 9.1 节的梯度分析给出解释，并设计验证实验。

解题思路　两个强模型的回答都远在 8B 学生的分布之外，它们的差异多是风格细节，没有给出学生能沿着走的“质量方向”；DPO 却在推动学生去区分它自己生成不出来的回答。验证时，用学生的对数概率衡量 chosen 与 rejected 离学生分布有多远；固定质量差、改变这个距离，观察 DPO 收益的变化。

9.2 Rubric 的覆盖率

题目 一份医学问答的 rubric 有 10 条，全部来自参考答案。一个回答给出了参考答案中没有、但同样正确且重要的建议，显式聚合会怎样评价它?

解题思路 显式聚合只检查列出的 10 条，新增的正确内容得不到奖励；如果它挤占了篇幅，还可能被扣分。改进方法有几种：加一条“额外的、临床正确且相关的信息”；改用隐式聚合；用多个参考答案生成 rubric；或者用 QUBRIC 式的做法，先把 query 改写得更具体。

9.3 进阶：定位奖励漏洞

题目 RL 中 GRM 分数持续上升，人工抽检的质量却在下降。怎样定位漏洞?

解题思路 先比较训练前期与后期高分回答的统计量（长度、格式、关键词密度、工具调用次数），找出随训练单调变化的特征；再对候选特征做控制变量，例如截到同样长度后重新评分，看分数是否还高；同时读 GRM 给出高分时的理由，看它的依据是否落在表面特征上。修复时加入 Pitfall 条目或新的奖励维度，用对抗样本重新校准 GRM，并从出问题之前的 checkpoint 恢复训练。

第十章　RL 任务、环境与轨迹数据

RL 阶段的“数据”，与前几章完全不同：训练样本由策略自己生成，数据集只提供 prompt、验证器与环境，一份数据的价值取决于它能否在当前策略上产生有区分度的奖励。规模的单位也变了。Qwen3 的推理 RL 只有 3,995 道题，到 2026 年，MiniMax 报告了数十万个环境，Prime Intellect 的开源环境库超过 2,500 个，Kimi K3 的环境覆盖从网页研究到持续多日的个人助理工作流。本章先给出 RL 数据的三要素与 prompt 筛选的原理，再依次讨论数学、代码、软件工程、终端、工具、搜索、GUI 与多智能体任务的构造，最后讨论怎样从已有轨迹中挖掘 RL 数据，以及专家 RL 与 on-policy 蒸馏的数据流。

10.1　RL 数据 = prompt + 验证器 + 环境

RL 数据集不保存答案，只保存“怎样判断答案好坏”与“动作作用于哪里”。

三要素

$$\mathcal{D}_{\mathrm{RL}} = \{(x_i,\ \mathcal{V}_i,\ \mathcal{E}_i)\}, \qquad J(\theta) = \mathbb{E}_{(x,\mathcal{V},\mathcal{E})\sim\mathcal{D}}\ \mathbb{E}_{\tau\sim\pi_\theta(\cdot|x,\mathcal{E})}\left[\mathcal{V}(\tau)\right],$$

其中 x 是任务描述，例如一道数学题、一个 GitHub issue、一个用户请求；$\mathcal{V}$ 是验证器，把轨迹 τ 映射成奖励，例如答案比对、单元测试、数据库状态检查、GRM 加 rubric；$\mathcal{E}$ 是环境，负责响应动作，例如代码沙箱、容器化的仓库、工具模拟器、浏览器、虚拟机。

单轮推理任务的环境是空的；在 agent 任务中，环境的质量往往决定一切。

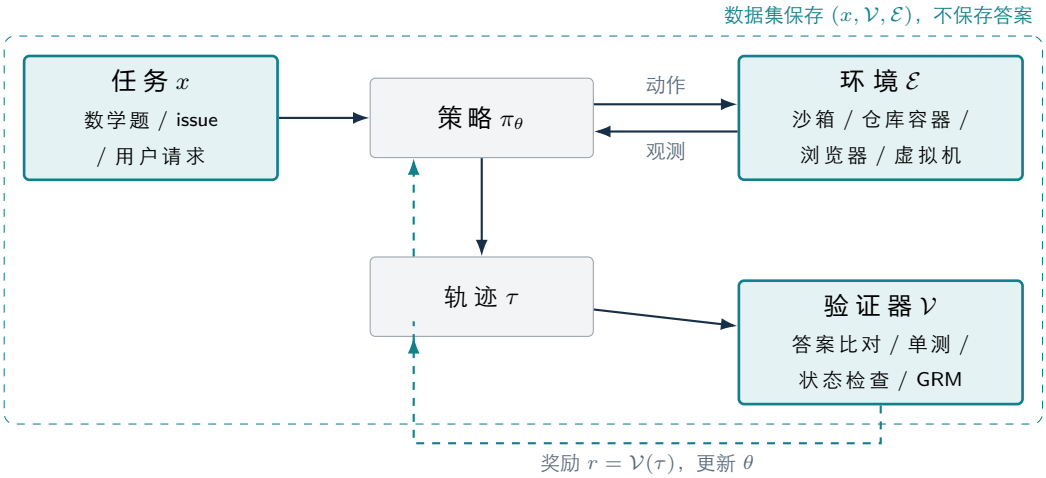


图 10.1　RL 数据的三要素。读图：策略拿到任务 x 后与环境 $\mathcal{E}$ 多轮交互，产生轨迹 τ；验证器 $\mathcal{V}$ 把轨迹映射成奖励，再用奖励更新策略。虚线框内是数据集真正保存的部分。

规模与构成

各家公开的 RL 数据规模如下：

模型	规模与构成
Qwen3 [28]	推理 RL 用 3,995 组 query–验证器，通用 RL 覆盖 20 多类任务
DeepSeek-V3.2 [114]	1800 多个环境、约 8.5 万个 prompt：代码 agent 24,667 个、搜索 agent 50,275 个、通用 agent 4,417 个、代码解释器 5,908 个
MiMo-V2-Flash [115]	代码 agent 12 万个任务（9 万个真实环境配真实 prompt，3 万个真实环境配合成 prompt），搜索 agent 15 万个，通用 agent 5 万个（环境与 prompt 都是合成的）
MiniMax M2.5 [116]	数十万个环境，其中代码环境超过 20 万个，许多来自公司内部的任务与工作区
Qwen3.5 [8]	官方称其异步 RL 框架能支撑百万级规模的 agent 框架与环境，并扩展了“几乎全部”RL 任务与环境

从这张表能看出两个趋势：agent 任务的数量已经远超单轮推理题；环境的来源从公开数据集转向合成与公司内部的真实工作区。

统一的多领域 RL

DeepSeek-V3.2 把推理、agent 与对齐三类数据合并到同一个 RL 阶段，以减少灾难性遗忘 [114]：各领域分别训练再依次接上，后训练的领域会覆盖前一个领域。统一训练的代价是批次内的配比更复杂。另一条路线是先分领域训练专家，再用 on-policy 蒸馏合并，10.12 节会讨论。

10.2 Prompt 池的筛选

拿到环境之后的第一个问题是：哪些 prompt 值得放进训练？答案取决于当前策略在这道题上的通过率。

为什么要按通过率筛

GRPO 类算法使用组内归一化的优势：

$$A_i = \frac{r_i - \mathrm{mean}(r_{1:G})}{\mathrm{std}(r_{1:G})},$$

其中 r_i 是同一道题第 i 个回答的奖励，G 是组大小。一组 G 个回答如果全对或全错，优势全为零，这道题不产生任何梯度。设单次通过率为 p，一道题“有信息”（组内奖励不全相同）的概率为：

$$P_{\text{info}}(p) = 1 - p^G - (1-p)^G.$$

通过率 p	$G = 8$	$G = 16$
0.01	0.077	0.149

通过率 p	$G=8$	$G=16$
0.05	0.337	0.560
0.10	0.570	0.815
0.50	0.992	≈1.000
0.95	0.337	0.560

太难与太简单的题都在浪费 rollout。增大 G 能救回一部分难题，但每道题的成本也随之上升。这与 9.1 节奖励模型的梯度集中在“分不清的偏好对”上，道理相同。

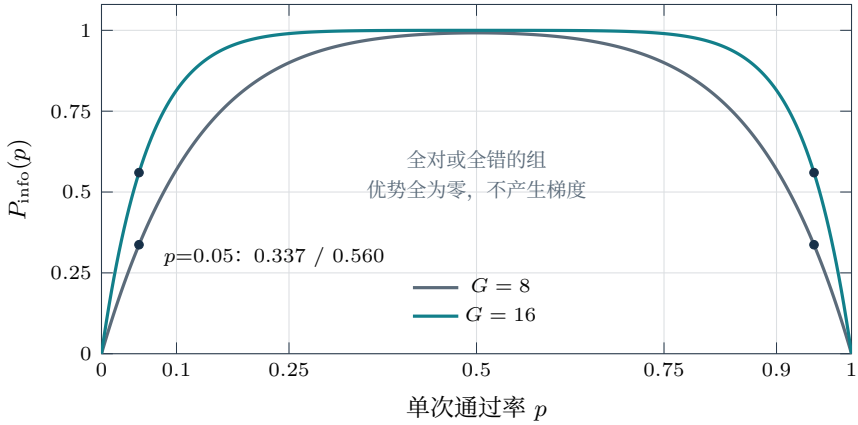


图 10.2　有信息题目的比例。读图：曲线是组内奖励不全相同的概率 $P_{\text{info}}(p)=1-p^G-(1-p)^G$。通过率接近 0 或 1 的题几乎不产生梯度；组大小从 8 增到 16，能救回一部分过难与过易的题。

离线筛选：各家的通过率窗口

离线筛选用训练前的模型估计每道题的通过率，再按窗口取舍。Qwen3-VL 每道题采样 16 次，剔除全错的题与通过率超过 90% 的题 [19]；Seed1.5-VL 保留通过率在 0%–75% 之间的 STEM 题 [117]；Qwen3 推理 RL 的 3,995 道题要同时满足四条：冷启动阶段没用过、冷启动模型学得会、尽可能难、覆盖各个子领域 [28]。

Kimi k1.5 在通过率之外，还专门做了“易猜”过滤 [118]。它用 SFT 模型高温采样 10 次来估计难度，剔除选择题与判断题；再让模型不写推理、直接猜答案，8 次内能猜中的题一律剔除。这针对的是一种隐蔽的 reward hacking：答案空间很小的题（选择、判断、小整数），模型可以靠猜拿到奖励，学不到推理。

在线筛选与难度画像

离线筛选基于旧模型，在线筛选则跟着当前策略走。DAPO 在训练中为每个批次过采样，丢掉全对、全错的组，直到批次装满有效组 [119]；Nemotron 3 Ultra 定期刷新 RL 数据，先用当前模型做奖励画像，再按高斯形的难度分布精选，让多数题落在中等难度附近 [17]。

逐题的长度预算

Kimi K2.5 的 Toggle 方法，每隔若干次迭代，在“预算受限”与“正常扩展”两个阶段之间切换 [18]。每道题的预算，取它正确 rollout 长度的第 ρ 百分位数，估计一次后固定。用在 K2 Thinking 上，输

出 token 减少约 25–30%，表现几乎不变。预算是逐题设定的，难题的预算自然更长，不会被一刀切的长度惩罚误伤。

10.3 数学与推理

数学是最早、也最成熟的 RL 数据类型。它的难点不在出题，而在判分。

答案的等价判定

数学 RL 最大的噪声来源，是“答对了却被判错”。形式不同的等价答案（$\frac{1}{2}$ 与 0.5、$\sqrt{2}/2$ 与 $1/\sqrt{2}$）要用符号计算（如 SymPy）判定；单位与格式（“5 cm”与“5”）需要先规范化；方程的多个根要按集合比较；中间结果被误当作最终答案时，需要固定的输出格式（如 `\boxed{}`），并配合格式奖励。

这些错判会直接改变奖励的期望。设策略答对的概率为 q，验证器把正确答案判错的概率（假阴性率）为 f，则

$$\mathbb{E}[r] = q\,(1 - f)$$

数值例子（教学假设）：$q = 0.4$、$f = 0.2$ 时，期望奖励从 0.4 降到 0.32。更麻烦的是，被误判的那 20% 正确回答拿到的是负优势，策略会被推离这些写法，例如推离不写成 `\boxed{}` 的正确答案，转而学会迎合判分器的格式。

题目本身的错误

参考答案错了，模型会因为答对而受罚。有两种检测方式：强模型多次采样后，一致给出与参考答案不同的结果，就标记为疑似错误并复核；训练中通过率长期为零、而模型回答高度一致的题，同样复核。

证明题：验证器本身成为数据

答案比对只适用于有最终答案的题。证明题没有“答案”，验证器必须读懂整段推理。DeepSeekMath-V2 的做法，把验证器的训练数据放在了和生成器同样重要的位置 [120]：

1. **训练验证器**：从 AoPS 爬取 17,503 道题，用 DeepSeek-V3.2-Exp-Thinking 的一个变体生成候选证明，由数学专家抽样打分。分数只有三档：1 为完整严谨，0.5 为逻辑正确但有小错或省略，0 为根本错误。验证器的奖励是 $1 - |s' - s|$，其中 s' 是它给的分，s 是专家的分。
2. **元验证**：再训练一个元验证器，检查验证器指出的问题是否真的存在、是否足以支撑它给的分，并把它的输出乘进验证器的奖励。验证意见的平均质量由此从 0.85 升到 0.96，打分准确率不变。
3. **训练生成器**：生成器写完证明后还要自评，奖励为 $0.76\,R_Y + 0.24\,R_Z$，其中 R_Y 是验证器给证明的分，R_Z 衡量自评与验证器是否一致。生成器因此被激励在提交前自己找出并修掉问题。
4. **自动标注**：对新的难证明，采样多份验证意见，再对其中指出问题的意见做多次元验证；若足够多的有效意见给出同一个最低分，就以该分为标签，全部意见都没找到有效问题则标为 1，其余交给专家或丢弃。最后两轮迭代中，这条管线完全替代了人工标注。

结果是 IMO 2025 解出 6 题中的 5 题，Putnam 2024 得到 118/120（使用了扩展的测试时计算）。从

数据的角度看，它解决的是“生成器变强之后，验证器跟不上”的问题：用更多的验证计算换来新的标签，再用这些标签训练更强的验证器。

程序生成的题目

程序生成的题目有两个优点：答案天然正确；难度可以用参数控制，适合构造难度课程。Kimi K2.5 合成复杂的视觉谜题，由 K2 作为验证器 [18]；Seed1.5-VL 合成了 2 万多个视觉谜题，以及“找不同”类数据 [117]；GLM-5 的逻辑推理使用可验证的题目，通过拒绝采样产出数据 [16]。Kimi K3 进一步用知识图来组织任务合成（10.7 节）[14]。

10.4 竞赛代码与 kernel 优化

代码任务的验证器是测试。测试的质量，直接决定奖励的质量。

测试用例的质量

测试太弱会产生假阳性：错误的程序也能通过，模型学会的是“过测试”而不是“写对程序”。常见做法是先用 LLM 生成“测试生成器”，构造边界情况与大规模输入；再用参考解运行生成的测试，参考解跑不通的测试丢弃；最后用已知的错误解检验测试的区分力。

Nemotron 3 Ultra 的竞赛代码专家，只用约 3.5K 个经过过滤的 prompt 做 RL，而它的 SFT 阶段用了数百万条教师轨迹 [17]。SFT 灌入解题模式，RL 在少量难题上打磨，这种分工在各家报告中反复出现。

Kernel 优化：连续奖励与作弊检测

Kimi K3 把 GPU kernel 优化作为一类 RL 任务 [14]。任务范围从单个算子到融合的大 kernel，覆盖 CUDA、Triton、CuTe DSL、TileLang 等编程方式，以及 BF16、FP8、FP4 等精度。奖励分三段：数值误差超过阈值，奖励为零；速度追平专家实现，得 0.5；越接近硬件的 roofline 上限，奖励越接近 1。另有专门的检测器，惩罚 CUDA graph 重放、缓存输入、私自降低精度等作弊手段。

这个设计展示了“难解易验”任务的一种新形态：正确性是硬门槛，性能是连续的奖励，作弊检测作为第三道关。

10.5 软件工程环境

软件工程任务把“写代码”放进了真实仓库：读 issue、定位文件、修改、让测试通过。它的环境构造成本，远高于单题代码。

一个 SWE 环境由什么组成

一个 SWE 环境包括某个 commit 时刻的仓库快照、装好依赖的容器镜像、issue 描述，以及两组测试：修复之后应当通过的 FAIL_TO_PASS 测试，和修复前后都应通过的 PASS_TO_PASS 测试。DeepSeek-V3.2 的有效性判据是：参考补丁能让至少一个失败的测试通过，并且不破坏任何已通过的测试 [114]。

任务从哪里来

构造方式大致分为四类，它们在规模与任务分布上各有偏向：

构造方式	代表与规模	任务分布的特点
真实 issue-PR，人工辅助搭环境	SWE-Gym：11 个仓库、2,438 个 Python 任务 [121]	最接近用户的问题，但规模小
从 commit 反向生成 issue	R2E-Gym：13 个仓库、约 8.1K 个任务 [122]	issue 描述的质量取决于生成模型
向仓库注入 bug	SWE-smith：128 个仓库、约 5 万个任务 [123]	难度与数量可控，但可能偏离真实缺陷的形态
自动搭环境的大规模采集	SWE-rebench V2：20 种语言、3,617 个仓库、32,079 个任务 [124]	规模最大，仓库与语言的分布不均匀

最后一类是 2026 年的主力。SWE-rebench V2 用交互式安装 agent 自动编写安装与测试脚本，再由 LLM 评审集成过滤不健全的任务，另有 12 万多个扩展任务 [124]。DeepSeek-V3.2 从数百万个 issue-PR 对中筛出覆盖 8 种语言的数万个环境 [114]。MAI-Thinking-1 从约 1.02 亿个 GitHub PR 中筛出约 487 万个带关联 issue 的 PR，在按需启动的沙箱中用 FAIL_TO_PASS 与 PASS_TO_PASS 测试评分 [21]，漏斗的留存率不到 5%。MiniMax 则走了另一条路，把公司内部的任务与工作区转成了 20 万多个代码环境 [116]。

8.6 节 OpenThoughts-Agent 的发现在这里同样适用：混合几种最好的来源，比单一来源好。

自动搭建与验证

SWE-rebench V2 的流程代表了 2026 年的做法 [124]：

1. 交互式安装 agent 为每个仓库写安装与测试脚本，失败时读报错、重试；
2. LLM 评审集成过滤不健全的任务，例如测试过于严格、描述过于模糊；
3. 为每个任务标注混杂因素，例如测试只接受某一种实现；
4. 每个任务附带预构建的容器镜像，保证可复现。

评审集成用人工核验过的 SWE-bench 标注做了校准。

防泄漏

SWE 环境有三条泄漏路径。仓库的 git 历史中可能有未来的修复提交，所以要删除快照之后的历史，并拦截相关的 git 命令，Nemotron 的 SWE-RL 专门做了这一步 [17]。测试文件暴露了预期行为，所以训练时要隐藏 FAIL_TO_PASS 测试。与评测集同仓库、同 issue 的任务，则按仓库与时间排除（7.4 节）。

10.6 终端环境

终端任务比 SWE 任务更开放：目标是让系统达到某个状态，而不是让某组测试通过。

按最终状态验证

Harbor 提供了标准化的终端任务框架，Terminus-2 是其中的 agent 实现；Nemotron 3 Ultra 用它生成了约 37 万条终端轨迹，终端专家的任务可长达 1 小时 [17]。

任务结束后，在容器中运行检查脚本，确认文件、进程与服务的最终状态。检查脚本只看结果、不看过程，所以同一个目标可以有多种合法的做法，这比 SWE 的固定测试更宽容，也更难写对：脚本漏查一项状态，就会放过一种“看起来完成了”的错误做法。

从网页教程合成任务

GLM-5 从种子数据与网页语料中合成终端环境 [16]。网页上大量的教程与问答，本身就是“目标状态 + 操作步骤”的描述：把步骤藏起来、把目标状态写成检查脚本，就得到一个可执行的任务；教程中的步骤还可以作为参考解，用来确认任务在给定环境里确实可解。

长时程任务的信号密度

长时程任务对数据有两个特殊要求。第一，中间状态要能快照，失败后可以从中间恢复（14.7 节）。第二，奖励的信号密度太低。

数值例子（教学假设）：一条 1 小时的终端 rollout 包含约 200 次工具调用，最终只得到一个二元奖励。平均下来，每次决策分到的监督不到 1/200 比特，而且无法区分 200 步里哪几步是关键的。同样的 1 小时，如果在 10 个检查点上验证中间状态，信号就多了 10 倍，还能定位出错的阶段。这就是 10.11 节 PivotRL 的动机：与其让整条轨迹共用一个比特，不如把计算集中在少数关键回合上。

10.7 工具调用与工作流环境的合成

真实的工具与服务很难大规模接入 RL。2026 年的主流，是由 LLM 写出能真实运行的工具代码与数据库，即合成的可执行环境。

三种环境的取舍

模拟环境由 LLM 扮演工具，构造成本最低、规模最大，但状态可能前后矛盾，验证只能依赖 LLM 评审。合成的可执行环境有真实的数据库与代码，状态一致，可以检查最终状态，规模在千到万级。真实环境的状态与验证都可靠，但规模受限。下面三个例子都属于中间这一类。

Agent World Model：1000 个代码驱动的环境

Snowflake 等提出的 AWM 按以下步骤合成环境 [111]：

1. 从 100 个种子领域生成约 1000 个有状态的应用场景，例如电商、金融、旅行；只保留需要增删改查的场景，并用 embedding 去重。
2. 为每个场景生成 10 个可以用 API 完成的任务，共 1 万个。
3. 设计 SQLite 数据库并填充样例数据，保证每个任务都能从初始状态完成。
4. 用 Python 实现工具接口，通过 MCP 暴露，平均每个环境约 35 个工具。

验证分两步：先比较执行前后的数据库状态，再由 LLM 评审给出最终结论。生成的代码出错时，把报错反馈给 LLM，最多重试 5 次；每个组件的成功率约 85–88%。用其中 526 个环境、3,315 个任务做 RL，Qwen3-8B 在 BFCLv3 上从 53.83 升到 65.94。

ScaleEnv：带依赖图的环境

美团的 ScaleEnv 从领域关键词出发，按以下步骤构造环境与任务 [125]：

1. 生成带前置条件与后置条件的工具 schema；
2. 数据库 agent 从工具推导出数据库结构；
3. 工具代码经过程序化测试，失败的由调试 agent 修复；
4. 按数据流、前置条件与共享表，建立工具之间的依赖图；
5. 从图上采样工具链作为参考解，构造支持它的初始数据库状态，并加入干扰记录；
6. 由参考解与最终状态生成用户意图，保证意图有据可依；
7. 奖励由规则检查器按字段类型比较最终数据库状态。

训练领域与评测领域不重叠，模型仍在 τ^2-Bench 与 VitaBench 上提升，作者给出了“领域越多，泛化越好”的环境扩展曲线。

DeepSeek-V3.2：难解易验

DeepSeek-V3.2 的通用 agent 环境，由一个环境合成 agent 在沙箱中构建 [114]。它先收集数据，用 Python 函数构建任务专属的工具集；再生成“难以求解、容易验证”的任务，每个任务带一个求解函数和一个验证函数；然后逐轮提高难度，需要时补充工具；最后只保留 DeepSeek-V3.2 在 RL 中 pass@100 不为零的任务。最终得到 1,827 个环境、4,417 个任务。

“难解易验”是合成任务的核心原则：验证器必须比求解器简单得多，否则奖励本身就不可靠。

Kimi K3：白盒环境与长时程工作流

Kimi K3 的环境设计有三个新特点 [14]。

第一，白盒统一环境。agent 框架由可配置的模块组合而成：工具、系统提示、上下文管理、技能、记忆、子 agent。每组任务采样不同的框架，包括 Kimi Code、Claude Code、Codex、OpenClaw 与 Hermes。8.6 节的 SFT 数据用多个框架与教师，目的也在这里：让模型学会任务本身，而不是某一个框架的习惯。

第二，知识图引导的任务合成。agent 从粗粒度的种子节点出发，构建一个由粗到细的概念有向无环图；采样节点及其祖先节点作为网页查询，检索论文、博客与代码仓库，再把检索结果转成任务。任务的覆盖面由图的结构控制，而不是依赖 prompt 的随机性。

第三，长时程与带隐藏验证器的任务。个人助理任务用模拟的 Gmail、Notion、Slack 与 Canvas，构造持续多个模拟日、包含几十个相互依赖事件的场景，一次 rollout 可能有数千次工具调用、数百万 token 的上下文，每个事件由确定性规则或 LLM 评估。自主执行任务给每个任务设定初始状态、目标、工具动作空间与执行预算，由一个独立的验证器检查最终的环境状态，而不是听信 agent 的自我报告；公开的验证器给出诊断反馈，隐藏的验证器测试留出场景，有限预算内的提交带惩罚。网页开发任务由专家整理，在多种 agent 框架下运行，奖励结合确定性检查（功能与像素级相似度）与模型评判；项目无法构建、运行时报错，或伪造产物，奖励直接归零。

公开验证器加隐藏验证器的设计，把第四章去污染的思想引入了 RL：agent 可以从公开反馈中学习，但只有隐藏场景上的表现决定最终奖励，从而防止它过拟合公开测试。

开放的环境库

环境也在走向开放与标准化。Prime Intellect 的 Environments Hub 在 2026 年 8 月超过了 2,500 个开源训练环境，分属 12 个类别，全部基于其开源的 verifiers 库构建；该库统一了环境创建、奖励函数、工具集成与评测框架 [126]。

10.8 搜索与深度研究任务

搜索任务的难点在两处：题目要足够难，难到现有模型都答不出；上下文会长到装不下。

难题的合成

DeepSeek-V3.2 的搜索 agent 数据，先采样长尾实体，再用多 agent 管线通过搜索构造问答对；然后让多个回答 agent 各自给出候选答案，由验证 agent 只保留“标准答案正确、且所有候选都答错”的样本 [114]。最终得到 50,275 个任务。“所有候选都答错”保证了题目对现有模型确实困难，这与 10.2 节按通过率筛选是同一逻辑，只是在构造阶段就完成了。

其他构造方式各有侧重：UI-TARS-2 通过模糊化维基事实、串联超链接来合成信息检索问题 [127]；QUEST 用树形 rubric 与打分脚本构造可验证的研究任务（9.4 节）[108]；GLM-5V-Turbo 引入“视觉跳转”约束，要求中间的推理跳转必须经过视觉信息，防止模型只靠文本捷径解题（11.6 节）[109]。

上下文管理也是数据设计的一部分

深度搜索会产生超长的上下文，RL 中必须决定上下文满了之后怎么办。Nemotron 3 Ultra 的搜索专家选择全部丢弃 [17]；QUEST 做结构化压缩，把历史压缩成“可信事实、未核实的说法、不确定项加下一步建议”三部分 [108]；Qwen3.5 的搜索 agent 在 256K 窗口中折叠上下文，超过阈值后剪掉早先的工具返回 [8]；GLM-5.2 则把压缩后的长轨迹切成多个子轨迹，全部作为可训练轨迹，并用 token 级损失处理长度不均 [113]。

无论哪种策略，训练与推理必须使用同一种上下文管理方式，否则策略学到的状态分布与部署时不一致。

10.9 GUI 与办公环境

GUI 任务让 agent 看着屏幕操作软件。它的环境最重，验证也最难。

UI-TARS-2：沙箱与数据飞轮

UI-TARS-2 的 GUI 环境，是数千台运行 Windows、Ubuntu、Android 的云虚拟机组成的集群；共享文件系统让 GUI 操作与终端命令作用于同一批文件 [127]。

它有三类 RL 任务。GUI 浏览任务用模糊化的维基事实与串联的超链接合成问题，由 LLM 评审，并过滤掉简单题。GUI 通用任务从 690 个网站合成单页任务，这类任务没有程序化的验证器，UI-TARS-2

用自身作为生成式的结果奖励模型，读取文本历史与最后 5 张截图做判断，在 300 条人工标注的轨迹上 F1 为 83.8。游戏任务使用公开的 HTML5 小游戏，以及由 LLM 合成、暴露状态变量的游戏，用 JavaScript 读出分数与关卡，给出二元奖励。

它的数据飞轮是这样转的：每一轮，最新的 RL 模型生成轨迹，验证通过的轨迹进入 SFT 数据，失败的进入继续预训练数据。继续预训练的数据还来自“边做边说”的标注：标注员在自己的电脑上操作并口述思路，因为事后给已录制的动作补写推理，效果不好。SFT 数据则由人机协同平台产生：模型给出每一步的思考与动作，标注员接受或改写，保证数据是 on-policy 的。

TreeCUA：树形探索复用前缀

美团的 TreeCUA 把 GUI 轨迹组织成一棵树 [128]。共享的早期步骤只探索一次，通过重置环境、重放动作历史回到分支节点，并用截图校验状态；探索、验证、总结、评估四个 agent 协作；应用类别的知识引导探索，全局记忆避免不同的树重复相同的开头。在分支点上，两条合法但通往不同目标的路径直接构成 DPO 偏好对，不需要额外标注。

从约 10 万条轨迹中，它筛出 5 万条长轨迹，得到 70.8 万个步级样本与 1.6 万个 DPO 对，7B 模型在 OSWorld-Verified 上达到 34.6%。

办公任务

办公任务的数据大多来自专业人士。Nemotron 3 Ultra 用 AfterQuery 的数据构建 GDPval 风格的办公任务 [17]；MiniMax 请金融、法律与社会科学领域的资深从业者定义需求、给出反馈，参与构建 Word、PowerPoint、Excel 任务的训练数据 [116]；Kimi K3 构造了投资银行、数据分析、法律等专业工作流任务，长度从几十步到几百步不等 [14]。办公任务的难点在于验证：产出是一份文档，无法用单元测试检查，只能依赖专家 rubric 与 GRM（第九章）。

10.10 多智能体与并行

当一个任务可以拆给多个子 agent 并行完成，RL 要学的就不只是“怎么做”，还有“怎么分”。

只训练编排者

Kimi K2.5 的 Agent Swarm 只训练编排者，子 agent 冻结不动，并且先用较小的子 agent 来降低成本 [18]。冻结子 agent 的好处，是环境对编排者来说是稳定的：同一个分派决策，得到的子任务结果分布不会随训练漂移。

任务数据是合成的 prompt，覆盖广度搜索、深度搜索、长文档分析与大规模文件下载。prompt 中不提示“要并行”，让编排者自己学会何时并行。数据侧的关键是让任务本身具有可并行的结构，例如一个需要同时查 20 家公司财报的问题；串行就能轻松完成的任务，学不出“怎么分”。

用脚手架奖励起步

奖励写成：

$$r = \lambda_1 r_{\text{parallel}} + \lambda_2 r_{\text{finish}} + r_{\text{outcome}},$$

其中 r_{parallel} 奖励并行地启动子 agent，r_{finish} 是子 agent 的完成率，r_{outcome} 是任务的最终结果；辅助权重 λ_1、λ_2 逐步退火到零。

两个辅助项是互相制衡的：只奖励并行，编排者会学会无意义地派出大量子 agent；完成率项则抑制这种行为，要求派出去的子 agent 能完成任务。两个辅助项最终都退火到零，保证最终优化的目标只有任务结果，辅助项只是早期的“脚手架”。

10.11 从已有轨迹中挖掘 RL 数据

端到端的 agent RL 很贵：每次 rollout 都是一条完整的长轨迹，而大部分回合的结果没有区分度。已有的 SFT 轨迹里，其实藏着值得做 RL 的地方。

PivotRL：只在关键回合上做 RL

PivotRL 从 SFT 轨迹中挑出“枢轴”回合 [129]：

1. 把轨迹中的每个 assistant 回合都作为候选；
2. 用冻结的参考策略在每个候选状态上做几次局部 rollout，由验证器打分；
3. 只保留奖励方差不为零、且平均奖励偏低的回合，即“有对有错，而且还难”的回合；
4. 验证器接受任何局部可接受的动作，而不只是演示中的那一个。

在 SWE-Bench 上，它达到了与端到端 RL 相同的 32.67%，rollout 回合数少约 4 倍，挂钟时间少约 5.5 倍。在八个非 agent 基准上，同样数据的 SFT 平均下降 9.83 分，PivotRL 几乎不变（+0.21）。

PivotRL 已用于 Nemotron 3 Super 的 agent 后训练，Nemotron 3 Ultra 的终端、对话式工具调用与软件工程专家也都用了它 [17]。它是 10.2 节通过率筛选在 agent 场景下的推广：筛选的单位从“题”变成了“轨迹中的一个回合”。

RL 产出回流为数据

RL 不只优化策略，它也是一台高质量数据的生成器。Kimi K2.5 的视觉 RL 轨迹，通过拒绝采样进入自我改进的 SFT 管线 [18]；Seed1.5-VL 做了四轮“RL 模型拒绝采样 → SFT”的迭代 [117]；UI-TARS-2 把通过的轨迹送进 SFT、失败的送进继续预训练 [127]；MAI-Thinking-1 的自蒸馏存档点，则定期在 RL 的 rollout 上重新做 SFT（8.4 节）[21]。

10.12 专家 RL 与 on-policy 蒸馏的数据流

多个领域的 RL 很难放进一次训练里同时做好。2026 年的主流做法是：先分领域训练专家，再用 on-policy 蒸馏（OPD）把它们合成一个学生。

OPD 的目标与 RL 的关系

学生在自己生成的序列上，逐 token 对齐教师：

$$\mathcal{L}_{\text{OPD}} = \mathbb{E}_{y\sim\pi_S(\cdot|x)}\left[\sum_t \text{KL}\left(\pi_S(\cdot \mid x, y_{<t}) \,\|\, \pi_T(\cdot \mid x, y_{<t})\right)\right],$$

其中 π_S 是学生策略，π_T 是教师策略，y 是学生自己采样的回答。如果只用采样到的那个 token y_t 来估计逐位置的反向 KL，就得到一个逐 token 的“奖励”：

$$r_t = \log \pi_T(y_t \mid x, y_{<t}) - \log \pi_S(y_t \mid x, y_{<t}).$$

于是 OPD 可以看作一种特殊的 RL：奖励不是只在结尾才有的一个比特，而是每个 token 都有的稠密信号，优势就是教师与学生在这个 token 上的对数概率之差。它与 RL 共享同一套 rollout 基础设施，只是把验证器换成了教师模型。

这个关系说明了 OPD 为什么便宜。Thinking Machines 的实验中，用 OPD 把一个数学推理模型训练到 AIME'24 70% 的准确率，用了 1,800 GPU 时，同等效果的 RL 基线用了 17,920 GPU 时，相差约 10 倍 [130]。

采样估计与全词表匹配

逐 token 的 KL 有两种实现方式。采样估计只用采样到的 token 计算 r_t，实现简单，但方差大；Kimi K3 用的是裁剪后的对数比，并以阈值 $R_{\max}$ 为上界，它报告说更细的 top-k 蒸馏并没有明显优势 [14]。全词表匹配在每个位置计算完整的词表分布 KL，信号更准，但要同时取得教师在整个词表上的 logits。

DeepSeek-V4 采用全词表匹配，并用三个办法控制显存与吞吐：缓存每个教师最后一层的隐状态，再用该教师的输出头重建 logits；按教师编号对样本排序，一次只加载一个输出头；用自定义的 TileLang kernel 计算精确的 KL [7, 104]。

各家的专家与合并

各家的专家划分与合并方式如下：

模型	专家与合并方式
DeepSeek-V4 [7]	为数学、代码、agent、指令遵循等领域分别训练专家，每个专家先 SFT、再 GRPO，再用十余个教师做 OPD，前后都没有单独的统一 RL 阶段
MiMo-V2-Flash [115]	多教师 OPD（MOPD）：教师可以是 RL 模型、SFT 模型，甚至学生自己；学生同时得到教师的 token 级奖励与可验证的结果奖励
Kimi K3 [14]	通用、通用 agent、代码 agent 三个领域 × 低、高、最高三档推理强度，共 9 个专家，合并成一个模型
Nemotron 3 Ultra [17]	两轮 MOPD，最大生成长度 192K，每批 1024 个 prompt
GLM-5.2 [113]	并行地把十多个专家合并到一个模型中，用时约两天
Keye-VL-2.0 [91]	用 13 个经过 RL 训练的领域教师做 MOPD
Nemotron-Cascade 2 [131]	教师不来自外部，而是 RL 过程中各领域表现最好的中间 checkpoint，用来修复后续 RL 造成的退化

合并的效果并不总是“学生不超过教师”。MiMo-V2-Flash 合并之后，AIME 2025 从 89.3 升到 94.1，超过了最好的教师（93.9）；SWE-Bench Verified 从 67.8 升到 73.4，略低于最好的教师（74.2）[115]。

合并之后，单 rollout 的 RL

GLM-5.2 在长时程 agent 阶段放弃了组相对的优化，改用带价值网络的 SAO（Single-rollout Asynchronous Optimization）[132]。每个 prompt 只采一条 rollout，轨迹一完成就可以训练，不必等一整组；训练一个真正的价值网络（critic），其注意力层冻结、MoE 层可训练；用 token 级的双侧裁剪屏蔽偏离过大的梯度；计算优势时跳过环境返回的 token，因为那些 token 不是模型生成的。

在 Qwen3-30B-A3B 上的对照中，SWE-Bench Verified 从基础模型的 23.0、GRPO 的 27.0 升到 29.8；标准 GRPO 在约 160 步时崩溃，SAO 稳定训练到约 1000 步。背后的数据原因是：长时程 agent 任务中，同一个 prompt 的不同 rollout 长度与结构差异极大，“同组比较”的前提不再成立。

数据层面的含义

1. **prompt 配比就是教师配比**：每个 prompt 由哪个教师监督，取决于它所属的领域，所以 prompt 池中各领域的比例，就是各教师的影响力。
2. **数据集只提供 prompt**：样本由学生实时生成，第五、八章的配比方法只作用在 prompt 层面。
3. **教师决定上限**：学生在某领域能达到的水平，取决于对应教师的水平。MiMo 的部分指标超过最好的教师，可能来自结果奖励与多领域之间的迁移。
4. **边界题的归属**：落在两个领域之间的 prompt，需要明确由哪个教师负责，或者按权重组合多个教师。

本章思考与进阶练习

10.1 难度区间

题目 prompt 池中 40% 的题通过率约为 0.02，40% 约为 0.5，20% 约为 0.98。$G = 8$ 时，有效组的比例是多少？G 加倍后呢？

解题思路 $G = 8$ 时为 $0.4 \times 0.149 + 0.4 \times 0.992 + 0.2 \times 0.149 \approx 0.486$；$G = 16$ 时约 0.566。rollout 翻倍，有效组只多了约 8 个百分点。更有效的做法是剔除那 20% 的简单题，把难题换成中等难度的题，或者为难题提供提示。

10.2 验证器的假阳性

题目 代码 RL 中，测试的假阳性率为 10%。若策略写对程序的概率为 q，奖励的期望是多少？这会怎样影响训练？

解题思路 $\mathbb{E}[r] = q + 0.1(1 - q)$。当 $q = 0.1$ 时期望奖励为 0.19，近一半来自假阳性，策略会被引向“刚好能过弱测试”的程序。对策是补充测试、用错误解检验测试的区分力，并抽查通过率突增的题。

10.3 进阶：环境的多样性

题目 预算固定时，100 个领域 × 每领域 100 道题，与 1000 个领域 × 每领域 10 道题，你会选哪个？需要什么实验？

解题思路 ScaleEnv 的扩展曲线与 Kimi K3 的多框架采样都支持“更多样”，但每个领域的题太少时，模型学不到领域内的工具组合规律，环境构造的固定成本也更高。实验上，固定总题数，扫描“领域数 × 每领域题数”的多种配置；评测同时包括训练领域内的新题与全新的领域；并记录环境构造成本，把它算进比较。

第十一章　多模态理解数据

视觉语言模型（VLM）的数据要同时完成两件事：让视觉编码器与语言模型对齐，再让模型掌握 OCR、定位、空间、STEM、视频、GUI 等专项能力，而每一项专项能力都有自己的构造方法。2026 年的趋势有三条：程序化渲染与“代码 ↔ 图像”的配对，正在大规模替代人工标注；各类数据开始按 token 预算与规模律来规划；只靠数据整理本身，就能把小模型推到接近前沿的水平。本章先讨论各阶段的数据与配比，再依次讨论 caption、交错数据、OCR 与文档、定位与空间、STEM、视频与 GUI agent 的数据。下一章转向视觉生成的数据。

11.1　各阶段的数据与配比

多模态训练通常分为对齐、预训练、长上下文三类阶段。每个阶段解冻的部分不同，所用的数据也不同。

阶段划分

Qwen3-VL 的预训练分四个阶段 [19]：

阶段	训练的部分	token	序列长度
S0 对齐	只训练 merger	约 67B	8K
S1 多模态预训练	全部	约 1T	8K
S2 长上下文	全部	约 1T	32K
S3 超长适配	全部	100B	262K

Keye-VL-2.0 的划分与之相近 [91]：先冻结 ViT 与语言模型、只训练投影层；再全参数训练约 1T token、32K 上下文；然后全参数训练约 2T token，上下文扩到 64K；最后把长上下文扩展到 256K，长样本与短样本按 1:1 混合。报告没有给出第一与第四阶段的 token 数。

阶段的逻辑都是一样的：先用少量数据把视觉特征接到语言模型上，再全参数地学多模态能力，最后拉长上下文。

纯文本数据与编码器的数据

有两类数据常被忽视。一是纯文本：多模态预训练中必须保留一定比例的纯文本，否则语言能力会退化，Seed1.5-VL 混入了约 5% 的纯文本 [117]。二是视觉编码器自身的数据：Kimi K2.5 的 ViT 训练阶段用约 1T token，只用 caption 损失、不用对比损失 [18]；GLM-5V-Turbo 的视觉编码器第一阶段用 80% 自然图像、10% 指令数据、10% 科学图像，第二阶段用 80 亿 token 的中英图文语料 [109]。

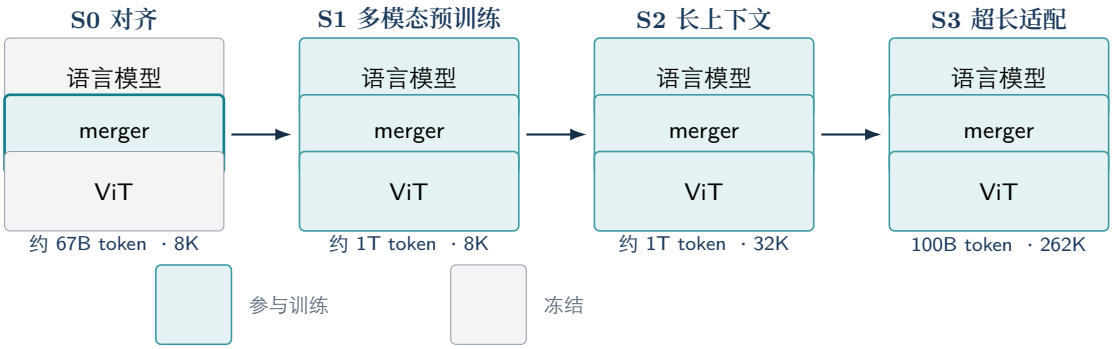


图 11.1 Qwen3-VL 的四个预训练阶段。读图：青色为参与训练的模块，灰色为冻结。S0 只训练 merger，把视觉特征接到语言模型上；之后全参数训练，并逐步拉长序列。[19]

按类别规划：多模态数据的规模律

Seed1.5-VL 观察到两条经验关系 [117]。第一，多数子类别的训练损失，随该类别训练 token 数 D_c 呈幂律下降；第二，在局部范围内，下游指标（ChartQA、InfographicVQA、RefCOCO 等）与该类别训练损失的对数近似线性相关。写成式子：

$$L_c(D_c) \approx a_c\, D_c^{-b_c}, \qquad \mathrm{Metric}_c \approx k_c - m_c \log L_c$$

两式合起来，下游指标随 $\log D_c$ 近似线性增长，斜率为 $m_c b_c$。这使配比可以按类别做预算：先在几个规模上拟合每个类别的 b_c 与 m_c，再比较“每多加一倍 token，指标涨多少”，把增量投到斜率最大的类别上。

口径上要注意：这是局部近似，只在拟合过的数据范围内成立；不同类别的指标量纲不同，比较斜率之前要先确认指标的单位。

只靠数据整理能走多远

DatologyAI 的 20/20 实验，固定模型架构、训练配方与计算量，只改变训练数据的整理方式 [133]。在 20 个公开 VLM 基准上，平均提升 11.7 个百分点；2B 模型比 InternVL3.5-2B 高 9.9 个百分点，训练计算只有后者的约 1/17；与 Qwen3-VL-2B 只差 1.8 个百分点，训练计算约为后者的 1/87；种子间各项能力的标准差还下降了约 67%。

这些结果只来自预训练阶段，而且作者是数据整理公司。即便如此，这组数字说明，多模态领域的数据工作远没有饱和。

开源数据的整理与去污染

Hugging Face 的 FineVision 把 200 多个开源数据集统一成 185 个子集，共约 1730 万张图像、2430 万个样本 [134]。去重用 SSCD embedding 比较图像，相似度阈值 0.95，宁可误删也不漏删；去污染对照 66 个公开基准，已有的开源混合中测试图像占 2–3%（例如 Cauldron 为 3.05%），FineVision 降到约 1.02%。

它的另一个发现与直觉相反：用两个 32B 模型从四个维度给每轮对话打 1–5 分，剔除低分样本并没有带来提升，全部保留反而最好。这与 8.4 节“SFT 重质量”的结论看似矛盾，原因在于场景不同：在千万级的视觉指令数据上，低分样本仍然提供了视觉覆盖，删掉它们的损失大于噪声的危害。评分过滤的阈值，必须在目标规模上验证。

后训练数据的规模

多模态 SFT 的数据量普遍远小于文本 SFT。Qwen3-VL 约 120 万条 [19]；Seed1.5-VL 的多模态 SFT 约 5 万条，训练两个 epoch，其中约 1.3 万条众包指令、约 3 万条从 150 万条开源数据中精选 [117]。视觉能力主要在预训练中建立，SFT 只负责对齐格式与交互方式，这与 8.8 节 Kimi K2.5“只用文本 SFT 就能激活视觉能力”的发现一致。

11.2 图像描述与重写 caption

caption 是视觉对齐最基础的数据。网页 alt-text 的噪声大，主流做法是重写；重写的难点，是不丢掉原文中的实体与事实。

以原网页文本为条件的重写

Qwen3-VL 的 caption 数据主要来自中英文网页图文对 [19]。它用微调后的 Qwen2.5-VL-32B 重写 caption，并以原网页文本为条件，保留原文中的专名与事实；在重写结果上做语义去重；再对视觉 embedding 聚类，找出稀疏的区域做定向增广。

以原文为条件，解决的是两种 caption 各自的缺陷：纯 VLM 生成的 caption 描述准确，却会丢掉实体名，只写“一座教堂”，不写“科隆大教堂”；原始 alt-text 有实体名，噪声却大。以原文为条件的重写兼顾了两者。Keye-VL-2.0 则用了“重写”与“重做”两种策略，并训练了一个 8B 的 caption 专家模型 [91]。

概念平衡与长尾

LLaVA-OneVision-1.5 把每张图匹配到 MetaCLIP 约 50 万个概念中最近的 top-K 个，按概念频率的倒数加权采样，得到 8500 万对数据（中文 2000 万、英文 6500 万）[135]。Seed1.5-VL 先在物种数据上做沙箱实验，发现限制常见概念的样本数能提升稀有概念的识别，但限得太狠会损害常见概念；最终的做法是，频率低于平均值一半的领域，复制其 alt-text 后并回数据集 [117]。

两者都是在 caption 数据上做 5.1 节的温度采样，只是采样的单位从“数据源”变成了“视觉概念”。

人工稠密描述

Molmo 的 PixMo-Cap 让标注员对着一张图口述至少 60 秒（后来延长到 90 秒），转写后作为稠密描述 [136]，覆盖 71.2 万张图像，产出约 130 万条 caption。口述比打字快得多、内容更细，也不受 VLM 风格的影响。在不依赖闭源模型的场景下，这是高质量 caption 的主要来源。

11.3 交错图文与书籍

caption 训练的是“一张图配一段话”，交错数据训练的是“在上下文中引用图像”，例如多图比较、图文对照阅读。

为什么需要交错数据

图文对里，每张图只对应一段紧挨着的文字；交错文档里，一张图可能被后面几段文字反复引用，多张图之间也要互相比较。模型从中学到的，是在长上下文里定位“这句话在说哪张图”，这正是多图问答与多模态上下文学习所需要的能力。

开源数据集给出了直接的证据。OBELICS 从 Common Crawl 中抽取了 1.41 亿个网页、3.53 亿张图像与 1150 亿个文本 token，用它训练的模型在多项多模态基准上优于只用图文对训练的模型 [137]。MINT-1T 把规模扩大到 1T 文本 token、34 亿张图像，约为此前开源数据集的 10 倍，并第一次纳入了 PDF 与 arXiv 论文；用它训练的模型与用 OBELICS 训练的相当 [138]。规模扩大十倍、效果只是持平，说明交错数据的瓶颈已经不在量上，而在图文相关性与文档质量上。

网页交错文档的清洗

产业做法正是把重点放在相关性上。Qwen3-VL 从中英文网站收集交错文档，用基于 Qwen 的领域分类器去掉广告、推广与标题党，并剔除纯文本段落以及图文弱相关的片段 [19]。

交错文档里最常见的噪声，是与正文无关的图：头像、广告横幅、装饰图标。它们不会让文本变差，却会教模型把无关的图当作上下文。按图文相似度剔除这类图，比按文本质量过滤整篇文档更有效。

书籍：逐页解析，合并成长序列

Qwen3-VL 用微调后的 Qwen2.5-VL-7B 逐页解析书籍，再把连续的页面合并成最长 256K token 的序列，并对长书设定最少页数与最低图文比 [19]。这类数据同时服务于多模态与长上下文两个目标，主要用在 S2、S3 阶段。

11.4 OCR 与文档解析

文字是图像中信息密度最高的部分。OCR 与文档解析的数据，正越来越多地由合成与程序化渲染提供。

OCR：真实数据加合成

Qwen3-VL 有约 3000 万条真实 OCR 样本，由粗到细地打伪标签：OCR 专用模型先标，Qwen2.5-VL 再修正，不需人工；并扩展到 39 种语言，另加约 3000 万条多语言合成样本 [19]。Seed1.5-VL 的规模更大，超过 10 亿条，包括 2 亿多张合成的文字密集图像、1 亿多条图表、5000 多万张渲染的表格，以及生成的 VQA 对 [117]。

合成数据的标签天然正确，可以任意控制字体、语言、版面与噪声；真实数据则负责覆盖渲染不出来的分布：手写、拍照造成的畸变、复杂背景。

文档解析：版面加区域识别

Qwen3-VL 的文档解析数据，先从 Common Crawl 收集 300 万个 PDF（10 种文档类型各 30 万），加上 400 万份内部文档；再由版面模型预测阅读顺序与区域，Qwen2.5-VL-72B 识别每个区域；最后输出两种格式：带元素坐标的 QwenVL-HTML，以及表格用 LaTeX 表示的 QwenVL-Markdown [19]。

这套“版面模型 + 逐区域识别”的流程，2.3 节已经在文本侧的 PDF 解析中出现过。高质量的抽取结果，既能进入文本预训练，又能作为 VLM 的文档解析训练数据。

从代码生成图像

程序化渲染的思路在 2025–2026 年快速扩展。PixMo-Docs 让 LLM 生成能渲染出图表、文档、表格、示意图的代码，共 25.5 万张图像，再基于代码而不是图像，生成 230 万个问答对 [136]。Kimi K3 把“代码与渲染结果”配对作为多模态数据，覆盖 SVG、3D 资产、网页、游戏与 CAD 图纸 [14]。GLM-5V-Turbo 在预训练中特别强调多模态编码数据，加入学科图像与对应 SVG 的配对数据，并报告它对下游的 STEM 解题有帮助 [109]。

代码就是图像的精确描述，问答与对齐的正确性由代码保证。这是 10.7 节“难解易验”原则在视觉数据上的应用：由代码生成图像很容易，从图像还原代码很难，而训练恰好需要后者。

11.5 定位、计数与空间

OCR 读出图里有什么字，定位回答东西在哪里。这类数据的关键，是把坐标表示成模型能直接生成的文本。

框、点与计数

Qwen3-VL 的框数据来自 COCO、Objects365、OpenImages、RefCOCO 系列，加上一条合成管线（Qwen2.5-VL 提取候选，Grounding DINO 与 Qwen2.5-VL 定位，过滤低置信度结果）；点数据来自 PixMo 等；计数分直接计数、基于框、基于点三种形式；坐标归一化到 [0, 1000] [19]。Seed1.5-VL 有开源框数据约 4800 万条、自动标注的框约 2 亿条、点约 1.7 亿条、计数约 800 万条，坐标归一化到 0–999 [117]。PixMo-Points 在 42.8 万张图上标注了 230 万个“问题–点”对 [136]。Kimi K3 则同时用绝对坐标与归一化到 [0, 1] 的坐标做监督 [14]。

把坐标统一归一化到一个整数区间，模型就能在任意分辨率下用同一套文本 token 表示位置，这是“定位即文本”的前提。代价是量化误差。归一化到 $[0, K]$ 时，每一格对应的像素宽度为

$$\delta = \frac{W}{K}$$

其中 W 是图像宽度。数值例子（教学假设：$K = 1000$）：1280 像素宽的截图，每格约 1.3 像素，最大误差约 0.6 像素，可以忽略；3840 像素宽的 4K 屏幕截图，每格约 3.8 像素，最大误差约 1.9 像素，对一个 16 像素宽的图标，误差约占其宽度的 12%。

GUI 与高分辨率文档上的小目标，最容易受这个误差影响。Kimi K3 同时监督绝对坐标与归一化坐标，可能就是为了兼顾“像素级精确”与“与分辨率无关”这两种使用场景。

三维与空间

Qwen3-VL 的空间数据包括关系、可供性与动作规划类问题；3D 数据把公开的室内外场景改写成 VQA，输出 JSON 格式的 9 自由度框，并按 Omni3D 统一相机坐标 [19]。Seed1.5-VL 有相对深度排序 32 亿 token、绝对深度估计 1800 万对、3D 定位 77 万对 [117]。

读表这样的单项能力，也可以用渲染数据专门补：PixMo-Clocks 用约 50 种表款、约 16 万种表盘样式，随机设定时间，渲染出 82.6 万个样本 [136]。

11.6 STEM 与多模态推理

STEM 数据既要“看得清”（读出图形元素），又要“想得对”（基于图形推理）。两者需要不同的数据。

感知：程序化渲染

Qwen3-VL 用程序渲染几何图形，生成 100 万条点定位、200 万条感知 VQA 与 600 万条图表描述，并经过生成与模型验证 [19]。先让模型看清图形元素，再谈推理。

推理：习题与长 CoT

Qwen3-VL 收集了 6000 多万道 K-12 与本科习题，经过清洗、中英互译与标准化，另有 1200 多万条长 CoT 多模态样本，来自强推理模型的 rollout，经规则与模型验证 [19]；Seed1.5-VL 有清洗后的 1 亿多道 K-12 习题 [117]。

剔除文本捷径

很多“视觉数学题”的题干已经包含了全部信息，不看图也能解。这类题训练的是文本推理，不是视觉推理。Qwen3-VL 用纯文本的 Qwen3-30B 测试，能解出的视觉数学题就从冷启动数据中剔除 [19]；Kimi K2.5 的视觉 RL 只选“必须看图”的 STEM 题 [18]。GLM-5V-Turbo 把这一思路推广到搜索任务：用“视觉跳转”约束要求中间的推理跳转必须经过视觉信息；OCR 搜索数据则要求先从图表、地图或海报中切出实体，再去搜索 [109]。

RL 数据

视觉 RL 的题库规模差别很大。Seed1.5-VL 有 100 多万道图像类 STEM 题，保留通过率 0–75% 的，另有 2 万多个合成的视觉谜题 [117]；Keye-VL-2.0 的视频 RL 只有约 3.1 万个样本，工具调用 RL 覆盖 150 多个模拟的 API 领域 [91]。GLM-5V-Turbo 的联合 RL 覆盖 30 多类感知、推理与 agent 任务，在统一的 VLM RL 环境中运行，每个样本都带数据来源标签 [109]。来源标签看似是个工程细节，却决定了能否按来源统计奖励与 pass@k，进而发现哪一类数据在训练中已经饱和、哪一类在拖后腿。

视觉 RL 还带来了跨模态的收益：Kimi K2.5 做完视觉 RL 后，纯文本的 MMLU-Pro 从 84.7 升到 86.4 [18]。

11.7 视频理解数据

视频数据在图像数据之上多了一个维度：时间。caption 要带时间戳，长度要分级，帧的预算要随时长调整。

时间对齐的 caption

Qwen3-VL 由短到长合成带时间戳、交错的故事级与事件级描述，做时空定位，并按标题、时长、类别等元信息平衡来源 [19]。Keye-VL-2.0 按场景生成稠密 caption，每个场景有起止时间戳与描述，外加一段全局概述 [91]。

分级的视频长度

Keye-VL-2.0 把视频数据分为三级：15 秒片段，每个样本 24K 视频 token；最长 15 分钟的视频，64K token；最长 2 小时的视频，180K token [91]。每帧作为一张带时间戳前缀的图像编码。

每段视频的像素预算随时长自适应。报告在 256、512、1024、2048 秒设了四个阈值，对应基础预算的 1/8、1/4、1/2 与全部，超过 2048 秒的视频用满基础预算 [91]。这个系数作用在整段视频的预算上，而不是每一帧：2048 秒以内，时长翻倍、预算也翻倍，每秒分到的像素大致不变；超过 2048 秒，预算封顶，视频越长，每帧越粗。

数值例子（教学假设：均匀抽帧，视频 token 全部用于画面）：15 秒的片段按 2 fps 抽 30 帧，24K token，每帧约 800 token；2 小时的视频按 0.5 fps 抽 3600 帧，180K token，每帧只有约 50 token。两者每帧的 token 相差 16 倍。长视频学到的是事件与情节，细粒度的文字与小物体只能靠短片段的数据来教，所以三级数据缺一不可。

流式视频数据

Seed1.5-VL 为流式场景构造了三类数据：边播边描述的交错 caption、在合适时刻主动发言的推理，以及实时解说 [117]。流式数据的关键是时间对齐：每段文本都要对应它“应该被说出”的时间点，否则模型学到的是看完再说，而不是边看边说。

视频的 token 预算

视频 token 的增长远快于图像，压缩方式决定了能训练多长的视频（1.5 节）。Kimi K2.5 每 4 帧一组做时间池化 [18]。数据构造时，应按目标压缩率来选片段长度与帧率，而不是先取视频再截断。

11.8 GUI 与多模态 agent

agent 数据要求模型在看懂界面之后采取行动。GUI 数据从感知开始，一直延伸到多步轨迹与“边看边操作”的推理。

GUI 的数据层级

GLM-5V-Turbo 把 GUI 数据分成元素感知、GUI 定位、单步动作预测与轨迹级动作预测四个层级，同时用于 SFT 与 RL [109]。另有一部分“GUI 评判”数据，专门针对推理错误：看错界面细节、认错目标元素、选错下一步动作。

Qwen3-VL 的 GUI 数据覆盖桌面、移动端与网页，包括元素描述、稠密 caption 与定位；多步轨迹来自一个自进化框架，再经人工审核 [19]。UI-TARS-2 的数据来自“边做边说”的标注与人机协同平台（10.9 节）[127]。

多模态函数调用与搜索

Qwen3-VL 的函数调用数据从图像出发生成 query 与函数定义，再采样调用与返回，剔除格式错误的轨迹；搜索数据则是使用图像搜索与文本搜索工具的多模态事实查询轨迹 [19]。GLM-5V-Turbo 把单轮的“UI 截图生成代码”作为 RL 任务，作为更复杂的多轮编码的基础 [109]。

带图思考

Qwen3-VL 分两步构造“边看边操作”的推理数据 [19]。先用约 1 万条定位样本训练一个 Qwen2.5-VL-32B 视觉 agent，让它按“思考 → 操作 → 分析反馈 → 回答”的循环工作，再做多轮工具集成 RL；然后从这些 agent 中蒸馏出约 12 万条多轮交互，用作 Qwen3-VL 的冷启动 SFT 与工具集成 RL 数据。奖励分三项：答案准确性、多轮推理质量，以及防止模型退化成只调用一次工具的工具调用奖励。

GLM-5V-Turbo 还发现，冷启动轨迹的多样性能帮助 RL 发现更好的策略 [109]。Kimi K2.5 走了另一条路：图像操作通过 IPython 代码完成，只用纯文本 SFT 就能激活（8.8 节）[18]。

本章思考与进阶练习

11.1 重写 caption 的幻觉

题目 用 VLM 给 1 亿张网页图片重写 caption，抽检发现 5% 的 caption 编造了图中不存在的文字。在不做人工全检的前提下怎样控制?

解题思路 对 caption 中的文字片段，用 OCR 在原图上核对，核对不上的删除或降权；以原 alt-text 与网页文本为条件重写，减少凭空生成；先识别图中的文字、再描述画面（12.2 节 Z-Image 的做法）；最后以幻觉率为信号，对 caption 模型做 DPO。

11.2 概念平衡的采样权重

题目 “猫”有 100 万张，“水獭”有 1 万张，“鸮鹦鹉”有 100 张。按频率的倒数采样，设 3 次重复上限，总预算 30 万张。三者各得多少?

解题思路 按频率倒数采样，三者应各得 10 万张；但重复上限让鸮鹦鹉最多得 300 张、水獭最多得 3 万张，剩下的 269,700 张全部给了猫。纯倒数采样在长尾上会立刻碰到重复上限。要覆盖长尾，只能靠合成或检索补充稀有概念的数据，或者改用 $1/\text{freq}^{\alpha}$（$\alpha < 1$）的平滑权重。

11.3 进阶：检测文本捷径

题目 一个多模态推理基准上分数很高。怎样判断模型是在做视觉推理，还是在利用文本捷径?

解题思路 把图像换成空白图或随机图，看分数下降多少；用纯文本模型直接作答，测量不需要图像的题目所占的比例；只修改图中的关键数值、题干不变，看答案是否随之改变。在训练数据侧，可以按 Qwen3-VL 的做法剔除纯文本模型能解出的题，或者按 GLM-5V-Turbo 的视觉跳转约束构造题目。

第十二章　视觉生成数据：图像、视频与音视频

生成模型对数据的要求，与理解模型正好相反。理解模型从图像中读出信息，低质量的图像也能提供识别信号；生成模型要根据文字画出图像，它会忠实地复现训练数据中的模糊、水印与构图，也会把描述中每一处遗漏或错误，变成推理时的“不听话”。所以生成数据的核心是两条管线：视觉质量管线负责切分、过滤、去重与平衡，描述管线负责 caption 体系与 prompt 改写，两者都按分辨率分阶段逐级收紧。本章先讨论图像的数据基础设施与 caption 体系、编辑数据，再讨论视频的漏斗、caption 与分辨率阶段，然后是音视频联合数据、SFT 与 RL 数据，最后是生成数据的飞轮。下一章转向语音、具身与世界模型。

12.1　图像数据基础设施

图像生成的数据瓶颈，正从“过滤掉什么”转向“缺什么概念、该补什么”。

Z-Image 的四个引擎

Z-Image 把图像数据的整理拆成四个组件 [52]。

数据画像引擎为每张图计算元信息与各类质量分，包括分辨率、pHash、压缩伪影、模糊、水印、美学分、AIGC 检测、NSFW 与 CN-CLIP 图文一致性（3.9 节）。

跨模态向量引擎在 k-NN 图上用社区发现做语义去重，十亿条约需 8 张 H800 运行 8 小时（4.3 节）；它还支持按文本检索图像，用来补齐概念空缺，或者找出导致模型失败的数据并剪除。

世界知识拓扑图以维基实体与超链接为骨架，剪掉低 PageRank 的节点，以及 VLM 判断“难以可视化”的概念；再补入内部 caption 中出现的概念，组织成层级分类，人工上调用户 prompt 中的高频概念，最后用 BM25 与层级关系计算每个概念的采样权重。

主动整理引擎以 Z-Image 自身为诊断器，找出它的长尾短板，例如“松鼠鳜鱼”这道菜。找到之后，先用知识图与奖励模型选出平衡的子集，由 captioner 与奖励模型打伪标签，再由人工与 AI 共同核验、被拒的样本人工修正；核验后的数据，又反过来用于训练 captioner 与奖励模型。

前两个引擎回答“数据有多好、有多重复”，后两个回答“缺什么、该补什么”。

合成图像必须剔除

Wan 发现，训练数据中混入 AI 生成的图像，即使比例不到 10%，也会损害效果 [139]。生成模型学习这些图像，等于在学上一代模型的纹理与构图偏差。这与 3.6 节、6.8 节讨论的 AI 文本问题是同一个机制，在视觉上表现得更快；Z-Image 的画像引擎为此专门设有 AIGC 分类器 [52]。

文字渲染数据

生成图中的文字是最难的能力之一，主要靠合成数据补。Wan 在白底上合成了数亿张含汉字的图像，同时收集真实的文字图像，由 OCR 模型与 Qwen2-VL 生成描述 [139]；Qwen-Image-2.0 为幻灯片、

海报、漫画、教学材料等文字密集的图像设计了专门的 caption 模板 [53]；Z-Image 则在最后的 SFT 阶段大幅降采样渲染出来的合成文字 [52]。合成文字教会了字形，但它的版面单调，到后期要让位于真实的设计图。

12.2 图像 caption 体系

caption 决定模型能“听懂”多少。不同类型的图像，需要不同的描述方式。

多粒度的 caption

Z-Image 用一个全能的 captioner，为每张图生成五种 caption：长、中、短三种描述，标签，以及模拟的用户 prompt [52]。它有两个关键设计：先做 OCR、再描述画面，文字保留原语言；以来源网页的元信息为条件补入世界知识，减少实体名的幻觉。

按图像类型选模板

Qwen-Image-2.0 用四类 caption 模板 [53]。通用模板用于照片、艺术与一般设计，描述物体、上下文、空间关系与可见文字；文字密集模板用于幻灯片、漫画、海报、教学材料，重点是文字抽取、版面与符号；知识型模板为需要背景知识的图像补充背景信息；结构化模板用于示意图、流程图、关系图，显式列出实体、属性与关系。用一套模板描述所有图像，必然会在某些类型上丢失信息。

训练 caption 与用户 prompt 的差距

训练 caption 长而密，用户 prompt 短而模糊。弥合这个差距有两种做法。训练侧混入短 caption、标签与模拟的用户 prompt，即 Z-Captioner 的做法。推理侧则把用户 prompt 改写成训练 caption 的风格：Seedance 1.0 的改写器基于 Qwen2.5-14B，先在人工标注的改写对上做 SFT（I2V 与 T2V 分开），再用 LoRA 做 DPO，以“正确改写 / 错误改写”的配对来抑制幻觉 [140]。

改写器本身也是一份数据工程：它的训练数据，决定了用户的意图能否被忠实地翻译成模型能理解的描述。

12.3 编辑数据

编辑能力建立在生成能力之上，数据也建立在生成数据之上：编辑数据大多由生成模型、视频帧与渲染合成而来。

编辑数据从哪来

Z-Image 的编辑数据有四个来源：多个专家编辑模型的输出；同一原图的不同编辑版本之间两两配对；同一视频中的不同帧，经 CN-CLIP 相似度过滤后，提供自然的姿态与视角变化；以及渲染的文字，用于文字编辑任务 [52]。编辑指令分三步合成：先分别描述原图与目标图，再分析两者的差异，最后写出编辑指令。先描述、再比较、后生成，比直接让模型看两张图写指令更准确。

Qwen-Image-2.0 的编辑任务分为两类 [53]：单图编辑包括属性修改、背景替换、风格迁移、文字编辑、修复与结构感知编辑；多图编辑包括基于参考的生成、主体一致性与组合合并。

比例随阶段变化

Qwen-Image-2.0 中文生图与编辑数据的比例，预训练为 0.9 : 0.1，继续预训练与 SFT 为 0.7 : 0.3 [53]；Z-Image 在继续训练中，文生图与图生图的比例为 4:1 [52]。早期以生成为主，后期再提高编辑数据的比例。

参考输入也在变多。Seedance 2.0 把参考输入扩展到最多 3 段视频、9 张图像与 3 段音频 [141]。这意味着参考类的训练数据，不仅要覆盖多种模态，还要覆盖多个参考之间的组合关系：哪一份参考提供主体，哪一份提供动作或风格，它们怎样共同约束目标（12.5 节）。

12.4 视频数据漏斗

视频管线在图像管线之上多了两件事：切分镜头，以及评估运动。

各家的管线

HunyuanVideo 1.5 的过滤链依次为 [38]：

1. 原始素材超过 1000 万小时，用 PySceneDetect 加自研算子切成 2–10 秒的片段；
2. 用转场分类器剔除残留的转场；
3. 裁掉字幕、logo 与水印，裁剪后保留面积不足 60% 的片段丢弃；
4. 去除黑边、拼接伪影、宫格拼图与静态片段；
5. 由画质与美学模型打分。

最终得到 8 亿个片段。

Seedance 1.0 的管线结构相同，细节上有三处不同 [140]：按镜头边界切分，每段最长 12 秒；检测 logo、水印与字幕后自适应地裁剪画面；去重时按 embedding 聚类，每个近重复簇只保留质量最高的一段；最后对过度代表的类别降采样、不足的类别上采样，并定向采集新数据。Cosmos 从 2000 万小时原始视频中，整理出约 1 亿个 2–60 秒的片段，每 256 帧由 VLM 写一次描述，并做语义去重 [142]。LAION-BVD 是开放的版本（2.6 节）[37]：从 CC 的链接图中找到视频 URL，下载约 1000 万小时，剔除过短与过长的视频，按场景切分并去掉静止片段，再用小型 VLM 与音频模型写 caption。

去重的保留规则，生成数据与第四章不同：第四章按时间保留最新的副本，生成数据则保留质量最高的版本。同一素材的多个版本中，分辨率最高、没有字幕与水印的那一份才有价值。

Wan：按簇打分，按运动分档

Wan 有两个做法值得单独讨论 [139]。第一是按簇打分：先把数据聚成 100 个簇，避免小而重要的类别在全局阈值下被整体删掉，每个簇内人工打 1–5 分，再用专家模型给全量数据打分。第二是按运动分档：把运动质量分为六档，最优运动与中等运动的片段保留，静态（如口播）以及主要靠镜头运动的片段降低采样比例，低质量运动与抖动的片段剔除。

运动是视频生成区别于图像生成的核心维度。为运动单独建立分档，是视频管线与图像管线最大的不同。

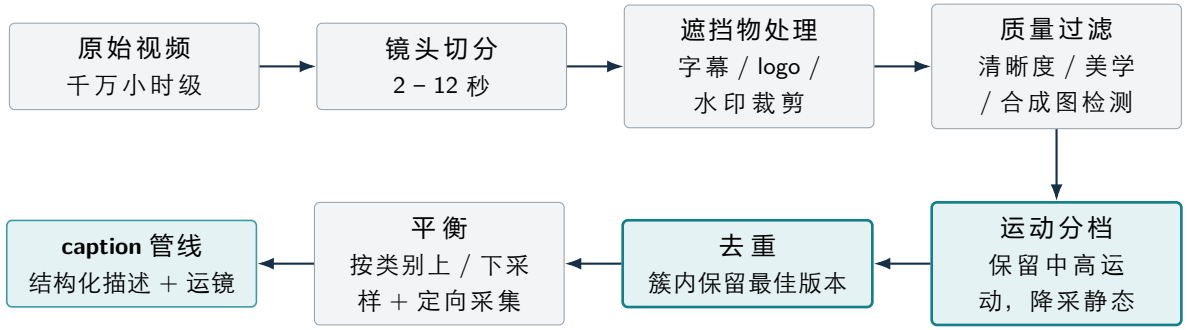


图 12.1　视频生成数据漏斗。读图：从左到右，每一步都在缩小数据量。“运动分档”与“簇内保留最佳版本”是视频管线特有的两步。caption 管线在漏斗的末端，只为留下来的片段付出描述的成本。[38, 139, 140]

规模要折算成 token

HunyuanVideo 1.5 是“1000 万小时 → 8 亿片段”，Cosmos 是“2000 万小时 → 1 亿片段”。片段的平均时长差别很大，“片段数”不能直接比较，最终要折算成 latent token，才能与计算预算对齐（1.5 节）。一个片段的 latent token 数，由帧数、分辨率与两级压缩决定：

$$N = \left(1 + \frac{F-1}{c_t}\right) \cdot \frac{H}{c_s\, p} \cdot \frac{W}{c_s\, p}$$

其中 F 是帧数，$H \times W$ 是分辨率，c_t、c_s 是 VAE 的时间与空间压缩倍数，p 是 DiT 的 patch 边长。数值例子（教学假设：$c_t = 4$、$c_s = 8$、$p = 2$，5 秒、16 fps，取 81 帧）：720p（1280×720）为 $21 \times 45 \times 80 = 75{,}600$ 个 token，480p（848×480）为 $21 \times 30 \times 53 \approx 33{,}400$ 个 token。按这个口径，2 亿个 480p 片段约为 6.7×10^{12} 个 token，已经与一次大语言模型的预训练同一量级。漏斗每砍掉一半片段，省下的是万亿级 token 的计算。

12.5　视频 caption

视频 caption 要描述的，不只是画面里有什么，还有它怎样动、镜头怎样走。

Caption 包含什么

HunyuanVideo 1.5 的 caption 是多层的叙事描述，加上景别、角度、构图、光照、风格、色调与氛围；I2V 任务使用指令式 caption，从首帧出发描述前景与背景怎样变化 [38]。Seedance 1.0 的稠密 caption 则融合了动态特征（动作、镜头运动）与静态特征（人物、场景）[140]。

Captioner 怎样训练

Seedance 1.0 的 captioner 在 Tarsier2 的基础上，冻结视觉编码器、全量微调语言模型 [140]。Wan 用自研的 LLaVA 式 captioner，训练数据覆盖名人与地标、计数、OCR、镜头角度与运动、细粒度类别、空间关系、重写的 caption、编辑指令与人工标注的 caption；在十个维度的 F1 评测上，它与 Gemini 1.5 Pro 相当，在事件、镜头角度、镜头运动、风格与色彩上更好 [139]。HunyuanVideo 1.5 用 OPA-DPO 做 captioner 的后训练，在细节与幻觉之间取平衡，并另用一个相机运动识别模型给出运镜，转成自然语言写进 caption [38]。

相机运动是通用 VLM 最弱的维度之一，所以各家都单独补：Wan 把它加进了 captioner 的训练数据，HunyuanVideo 用专门的识别模型。

流式生成与多参考条件的 caption

整段视频共用一条 caption，对两类新的生成方式不够用。

流式生成按块推进，每一块生成时只能看到“到目前为止”的条件。如果 caption 只有全片一条，模型在生成当前块时就读到了未来才发生的事件；如果把未来的语音或动作描述提前送进当前块，模型学到的就是一种推理时并不存在的信息通路。所以流式数据的 caption 需要两级：一条全片 caption 提供稳定的语义，再按时间（例如按语音片段）切出块级 caption，描述局部的动作与交互，并严格按时间对齐。

多参考条件的生成则要求 caption 描述条件之间的关系。12.3 节提到，参考输入已经扩展到多段视频、图像与音频；此时 caption 除了描述目标内容，还要说明每一份参考与目标的关系（提供主体、动作还是风格），以及多份参考之间的关系。caption 由此从“描述目标”扩展到“描述条件”。

世界模型的数据也在往多层 caption 走。Matrix-Game 3.0 用 InternVL3.5-8B 为每个片段生成四层 caption：叙事、静态场景、稠密的时序描述，以及五个维度的感知质量分（13.6 节）[143]。

caption 的验证有三种做法：在人工测试集上按维度计算 F1；对文字片段用 OCR 核对；用视频-文本 embedding 计算对齐度，并抽查低分样本（第十四章）。

12.6 分辨率与训练阶段

视频生成的训练，从低分辨率的大量数据开始，逐级换成更高分辨率、更少、更干净的数据。

HunyuanVideo 1.5 的八个阶段

阶段	分辨率与帧率	数据
I	256p 图像	50 亿张（从 100 多亿张中筛出）
II	512p 图像	10 亿张
III	256p，16 fps	8 亿个片段
IV	480p，16 fps	2 亿个片段
V	720p，16 fps	1 亿个片段
VI	720p，24 fps	1 亿个片段
VII–VIII	480p 或 720p，24 fps 继续训练	各 100 万个片段（分别用于 T2V 与 I2V）

数据来源：[38]。预训练的任务比例为 T2I : T2V : I2V = 1 : 6 : 3。

为什么数据量逐级收缩

由 12.4 节的公式，token 数与像素面积成正比，与帧率成正比。沿用那里的教学假设：256p（448×256）到 480p，空间 token 从 $16 \times 28 = 448$ 涨到 $30 \times 53 = 1590$，约 3.5 倍；480p 到 720p，从 1590 涨到 3600，约 2.3 倍；720p 从 16 fps 换到 24 fps，同样 5 秒的片段帧数多 1.5 倍。

所以分辨率每升一级，单个样本的 token 数大约翻 2–4 倍。HunyuanVideo 1.5 从 8 亿个片段缩到 2 亿、再缩到 1 亿，每一级的片段数降到 1/2–1/4，正好抵消单样本 token 的增长，各阶段的计算量才大致可控。

数据收缩的同时，阈值也在收紧，3.9 节 Qwen-Image-2.0 的过滤器随阶段逐级启用，也是按这个思路安排的：低分辨率阶段学构图、语义与运动，需要覆盖面；高分辨率阶段学细节，需要干净的高质量样本。

后训练数据按类别组织

后训练数据的量不大，但要把用户关心的风格、运动与主题全部覆盖到，所以各家都按类别组织。Wan 经专家打分与人工整理，选出数百万张最高分的图像，以及覆盖简单与复杂运动、分属 12 个类别的数百万个视频 [139]。Seedance 1.0 按视觉风格、运动类型等属性定义数百个类别，用来指导 SFT 数据的定向采集 [140]。HunyuanVideo 1.5 在继续训练阶段每个任务用 100 万个按类别平衡的高质量片段，T2V 偏向高运动片段，I2V 使用运动导向的指令式 caption [38]。

12.7 音视频联合数据

音视频联合生成，在视频管线之上又叠加了三条支线：音频质量、音画同步与声音描述。

Kandinsky 6.0 的三层数据

Kandinsky 6.0 用三层数据训练 [40]。纯音频预训练用 4000 万条 5–20 秒的音轨，从视频库中抽取，剔除静音、损坏与低质量的音轨，要求足够的动态范围、低削波率与适当的幅度，再由 Qwen3-30B-A3B 描述其中的声音、音乐、语音与环境声。音视频联合预训练用 700 万段音视频，同时经过音频质量、音画同步与视觉三类过滤。首帧生成音视频用 11.6 万个样本，要求首帧是动作开始或停顿的瞬间，主体可见、未被裁切。

同步阈值决定能学到多精确的同步

Kandinsky 6.0 的同步过滤有三条规则：检测到人物的片段，唇形同步分要不低于 3（满分 5）；有声画事件的片段，不同步分要低于 0.1；没有可见人脸的片段不需要唇形同步，直接保留，约占 25%。

阈值的两头都有代价。太松，模型学到错位的口型；太严，数据集会偏向正脸、近景、发音清晰的场景，模型在侧脸、远景与多人场景上失去样本。

三步 caption：先分开描述，再合并

Kandinsky 6.0 的音视频 caption 分三步：Gemma-4-E4B-it 提取语音转写，Qwen3-30B-A3B 写音频描述，Qwen3.5-35B-A3B 把两者与画面描述合并成一条 caption [40]。首帧数据还加入相机运动标签，每条 caption 改写成四个详略级别。

先分开描述、再合并，背后的原则是视觉属性只从画面判断、声音属性只从音轨判断，以减少跨模态的误判：只看画面的模型会臆测声音，只听音轨的模型会臆测画面。描述音频需要专门的模型，Qwen3-Omni 为此单独发布了一个音频描述模型 [144]，LAION-BVD 则用 Audio Flamingo 3 为音频写 caption [37]。

12.8 SFT 与 RL 数据

后训练数据只有几千到几万条，每一条都会被模型记住。所以这里值得投入人工：专家审片、按维度的偏好标注、多维奖励模型。

SFT：专家审片

Kandinsky 6.0 的 SFT 数据按 11 个领域组织 [40]：第一阶段 4,663 个场景，侧重画质；第二阶段 7,686 个场景，侧重音画同步与唇形同步，语音与音乐的比例更高。两轮审片分别由技术筛查员与摄影专家完成，语音场景逐条检查唇形同步。Z-Image 的 SFT 则用高度精选的图像与有依据的详细 caption，用 BM25 稀有度权重重新平衡概念，再把多个 SFT 变体的权重插值合并 [52]。

偏好标注的规则

Seedance 1.0 的人工偏好标注有三条规则 [140]：prompt 来自训练数据与线上用户，去重并剔除有歧义的 prompt；标注员针对一个具体维度，选出最好与最差的视频；同时约束最好的视频在其他维度上不能比最差的差。“按维度挑出好坏，再加上跨维度的约束”，避免了“这个维度好、那个维度很差”的偏好对误导奖励模型。

奖励模型的分工

Seedance 1.0 有三个奖励模型 [140]：基础奖励模型是一个 VLM，负责图文对齐与结构稳定；运动奖励模型负责减少伪影、提高运动幅度与生动度；美学奖励模型输入关键帧，在图像空间做判断。HunyuanVideo 1.5 的奖励数据见 9.5 节：VLM 奖励模型评估文本对齐、图像对齐、画质与运动动态，T2V 先用约 1 万个 prompt 的人工 GSB 偏好做离线 DPO，再做在线 RL [38]。

Seedance 2.0 公开的主要是评测体系 [141]：按主体、运动、场景、风格、音频分别构建评测集，并调整样本分布，以减少小规模评测的方差。主观评测由广告与游戏行业的专家盲评；一项让评审区分生成视频与真实视频的“真实感”研究，结果被反馈到美学调优中。评测协议与奖励数据之间的界线，在这里变得很模糊。

这些奖励维度与 9.5 节“拆成元素、逐项检查”的思路一致。从数据侧看，每个维度都需要单独的标注协议与校准集。

12.9 生成数据飞轮

生成模型的失败案例，是下一轮数据最直接的来源。飞轮能否转起来，取决于能否把失败正确地归因。

按错误归因分流

Qwen-Image-2.0 从标准化评测、bad case 挖掘与用户反馈中收集失败案例，按原因送进三条修复通道 [53]：偏好与对齐类问题进入 RL 通道；概念缺失进入预训练的数据补偿通道，用向量检索加增广补数据，人工审核是唯一的手工环节；用户表达不清则进入 prompt 工程通道。

同样是“画不出松鼠鳜鱼”，可能是数据里没有这个概念，也可能是 prompt 改写丢了信息，修复的通道完全不同。

以模型为诊断器

Z-Image 的主动整理引擎，用模型自身的失败来定位数据缺口，再定向补数据，并重新训练 captioner 与奖励模型 [52]。10.9 节 UI-TARS-2 的数据飞轮也是这样转的：模型的输出，决定下一轮要收集什么数据。

超分辨率数据

超分阶段需要原生的高分辨率数据。HunyuanVideo 1.5 用了约 100 万个 1K–4K、3–10 秒、24 fps 的片段，加上高分辨率图像 [38]。数据侧的要求是：必须是原生高清，不能是上采样得到的伪高清，否则超分模型学到的只是上采样算法本身的痕迹。

本章思考与进阶练习

12.1 Caption 的粒度

题目 视频模型对短 prompt 响应差、对长 prompt 很好。训练 caption 平均 200 词，用户 prompt 平均 15 词。给出训练侧与推理侧各一个方案，并说明代价。

解题思路 训练侧为每个样本生成多种粒度的 caption，训练时按比例抽取，代价是 caption 成本成倍增加，短 caption 下的生成更不确定。推理侧训练 prompt 改写器，代价是改写器可能加入用户没要求的细节，需要用 DPO 抑制幻觉，并且增加推理延迟。两者通常同时使用。

12.2 阶段之间的数据重叠

题目 480p 阶段用 2 亿个片段，720p 阶段用 1 亿个。这 1 亿个应该是那 2 亿个的子集，还是新数据?

解题思路 用子集，迁移平滑，但没有新的语义覆盖；用新数据，带来新的覆盖，但分辨率与内容两个变量同时变化，难以归因。实践中，多是在高分辨率的原始片段中按更严的阈值重新筛选，结果往往是“大部分重叠，加少量新增”。应在元信息中记录每个片段出现在哪些阶段。

12.3 进阶：音画同步的阈值

题目 把唇形同步阈值从 3 提到 4，同步指标上升了，用户却反馈侧脸与多人场景的口型变差。为什么？怎样修正?

解题思路 唇形同步检测器对正脸、近景、单人场景最可靠。提高阈值后，侧脸、远景与多人片段被大量剔除，数据分布向“容易检测”的场景塌缩。修正方法是按场景类型分别设定阈值或采样配额；对检测器不可靠的场景，用人工或更强的同步模型复核；评测集同样按场景分层。

第十三章　语音、具身与世界模型数据

语音、机器人与交互式世界模型面对同一个数据难题：原始信号很容易拿到，可用的标签却很难拿到。语音有亿小时级的音频，缺的是干净的单人分段与准确的转写；机器人有数万小时的人类视频，缺的是可执行的动作标签；世界模型有海量游戏录像，缺的是每一帧对应的键鼠动作。本章围绕“怎样把廉价信号变成可训练的标签”展开：先是语音的处理管线与对话数据，再是机器人的数据金字塔、人类视频与伪动作、仿真与真机经验，最后是交互世界模型的动作标注。数据系统层面的吞吐问题放在第十四章。

13.1　语音预训练数据

语音数据在两年内从十万小时级涨到亿小时级，瓶颈随之从“找数据”变成了“处理数据”。

规模：亿小时级

MiMo-Audio 的预训练语音超过 1 亿小时，audio tokenizer 用 1000 万小时从头训练 [39]。Qwen3-Omni 的音频编码器 AuT 用 2000 万小时有标注音频从头训练，第二阶段约 2T token 的预训练中，音频占 0.77T，与文本的 0.57T、图像的 0.82T 同一量级 [144]。Kimi-Audio 的预训练音频超过 1300 万小时，覆盖语音、声音与音乐 [145]。

MiMo-Audio 报告的主要发现是：音频语言模型在足够大的规模下，同样表现出少样本学习能力。这与文本 LLM 的历史一致：数据规模本身就是能力的来源，处理管线因而成了瓶颈。

Emilia-Pipe：野外语音的六步处理

播客、访谈与视频里的语音，混着音乐、多人对话与噪声，要先变成干净的单人片段。Emilia-Pipe 的六步做法是这一类管线的模板 [146]：

1. **标准化**：转成单声道、16 bit、24 kHz 的 WAV，并做响度归一化。
2. **人声分离**：用 UVR-MDX-Net 去掉音乐与背景噪声。
3. **说话人日志**：用 PyAnnote 3.1 切成单人片段。
4. **VAD 切分**：用 Silero-VAD 切成 3–30 秒的片段。
5. **ASR**：用 WhisperX（Whisper-medium）转写，比官方实现快约 4 倍。
6. **过滤**：语种置信度不低于 80%，DNSMOS 不低于 3.0，按 IQR 剔除字符时长异常的片段。

实测吞吐：8 张 RTX 4090 处理 666.94 小时原始音频，耗时 240.5 分钟，产出 96,117 个片段、258.44 小时，留存率 38.75% [146]。

处理成本的估算

把这个实测折算成 GPU 时。设 ρ 为挂钟时间与音频时长之比，G 为并行卡数，则每小时音频的成本为

$$c = \rho \cdot G \quad \text{(GPU 时 / 小时音频)}$$

这里 $\rho = 240.5/60/666.94 \approx 0.006$，$G = 8$，得 $c \approx 0.048$ GPU 时。

数值例子（教学假设：处理速度与 Emilia-Pipe 相同）：1 亿小时原始音频约需 480 万 RTX 4090 时，1000 张卡并行约需 200 天。这就是 MiMo-Audio 强调“专门构建的端到端数据管线”的原因。管线的主要成本在人声分离与 ASR，要压低成本，只能换更快的模型并做批处理优化。估算时注意口径：这里的分母是原始音频时长，而不是留存下来的 38.75%。

MOSS-TTS：过滤在去噪之前做

MOSS-TTS 的管线与 Emilia-Pipe 结构相同，但在几个细节上更严 [147]：用 MossFormer2 去噪，用 DiariZen 做说话人日志；质量过滤同时要求 DNSMOS 高于 2.8、AudioBox 的 PQ 分高于 6.5，两个分数都在去噪之前计算；检查转写语言与目标语言是否一致；并为每秒字符数设上下界，剔除语速异常的片段。

在去噪之前打分，是为了避免去噪模型把劣质录音“修饰”成高分。去噪能去掉背景噪声，却去不掉混响、削波与低码率压缩带来的失真，这些失真会被 TTS 模型原样学走。每秒字符数的上下界，则主要用来抓转写错位：一段 5 秒的音频配了 80 个字的转写，多半是 VAD 切错了边界。

转写与标签的来源

语音数据的标签几乎全是模型生成的：文本转写来自 Whisper 类 ASR 的伪标注，说话人标签来自说话人日志与声纹，质量分来自 DNSMOS、AudioBox 等无参考模型，情感与副语言标签来自专门的分类器或 LLM 描述，音频描述则来自专门的 audio captioner，例如 Qwen3-Omni-30B-A3B-Captioner [144]。

伪标签的误差会直接进入语音生成模型的读音与韵律，所以过滤不能只看声音质量，还要看转写置信度与语速这类“标签是否可信”的信号。

13.2 语音生成与对话数据

上一节解决了“干净的语音从哪来”，这一节讨论“什么样的语音才适合训练对话模型”。

有声书与野外语音

Emilia 做过一组对照实验 [146]。在 LibriSpeech-Test（正式朗读）上，用 Emilia 英文部分训练的模型，与用有声书数据（MLS）训练的模型相当；在 Emilia-Test（自发语音）上，Emilia 训练的模型明显更好，生成的语音更自然、更像真人，可懂度持平。有声书数据干净但单一，都是朗读腔；野外语音嘈杂，但贴近真实的对话。对话式语音模型需要后者。

小语种：用质量换数量

Emilia-Large 把规模从约 10.16 万小时扩到约 21.63 万小时，扩充的部分主要来自 YouTube（YODAS2）：德语扩到原来的 4.3 倍，法语扩到 6.0 倍，韩语扩到 34.3 倍 [146]。代价是扩充数据的 DNSMOS 阈值从 3.0 降到了 2.4。小语种的高质量语音不够时，只能用质量换数量。这与 5.1 节“小语种上采样会碰到重复上限”是同一个问题在语音上的版本：重复 4 遍和放宽阈值，都是在稀缺数据上做的取舍。

用 TTS 合成对话数据

真实的多轮语音对话很少，主流的做法是把文本对话“读”出来。MiMo-Audio 用 MiMo-TTS-7B（训练数据超过 700 万小时）把文本对话转成语音对话 [39]；Qwen3-Omni 的语音生成模块 Talker，先用数亿条带多模态上下文的语音样本训练，再做长上下文继续预训练，用多语言的语音偏好对做 DPO，最后针对特定音色微调 [144]。

合成对话的风险与第六章相同：TTS 的韵律分布会被下游模型继承。所以合成对话中要混入真实的对话片段，保留自然的停顿、重叠与语气词。合成数据教会模型“说什么”，真实数据教会它“怎样说”。

13.3 机器人数据金字塔

语音的难点是标签不准，机器人的难点是标签根本没有：除了真机数据，其他来源都不直接提供可执行的动作。

五层金字塔

2026 年的一篇综述，把具身操作数据从“最贴近机器人”到“最可扩展”分为五层 [148]：

层	数据	优点	缺点
1	真机数据（遥操作、脚本）	动作直接可执行	需要硬件、操作员与场景复位，最贵
2	UMI 类数据（手持设备采集末端运动）	可在野外采集，比遥操作便宜得多	缺少本体感知，需要重定向
3	第一视角 / 第三视角的人类视频	真实物理，日常多样	动作要重建，并映射到机器人
4	仿真数据	动作可执行，有特权标签，边际成本低	物理是近似的
5	通用数据（网页图文、视频、语言）	语义与推理	不提供动作

越往上越贵、越准，越往下越便宜、越多。

真机数据的规模

几个代表性的真机数据集 [148]：Open X-Embodiment 约 240 万条轨迹、22 种本体、527 个任务；AgiBot World Beta 有 100 万条轨迹、约 2,976 小时、217 个任务；RoboMIND 2.0 有 31 万多条轨迹、1000 多小时、739 个任务；DROID 有 7.6 万条轨迹、约 350 小时、86 个任务，只用一种本体。

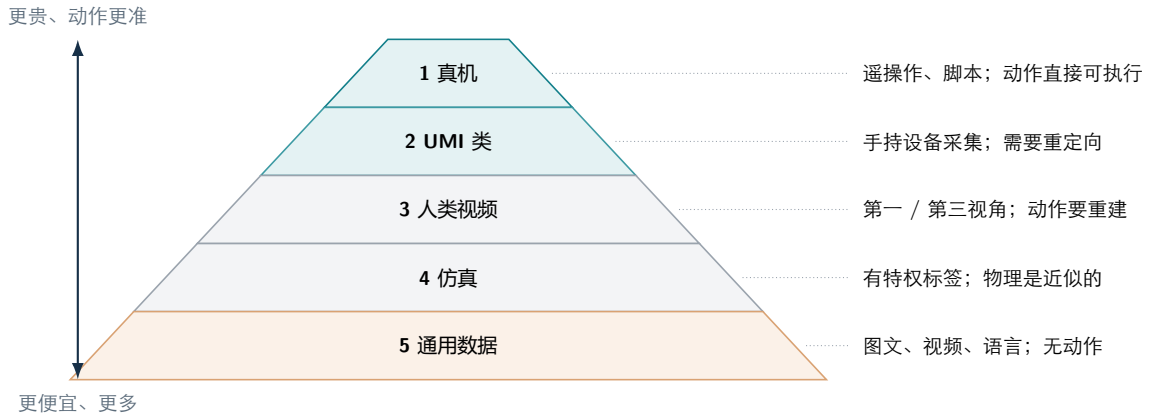


图 13.1　具身数据金字塔。读图：越往上越接近机器人、动作越准，也越贵；越往下越便宜、越多，但动作需要重建，最底层根本不提供动作。最大的真机数据集约 3000 小时，第一视角人类视频的数据池超过 80 万小时。[148, 149]

最大的真机数据集约 3000 小时。而第一视角人类视频这一层，HumanNet 的数据池超过 80 万小时 [149]，EgoScale 用于预训练的人类视频有 20,854 小时 [150]。两者相差一到两个数量级，这就是下一节伪动作研究的动机。

GR00T 的金字塔实践

GR00T N1 按同样的金字塔组织数据 [151]。底层是人类视频，包括 Ego4D、Ego-Exo4D、Assembly-101、EPIC-KITCHENS、HOI4D、HoloAssist 与 RH20T-Human；中层是仿真轨迹与神经轨迹（13.5 节）；顶层是真机数据，包括 Fourier GR-1 的内部遥操作数据（20 Hz）、Open X-Embodiment 的子集（RT-1、Bridge-v2、DROID 等）以及 AgiBot-Alpha。

到 GR00T N1.7，人类视频这一层换成了 2 万小时的 EgoScale 数据。动作空间改为人与各种机器人共享的相对末端执行器动作，即相对当前位姿的增量，而不是绝对目标；状态与动作的维度从 29 扩到 132，以容纳更多本体 [152]。

共享的相对动作空间，是让各层数据真正进入同一个模型的前提。综述把“各层的混合比例怎样确定”列为开放问题，并指出跨层共同训练的前提是动作空间与几何的对齐 [148]。

13.4　人类视频与伪动作

人类视频要进入机器人训练，必须先回答“这段视频里的人做了什么动作”。业界有两条路线：学一个潜在动作，或者从手部姿态重建显式动作。

潜在动作：没有标签也能训练

GR00T N1 处理无标签人类视频的做法分三步 [151]：先在相邻帧对上训练一个 VQ-VAE，学到“潜在动作”；再把它的编码器当作逆动力学模型（IDM），为每一对帧输出潜在动作的 embedding；最后把这些 embedding 当作 flow matching 的目标，作为一个独立的“LAPA 本体”参与训练。

真机数据同时使用真实动作与潜在动作，两种表示由此对齐。潜在动作的好处是不需要任何姿态估计，缺点是它只描述“画面怎样变化”，不能直接执行。

手部姿态重定向：HumanScale

HumanScale 走的是显式动作的路线 [149]：从手部姿态信号重定向，估计每个片段的末端位姿与夹爪状态，生成与机器人数据处于同一动作空间的伪动作标签。

在相同 5000 小时的对照中，已见任务的验证损失，人类视频预训练为 0.0067，真机数据预训练为 0.0071；未见任务上分别为 0.0204 与 0.0254；随着数据量增加，人类视频预训练持续改善，真机数据预训练基本持平。

在 AgiBot 双臂机器人上，经过人类视频预训练的模型，分布内成功率 92.5%、分布外 90.0%。这组数字的对照是不做预训练的 Wan2.2 基线（40.0% 与 0.0%），而不是真机预训练；作者也说明结果是初步的，只在一种架构上测试过。

EgoScale：人类数据的规模律

EgoScale 把第一视角人类数据推到了 2 万小时级，并第一次给出了清晰的规模律 [150]。数据有 20,854 小时，覆盖 9,869 个场景，带有手腕与手指的姿态。在这个规模上，预训练的验证损失 L 与人类数据时长 D（小时）呈对数线性关系：

$$L = 0.024 - 0.003 \ln D, \qquad R^2 = 0.9983$$

其中 $\ln D$ 是自然对数，拟合点是 1k、2k、4k、10k、20k 小时五个规模。R^2 接近 1，说明在这个区间内，人类数据量每翻一倍，损失下降一个固定的量：

$$\Delta L = 0.003 \ln 2 \approx 0.0021$$

数值例子（教学假设：只在拟合区间内使用）：从 2k 小时加到 4k 小时，与从 10k 小时加到 20k 小时，损失的下降量相同，都约为 0.0021，但后者要多花五倍的采集成本。

注意口径：把报告中的系数直接代入绝对值，20k 小时处会得到负的损失，说明报告的 L 或 D 带有未写明的归一化。这条规模律只适合比较“再加多少数据能换来多少下降”，不适合预测损失的绝对值，作者也明确不向区间之外外推。

EgoScale 的另一个关键做法是“对齐中期训练”：在人类视频预训练与机器人后训练之间加一个小阶段，用约 50 小时人类数据加约 4 小时机器人数据（覆盖 344 个桌面任务），让同一批任务的人类动作与机器人动作对齐。相对不做预训练的基线，这套流程使机器人的平均成功率提高了 50% 以上，报告中不同位置给出的数字在 54% 到 55% 之间。

少量配对数据就能把两个动作空间接起来，说明人类视频的瓶颈不在“标签不够准”，而在“两种本体的动作没有对齐”。

伪动作的误差来源

伪动作的误差主要有四类：遮挡与运动模糊造成的姿态估计误差，让动作抖动；人手与夹爪的自由度不同，动作可能无法执行；单目深度不准，位移幅度出错；帧率与延迟不同，动作与画面错位。

所以伪动作数据适合做预训练，学“什么样的动作导致什么样的结果”；精确控制仍然要靠真机数据做后训练。HumanScale 在未见任务上的优势，与 EgoScale 的规模律指向同一个结论：人类视频的价值主要在多样性。

13.5 仿真、神经轨迹与真机经验

人类视频提供多样性，真机数据提供精度，中间的空当由两类合成数据填补：仿真轨迹，以及视频生成模型产出的“神经轨迹”。最后，部署中的机器人本身也在产出数据。

仿真轨迹：从少量演示放大

GR00T N1 用 DexMimicGen，把少量人类演示扩展成大规模合成轨迹 [151]：11 小时生成了 78 万条轨迹，相当于 6,500 小时（约 9 个月不间断）的人类演示，其中 54 万条用于预训练，每个“源–目标容器”组合 1 万条。仿真轨迹的边际成本很低，但它继承了源演示的策略风格，也继承了仿真器的物理近似。

神经轨迹：用视频生成模型扩充数据

GR00T N1 的另一种合成方法，是用视频生成模型生成轨迹 [151]。它在约 88 小时的遥操作数据上微调一个图生视频模型，配合新的语言指令，生成约 827 小时的视频，约为原数据的 10 倍。报告给出的生成成本约为 10.5 万 L40 GPU 时，在 3600 张卡上历时约 1.5 天；两个数字按满载换算并不严格一致（满载约 29 小时），比较成本时以 GPU 时为准。神经轨迹同时使用潜在动作，以及由真机数据训练的 IDM 给出的伪动作。

后训练时，真机轨迹与神经轨迹按 1:1 采样。在 RoboCasa 上，每个任务用 30 / 100 / 300 条演示时，成功率分别提高 4.2% / 8.8% / 6.8%；GR-1 的 8 个真实任务平均提高 5.8%。

提升在中等数据量时最大。演示太少时，生成模型本身学不好；演示足够多时，合成数据带来的新信息就少了。这类做法把第十二章讨论的视频生成模型，变成了机器人数据的合成器。

真机经验数据：π*0.6 的 Recap

Physical Intelligence 的 π*0.6 用 Recap 方法，从三类真机数据中学习 [153]。演示是遥操作的成功示范，用来监督学习基础策略。专家纠正发生在机器人快要失败时：人接管并纠正，再交还控制，这类数据覆盖演示中到不了的状态，教会模型恢复。自主 rollout 是机器人长时间无人值守地运行、按结果打标签的数据，属于 on-policy 的自我改进数据。

方法分两步：先训练一个价值函数，估计每个动作相对于策略平均表现的优势；训练策略时，再把优势作为条件输入，模型学会区分好动作与差动作，推理时以“高优势”为条件生成动作。失败的尝试也留在数据里，策略能从全部结果中学习，而不只是从成功轨迹中学习。训练先在大规模混合数据上做离线 RL（取代 π0.6 的纯监督预训练），再用演示做任务微调，最后在真机上用带纠正与奖励的经验数据继续 RL。

官方公开的结果以定性为主：在最难的几项任务上，加入自主经验后吞吐量提高一倍以上，失败率降到一半或更低。演示的任务包括从早上 5:30 到晚上 11:30 连续做咖啡，在一个新家庭中折叠 50 件没见过的衣物，在真实工厂里组装 59 个巧克力包装盒 [153]。

这是 10.11 节“RL 产出回流为数据”在物理世界的版本：部署本身就是数据采集。

13.6 交互世界模型数据

交互世界模型面对的是机器人问题的镜像：它不需要输出动作，但每一帧都需要以动作为条件。

动作标注是核心难点

交互世界模型要学的是

$$p(x_{t+1} \mid x_{\le t}, a_t)$$

其中 x_t 是第 t 帧，a_t 是这一刻的动作（按键、鼠标或位移）。网络上的游戏视频没有动作标签，所以数据分为两类：大量无标签视频用来学视觉动态，少量有标签视频用来学动作控制。

Matrix-Game：Minecraft 上的两类数据

Matrix-Game 的无标签数据，来源是 MineDojo 中约 6000 小时的 Minecraft 原始视频，包括教程、自由游戏与环境交互演示 [154]。它经过一条三级过滤的管线：

1. 用 TransNetV2 切成单镜头片段，去掉每段首尾各 4 帧；
2. 第一级过滤视频质量（DOVER）与美学（LAION），按游戏类型设阈值；
3. 第二级用逆动力学模型检测菜单状态，用 CRAFT 去字幕与水印，用 DeepFace 去掉主播的摄像头画面；
4. 第三级用 GM-Flow 光流过滤运动，并用逆动力学模型估计的偏航与俯仰角过滤相机运动。

过滤之后得到约 2,700 小时的无标签数据，其中相机运动稳定、界面干净、画面清晰的约 870 小时，又被单独作为精选子集。

有标签数据超过 1000 小时，来自两个来源。一是 MineRL 中的 agent 游玩：按课程训练的 VPT agent 在 MineRL 中游玩，逐帧提取键鼠动作，采样到 16 Hz；偏航与俯仰每帧上限 15°；关闭区块加载，避免地形突然出现；agent 濒死、卡住或进入菜单时停止录制。二是 Unreal Engine 的程序化场景：城市、沙漠、森林等场景，提供离散按键、连续视角向量，以及真实的位置、速度、朝向与方块操作结果，标注没有噪声。有标签部分最终整理为约 1,200 小时的训练集，都是 720p、65 帧的片段，按 8 种地形均衡。

Matrix-Game 3.0：从 Minecraft 到 AAA 游戏

Matrix-Game 3.0 把数据从 Minecraft 扩展到写实场景，三个来源各解决一个问题 [143]。UE5 合成数据来自 1000 多个 UE5 场景，由脚本控制的 agent 游玩，动作与位姿完全已知。AAA 游戏录像包括 GTA V、荒野大镖客 2 等游戏，从角色位置的变化推断 8 个方向的移动动作。真实世界视频用 ViPE 重新估计相机位姿，作为动作条件。所有数据都经过质量过滤，约 20% 被剔除，最终得到 480 万个片段，并由 InternVL3.5-8B 生成四层 caption（12.5 节）。

“从位置变化推断动作”是一个关键的折中：它只恢复了移动方向，丢掉了视角与交互动作，却让海量的商业游戏录像第一次可以作为有标签数据使用。

三种动作标签的取舍

三种动作来源各有代价。agent 自动游玩的动作精确，可以大规模生成，但行为分布受 agent 策略的限制，不像人类玩家。程序化引擎的标签没有噪声、场景可控，但画面与真实游戏、真实世界之间有差距。逆动力学或位姿伪标注能利用海量真人录像，但标签有误差，粒度也受限（13.4 节）。Matrix-Game 系列的做法是三者同时使用：引擎数据提供精确的控制信号，游戏与真实视频提供视觉多样性。

物理世界的视频数据

Cosmos 世界基础模型从 2000 万小时原始视频中，整理出约 1 亿个 2–60 秒的片段 [142]。它按场景切换切分，过滤出高质量且有动态的内容，用 VLM 每 256 帧生成一次描述，做语义去重，并用 GPU 硬件编解码转码、基于 Ray 编排不同速度的模型来提高吞吐（14.4 节）。

物理世界的视频没有动作，但有丰富的物理规律。这类数据为下游的机器人与驾驶模型提供“世界会怎样变化”的先验，再由少量带动作的数据学习控制；13.4 节的人类视频预训练，也是先学“世界怎样变”，再学“怎样控制”。

本章思考与进阶练习

13.1 伪动作的误差

题目 你用手部姿态估计给 1 万小时人类视频打伪动作标签，估计误差的标准差约为 2 cm。这个误差对预训练与后训练分别有什么影响?

解题思路 预训练学的是动作与视觉变化的对应关系，2 cm 的随机误差会被大量数据平均掉，主要影响是动作预测的分布变宽。后训练与执行则不同：抓取、插接这类精细操作的容差可能只有几毫米，2 cm 的误差足以导致失败。所以伪动作数据用于预训练，精细任务的后训练必须用真机数据或高精度的 UMI 数据；还应把姿态估计的置信度写进元信息，后训练混合时剔除低置信度的片段。

13.2 人类视频与机器人数据的安排

题目 你有 5 万小时第一视角人类视频、500 小时真机遥操作数据，以及其中 50 小时同一批任务的人类–机器人配对数据。怎样安排训练阶段?

解题思路 预训练以人类视频为主，例如 90%，保留约 10% 的真机数据；真机数据同时提供真实动作与潜在动作，把伪动作空间对齐到机器人动作空间（GR00T N1 的做法），动作统一用相对末端位姿（GR00T N1.7 的做法）。接着用配对数据做一个对齐中期训练的小阶段，让同一任务的人类动作与机器人动作对齐（EgoScale 的做法）。后训练以真机数据为主，若有神经轨迹或仿真轨迹，可按 1:1 与真机数据共同训练。全程监控未见任务上的损失（HumanScale 的指标）与真机成功率，并用 EgoScale 式的规模律估计再增加人类数据的边际收益。

13.3 进阶：经验数据的分布偏移

题目 π*0.6 的自主 rollout 数据，会随着策略的改进而改变分布。这对价值函数与优势估计有什么影响？怎样处理?

解题思路 价值函数在旧策略的数据上训练。策略改进后，新数据的状态分布变了，旧的价值估计在新状态上可能有偏差；优势的基准（策略的平均表现）也在上升，同样的动作在旧策略下是正优势，

在新策略下可能是负优势。处理方法是交替进行数据采集与价值函数更新，让价值函数跟上当前策略（Recap 的交替训练正是如此）；给旧数据降权；并在元信息中记录每条数据由哪个策略版本产生，便于做 off-policy 修正或按版本切分评测。

第十四章　数据系统与工程

前面十三章的每一个数据决策，最终都要落到一个系统上执行：对 250 亿篇网页做三级去重，用 1600 张 GPU 在 95 分钟内为 600TB 视频写 caption，同时让几十万个 RL 沙箱在线。数据系统决定了一个想法能在一周内验证完，还是要拖上一个季度。

本章按数据流的顺序展开：存储格式，处理框架与 GPU 加速，视频与 caption 的大规模推理，合成数据生成，RL rollout 与环境服务，训练时的加载与打包，最后是版本、血缘与监控。

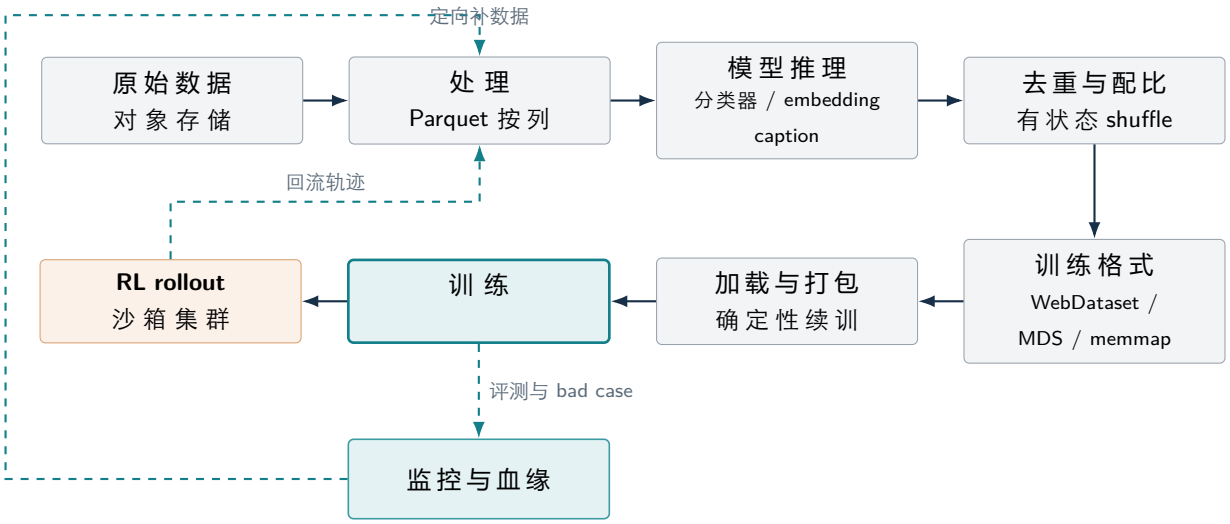


图 14.1　数据系统的主干。读图：上半段（A→G）是离线的批处理，瓶颈在吞吐与 shuffle；RL rollout（H）是在线的服务，瓶颈在沙箱的并发与恢复；两条回流箭头把它们连成飞轮，血缘系统保证每一次回流都可追溯。[155, 156, 157]

14.1　存储格式与分片

处理阶段与训练阶段对存储的需求正好相反：处理时按列读，训练时按样本读。

四类格式

处理时通常只要质量分、语言、长度这几列，训练时一次要一条完整的样本。几种主流格式，就是在这两种访问模式之间各取所长：

格式	结构	适合	不适合
JSONL	每行一个 JSON	交换数据、人工查看、小规模	列裁剪、大规模随机访问
Parquet	列式，按行组组织	处理阶段只读某几列，压缩率高	逐样本的随机读

格式	结构	适合	不适合
WebDataset [158]	tar 分片，同一样本的多个文件放在一起	训练时顺序流式读取，多模态	修改单个样本
MDS [159]	带索引的二进制分片	确定性打乱、断点续训、节点数可变	分析查询
Lance [160]	列式，支持快速随机访问与向量索引	多模态检索、去重、抽样分析	生态较新

所以数据在处理阶段一般存为 Parquet，处理完再转成训练格式。Lance 试图用一种格式同时满足两边，适合需要频繁按向量检索样本的多模态团队。

分片大小

分片太小，元数据与文件打开的开销大，对象存储的请求数暴涨；分片太大，打乱的粒度变粗，断点续训时要跳过的数据多，一个坏分片的损失也大。经验区间是每个分片几百 MB 到 1–2 GB。

多模态数据按样本数而不是字节数控制分片，让每个分片的样本数大致相同，便于均匀分给各个数据并行进程。视频先按片段时长分桶再分片，同一分片内的片段长度相近，便于后续打包（14.9 节）。

元信息与正文分离

URL、各类分数、主题、去重簇 ID 等元信息与正文分开存储。改阈值、重新配比时，只需扫描元信息表，不必读取正文。2.7 节说“保存所有中间分数”，这一节是它在存储上的落地：分数存得越全，下一次调配比就越便宜。

14.2 文本处理框架

存储决定了数据怎么放，处理框架决定了算子怎么跑。这里要分清两层：负责调度与容错的执行引擎，以及负责实现算子的数据处理框架。

执行引擎与上层框架

执行引擎有 Spark、Ray（Ray Data）、Dask、MaxCompute 等，负责分布式调度、shuffle 与容错。数据处理框架建在其上，实现抽取、过滤、去重、分类器等算子并编排流水线：Datatrove 是 FineWeb、FinePDFs、FinePhrase 背后的处理库 [161]；NeMo Curator 是 Nemotron 系列数据的处理库，同时支持文本、图像与视频 [162]；Data-Juicer 2.0 有 100 多个算子，覆盖文本、图像、视频与音频，统一的数据集抽象可以在 Hugging Face Datasets、Ray Data 与 MaxCompute 之间切换 [155]。

Data-Juicer 2.0 的优化

Data-Juicer 2.0 在最多 100 个节点、1.28 万个 CPU 核上报告了几项优化 [155]。按负载重排并融合算子，处理时间最多减少 70.22%；自动分配 GPU，最多节省 99% 的时间，并避免显存溢出；调优批处理参数，时间最多减少 84%；自适应切分子集，1.28 万核上从 5000 多秒降到约 2000 秒；算子级检查点则让失败后可以从上一个成功的算子恢复，不必从头重跑。

纯文本处理时，MaxCompute 的耗时约为 Ray 的四分之一，所用核数只有一半。执行引擎没有通用的最优解，要按负载选。

有状态算子：去重的 shuffle 成本

过滤与打分是无状态的，每篇文档独立处理，可以无限并行；去重是有状态的，必须把相似的文档汇集到一起。MinHash LSH 分三步：计算每篇文档的签名，无状态，可并行；按 band 哈希把文档 shuffle 到桶里，这是一次全局 shuffle；桶内找出候选对，再用连通分量算法求出重复簇，这一步需要迭代。

Data-Juicer 2.0 的 MinHash 去重器在 8 节点 × 160 核上，处理 200 GB 用 7.47 分钟，1 TB 用 30.08 分钟，5 TB 用 2.8 小时 [155]。核数翻倍时，时间只降到原来的 59%–67%，限制线性扩展的正是 shuffle 与连通分量计算。作为参照，论文引用 NeMo Curator 在 64 张 A100 上处理 1.1 TB 用了 1.8 小时。

Olmo 3 把模糊去重分 32 个分片、子串去重分 57 个分片执行 [13]，就是为了把全局 shuffle 拆成可管理的块。4.2 节 MAI 按“全局丢弃顺序”去重，也需要这样一个能跨分片维护簇的系统 [21]。

14.3 GPU 加速的数据处理

文本管线中，越来越多的步骤是模型推理。哪些步骤放到 GPU 上，决定了整条管线的成本结构。

哪些阶段适合放到 GPU

NeMo Curator 把两类阶段放到 GPU 上 [162]：精确、模糊、语义三类去重中的哈希、MinHash 签名、embedding 与相似度计算；以及领域、质量、内容类型、安全与 FineWeb 教育分等各类文本分类器。

经验上，规则过滤、正则与语言识别留在 CPU 上，便宜且易扩展；分类器、embedding 与 caption 这类模型推理放到 GPU 上；大规模哈希与 join 则视数据量与可用资源，用 GPU 加速的数据帧（RAPIDS cuDF）或 CPU 集群。

先便宜，后昂贵

一条典型的文本管线，是 CPU 阶段（抽取、语言识别、规则过滤）在前，GPU 阶段（质量分类器、embedding）在后，MinHash 去重视情况放在任一侧。关键是让两种资源同时忙碌：CPU 阶段的产出速度，要刚好匹配 GPU 阶段的消费速度。

顺序也很重要。启发式过滤就去掉了 84.6% 的文档（3.2 节），GPU 只需处理剩下的 15%；MAI 的抓取漏斗从约 1.2T 个网页收缩到约 1900 亿个（2.1 节）[21]，昂贵的模型只出现在漏斗的最后一段。管线的成本结构，在很大程度上由“便宜的步骤能先去掉多少”决定。

14.4 大规模视频处理与 caption 推理

视频把 CPU 与 GPU 的配比问题放大到了极致：解码与切分是重 CPU 的，caption 是重 GPU 的，任何一边跟不上，另一边就在空转。

NeMo Curator 的视频管线

NeMo Curator 的视频管线有九步 [57]：

1. 读取本地路径、S3 兼容存储或 HTTP(S)；
2. 按固定步长切分，或用 TransNetV2 检测场景切换；
3. 转码为 H.264，可用 CPU 编码器或 GPU 的 NVENC；
4. 按运动与美学过滤；
5. 用 NVDEC 加 ffmpeg 抽帧；
6. 用 Cosmos-Embed1 计算片段级向量；
7. 用 Qwen-VL 生成 caption，可选用 Qwen-LM 精炼；
8. 用 k-means 聚类加成对相似度去重；
9. 导出时 embedding 存为 Parquet，样本打包成 WebDataset。

Cosmos 用类似的管线，把 2000 万小时视频整理成约 1 亿个片段，用 GPU 的硬件编解码器转码，用 Ray 编排不同速度的模型 [142]。

600TB / 95 分钟：一个公开案例

Anyscale 与 CoreWeave 公开过一个案例 [156]：最多 1600 张 RTX PRO 6000 Blackwell GPU，加 200 个 AMD CPU 节点。CPU 阶段做视频解码、场景切分与过滤，GPU 阶段用 Qwen3-VL-8B 生成 caption。

吞吐从 600 GB、256 张 GPU 时的每秒 1,273 条 caption，扩到 600 TB、1,600 张 GPU 时的每秒 12,666 条。GPU 数增加约 6.3 倍，吞吐增加约 10 倍，每 GPU 时的 caption 数在大规模下反而更高。博客没有拆解原因，至少说明在这个规模上，系统没有因为调度与存储而退化。

案例中遇到的四个瓶颈很有代表性。第一是 GPU 空闲：原来的 Ray Core 用固定的 actor 池，GPU 经常闲置，改用 Ray Data 的流式执行加自动扩缩容后，GPU 时用量降到 73%，每 GPU 时的 caption 数提高到 3.7 倍。第二是解码饥饿：CPU 解码跟不上 GPU 推理，只能加大 CPU 池；用 FFmpeg 做数据增广时，4.37 万个片段扩展到 7000 万个，峰值用了 35,820 个 CPU 核，全量运行时对象存储持续读带宽 110–120 GB/s。第三是模型加载的惊群：多达 255 个推理引擎同时请求约 4.3 TB 的模型权重，改为节点本地缓存后，每个引擎约 2 秒加载完 17 GB 的模型。第四是隐蔽的硬件故障：Ray 看不到 GPU 降频与慢节点，要靠平台的健康监控与节点替换。

这些数字来自厂商博客的自报，没有经过独立验证，但瓶颈的类型在各家的视频管线中都会出现。Keye-VL-2.0 的数据生产管线给出了同样的对策：用一个异步的双队列把 CPU 预处理与 GPU 推理解耦，两边各自满载，报告的实际吞吐提升 3–5 倍 [91]。

资源配比的估算

设 t_c 为 CPU 处理一个片段的核·秒，t_g 为 GPU 生成一条 caption 的均摊 GPU·秒，则让 GPU 满载所需的 CPU 核与 GPU 之比为

$$\frac{N_{\text{CPU}}}{N_{\text{GPU}}} \approx \frac{t_c}{t_g}$$

数值例子（教学假设）：$t_c = 2$ 核·秒，$t_g = 0.08$ GPU·秒，比值为 25；1000 张 GPU 需要约 2.5 万个

核，加上 I/O 与调度余量约 3 万核。caption 模型越小、输出越短，t_g 越小，需要的 CPU 就越多；这个比值必须先在小规模上实测，再决定集群的采购配比。

14.5 合成数据生成系统

合成数据的生成是“超大批量、没有延迟要求”的推理，它的优化目标与在线服务不同：只看吞吐，不看首 token 延迟。

离线批量推理

几项主要的优化都围绕吞吐。用 vLLM 或 SGLang 的连续批处理，批次尽可能大 [163, 164]；同一个 prompt 模板对大量文档复用，用前缀缓存省掉重复的 prefill；改写类任务选用 1–3B 的小模型（6.1 节），吞吐比 70B 高一个数量级；并限制最大生成长度，避免少数超长输出拖慢整个批次。FinePhrase 报告，通过选择小生成器、优化 prompt，生成成本比已有方法降低约 30 倍 [71]。

长 CoT 的长尾延迟

推理轨迹的长度是长尾分布：大部分几千 token，少数几万 token。同步批处理时，整批要等最长的样本，GPU 利用率很低。连续批处理让短样本完成后立刻补入新样本；按预估长度分组，让同一组的长度相近；超过最大长度的样本截断并单独标记。截断的样本不能直接当作完整轨迹使用，要按 1.4 节的 loss mask 处理。

验证与沙箱的并发

代码与数学类的合成数据要逐条验证：编译、运行单元测试、比较数值答案。验证集群的吞吐必须与生成集群匹配，否则生成出来的数据会在验证队列里堆积。代码执行必须放在隔离的沙箱里，限制 CPU、内存、网络与运行时间，因为生成的代码可能死循环、写文件或发网络请求。这个沙箱需求，在 RL 中会放大几个数量级，14.7 节专门讨论。

14.6 RL rollout 与训练的异步

合成数据是离线生成、离线验证的；RL 的数据则在训练过程中实时产生，生成与训练必须同时进行。

为什么要异步

同步 RL 的一步是：生成 → 打分 → 训练 → 同步权重 → 再生成。agent 任务中，生成与打分可能占到 90% 以上的时间，训练 GPU 大部分时间在等。长尾也更严重：一个 SWE 任务可能跑几分钟，另一个跑一个小时。

AReaL：完全异步与陈旧度控制

AReaL 的设计有四个要点 [165]。生成与训练解耦：rollout 进程持续生成，训练进程在批次就绪时更新，互不等待。生成可以中断：新权重到达时，中断正在进行的生成，丢弃旧权重的 KV 缓存，用新权重重新计算后继续解码，所以一条轨迹可能混有多个版本的模型生成的 token。陈旧度有上限 η：

训练数据最多落后当前模型 η 个版本，由 rollout 控制器限流实现，$\eta = 0$ 即同步 RL，$\eta \leq 8$ 对最终效果影响很小，不设上限则会损害效果。PPO 目标被解耦：把“采样数据的行为策略”与“近端策略”（每次更新前的快照）分开，让混合版本的数据也能稳定更新。

结果是在相同 GPU 上，端到端训练比同步系统最多快 2.77 倍，最终精度持平或更好；扩展到 512 张 GPU，吞吐近似线性增长。

陈旧度的估算

一条轨迹在生成期间会跨越多少个模型版本？设轨迹的生成时长为 T_{roll}，一次训练更新的时长为 T_{step}，则这条轨迹的陈旧度约为

$$s \approx \left\lceil \frac{T_{\text{roll}}}{T_{\text{step}}} \right\rceil$$

数值例子（教学假设）：训练一步 5 分钟。数学题 rollout 2 分钟，$s = 1$；SWE 任务 30 分钟，$s = 6$；一个两小时的长时程任务，$s = 24$。设 $\eta = 8$，后两类中的长任务就会被限流或丢弃，训练数据于是系统性地偏向短任务。这就是长时程 agent RL 要配合可中断生成或部分 rollout 的原因：否则陈旧度约束本身就会改变数据分布。

产业系统

各家的异步框架在同一结构上做了不同的取舍：

系统	做法
MiniMax Forge [116]	把训练与推理引擎和 agent 完全解耦，任意 agent 都能接入；异步调度，训练样本按树结构合并，报告约 40 倍的训练加速
Qwen3.5 [8]	异步 RL 框架覆盖文本、多模态与多轮场景，用到 rollout 路由回放、投机解码与多轮 rollout 锁定，报告端到端提速 3–5 倍，并称约束了梯度陈旧度、缓解了数据偏斜；博客没有解释技术细节，也没有给出提速的基线
GLM-5.2 [113]	基于开源的 slime 框架做异步 RL，并在其上合并十多个领域专家
Nemotron 3 Ultra [17]	异步 GRPO，全局批次 8192，每个样本 16 条 rollout，生成长度从 48K 逐步增加到 64K
DeepSeek-V4 [7]	报告单列了“可抢占、可容错的 rollout 服务”“支持百万 token 上下文的 RL 框架”与“全词表 OPD 的教师调度”三节

路由回放值得单独一提。按这一技术通常的含义，它解决的是 MoE 特有的问题：推理引擎与训练引擎数值上并不完全一致，同一个 token 可能被路由到不同的专家，导致重要性权重失真；把 rollout 时的专家路由记下来、训练时重放，就能消除这部分偏差。这是一种与陈旧度无关的 off-policy 来源，却同样会让数据“看起来不是当前策略采的”。

陈旧度对数据的含义

异步 RL 中，训练数据不再严格是当前策略的分布。10.2 节的通过率筛选与难度画像，都是基于某个版本的模型算出来的；陈旧度越大，筛选依据与当前策略的偏差越大。所以元信息中要记录每条 rollout 由哪个模型版本生成，作为陈旧度分析与 off-policy 修正的依据。

14.7 环境服务

异步框架解决了“训练等生成”的问题，下一个瓶颈是“生成等环境”。agent RL 的每一条 rollout 都需要一个独立的沙箱，沙箱集群的规模已经与训练集群相当。

DSec：四种底座，一个接口

DeepSeek 的 DSec 是目前公开得最完整的生产级沙箱系统 [157]。它在一个统一的 SDK 后面提供四种执行底座：最轻量的函数调用（FnCall），用于无状态的工具与代码片段；容器，用于大多数 SWE 与终端任务；microVM，提供更强的隔离，用于需要内核能力或不可信代码的任务；以及完整虚拟机，用于 GUI 与操作系统级任务。

隔离越强，启动越慢、密度越低。按任务选底座，而不是所有任务都用最重的那一种，是把成本压下来的第一步。

镜像按需加载

SWE 任务的难点是镜像：每个任务都有自己的仓库与依赖，冷启动拉取一个完整镜像可能要几分钟。DSec 的观察是，agent 在一次任务中实际只访问镜像的 4%–9% [157]。于是镜像存放在 3FS 分布式文件系统上，用 EROFS 格式按需加载，用到哪个文件才读哪个；再用 OverlayFS 分层，同一仓库的不同任务共享只读的依赖层，每个沙箱只有一个薄的可写层。报告的效果是镜像加载快 1.71 倍，磁盘写入减少约 57%。

密度与规模

DSec 的一个生产单元约 160 个节点，每个节点可以运行至少 3,200 个容器或 800 个 microVM。整个系统在生产中同时运行超过 38 万个沙箱，每秒创建超过 5,000 个，每天约 300 万个实例 [157]。

做个对比：如果每个沙箱冷启动要 2 分钟，每秒 5,000 次创建意味着任何时刻有 60 万个沙箱处在启动中，比在线运行的还多。按需加载与分层镜像，是让这个创建速率成立的前提。

抢占与恢复

训练 GPU 是可抢占的，沙箱里的状态却不能丢。DSec 与 RL 框架协同设计：GPU 被抢占时，沙箱被冻结或做快照，恢复时从快照继续，而不是重跑整条轨迹 [157]。Kimi K3 的环境同样运行在可快照、可恢复的 microVM 中 [14]。长时程任务尤其依赖这一点：一条两小时的轨迹如果在第 100 分钟被抢占，从头重跑不仅浪费两小时，还会让它的陈旧度再翻一倍（14.6 节）。

各类环境的状态管理

不同的环境，状态管理的方式也不同。SWE-rebench V2 为每个 SWE 任务预构建容器镜像，保证可复现 [124]；UI-TARS-2 运行数千台 Windows、Ubuntu 与 Android 云虚拟机，管理器每秒处理数千个请求，共享文件系统让 GUI 与终端操作作用于同一批文件，游戏则跑在支持崩溃恢复与时间控制的浏览器沙箱中 [127]；AWM 的合成工具环境用 SQLite 保存状态，每个任务从同一个初始数据库开始，便于重置与比较 [111]。

环境的健康度

环境本身也会出错：依赖下载失败，测试不稳定（同一份代码有时通过、有时失败），网络超时。AWM 的评审区分“agent 错误”与“环境错误”[111]，环境错误产生的奖励必须剔除，否则策略会因为环境的随机故障而被惩罚。

要持续监控每个环境的错误率、同一参考解多次运行的通过率是否稳定，以及启动时间的分布。参考解的通过率低于 100%，这个环境就不该进入训练。

14.8 训练时的数据加载

数据从处理系统出来，进入训练之前，还要经过加载器。预训练要连续跑几个月，加载器的首要任务是在多次重启之间保持数据顺序可解释。

确定性与断点续训

数据加载要满足三点。第一是确定性：给定随机种子与步数，能精确复现这一步用了哪些样本，OLMo 把全局打乱后的样本索引预先算好并保存，任意一步的批次都能还原 [166]。第二是可续训：重启后从中断处继续，不重复、不遗漏。第三是支持可变的节点数：重启时节点数可能变化，数据切分要随之调整，MDS 支持在不同节点数下保持相同的全局样本顺序 [159]。

多源混合的流式加载

配比 π_b 在加载时实现，有两种方式。预混合是在处理阶段就按配比生成一份混合数据，每次改配比都要重新生成。流式混合是每个数据源一个流，加载时按 π_b 采样，灵活，但要跟踪每个源的消耗进度。

多阶段训练（第五、七章）中每个阶段的配比都不同，流式混合更合适。代价是加载器的状态（每个源读到哪里）必须随 checkpoint 一起保存。

预先 tokenize 与在线 tokenize

预先 tokenize 把数据存成 token id 的 memmap，训练时零 CPU 开销，长度已知，便于打包；代价是换 tokenizer 就要重做，存储量也大。在线 tokenize 灵活，但占 CPU，长度未知，打包更难。所以预训练普遍预先 tokenize，SFT 与 RL 数据量小、格式多变，常用在线 tokenize。

加载瓶颈的诊断

记录每一步的数据等待时间与 GPU 计算时间。等待时间超过总时间的几个百分点，就说明加载成了瓶颈，再逐项排查存储带宽、解码、tokenize、打包与跨节点传输。视频与多模态训练中，加载瓶颈最常见。

14.9 多模态数据加载

多模态加载比文本多出两个问题：解码放在哪里，以及变长样本怎样组批。

解码的位置

解码有三种放法。CPU 实时解码灵活，但 CPU 常常跟不上；GPU 硬件解码用 NVDEC，占用少量 GPU 资源，吞吐高；预计算 latent 则离线用 VAE 编码好，训练时直接读 latent，最快。

预计算省掉了冻结模型的前向，代价是把一批选择固定了下来：VAE 与文本编码器的版本、裁剪方式、分辨率、caption 与部分增广。训练如果需要随机时序裁剪、多尺度增广，或者 caption 还在不断修订，就要保留重新计算的路径，或者只缓存到更早的中间表示。

变长样本的分桶

图像有不同的宽高比，视频有不同的时长与分辨率。通常按宽高比、分辨率与帧数分桶，每个批次只从一个桶里取样本，避免大量 padding；桶内顺序打乱，桶之间按概率交替。

分桶省下的是 padding。数值例子（教学假设：视频片段时长在 2–10 秒之间均匀分布，token 数与时长成正比）：不分桶时，每个批次都要 pad 到 10 秒，平均利用率为 $6/10 = 60\%$，四成计算花在 padding 上；按 2 秒一档分成 4 个桶，每个桶 pad 到上限，利用率分别约为 3/4、5/6、7/8、9/10，平均约 84%。只分 4 个桶，就收回了大部分浪费；桶分得更细，收益递减，而每个桶的样本数变少，打乱的随机性也变差。

每个桶的采样概率要按 token 量或配比设定，否则大桶会主导训练。HunyuanVideo 1.5 的文生图阶段按宽高比分桶 [38]；Keye-VL-2.0 按自适应像素预算处理不同分辨率的图像与视频（11.7 节）[91]。

14.10 打包与 mask

分桶解决的是多模态的组批，打包解决的是所有模态共有的问题：怎样把长短不一的样本塞进定长的序列，而不浪费计算、不泄漏信息。

预训练打包

顺序拼接把文档依次拼成定长序列，跨序列边界的文档会被截断；best-fit 打包按长度把文档放进剩余空间最合适的序列，几乎不截断。DeepSeek-V4 把来自不同来源的文档打包在一起以减少截断，并使用样本级注意力 mask [7]。第七章的长上下文数据对截断最敏感：一篇 200K token 的文档被截成两半，模型就学不到它需要的长距离依赖。

SFT 与 RL 的打包

SFT 中，Nemotron 3 Ultra 用按长度的 best-fit packing，不截断，包内去重（8.8 节）[17]。RL 中，同一个 prompt 的多条 rollout 共享 prompt 前缀，MiniMax Forge 的“树状样本合并”就利用这一点，把共享前缀只计算一次 [116]；agent 轨迹的多轮之间也有大段共享前缀，树状合并的收益更大。

三层 mask

打包后有三样东西必须同时正确：注意力 mask 保证样本之间不互相可见（1.4 节）；loss mask 决定哪些 token 计入损失，用户输入、工具返回与错误片段都不计（1.4、8.6 节）；位置编号让每个样本从 0 开始重新编号。

错一个，就会出现跨样本泄漏或错误的监督，而且训练曲线往往看不出来。打包后要抽样解码检查：还原每个样本的边界、可见范围与计入损失的 token，与原始数据逐一比对。

14.11 版本、血缘与数据卡

前面每一节都在让系统更快，这一节让系统可追溯。数据决策只有在可追溯时才能被复盘。

血缘：从训练样本回到原始文档

数据集版本的定义是

$$\text{version}(D_{\text{out}}) = \text{hash}\big(\text{version}(D_{\text{in}}),\ \text{code@commit},\ \text{config}\big)$$

其中 D_{in} 是输入数据，code@commit 是处理代码的版本，config 是这一步的参数。每一步处理都记录这三项，任何一条训练样本都可以沿着版本链回溯：训练 shard → 打包 → 配比 → 去重 → 过滤 → 抽取 → 原始 URL 或文件。DVC、LakeFS、Iceberg 等工具，可以为大规模数据提供这种版本管理。

实验与数据的绑定

每个训练实验都应记录：各数据源的版本 hash，每个阶段的 π_b 与重复次数，tokenizer 版本，加载器的种子与起始步，评测集版本与所用的去污染方法。缺一项，消融结果就无法复现，也无法与其他实验比较。RL 实验还要加上环境镜像的版本与验证器的版本：10.7 节提到的公开 / 隐藏验证器，一旦悄悄更新，奖励曲线就不可比了。

数据卡

每个数据集附一张数据卡，记录来源与时间截止，处理步骤与各步的保留率，规模（三种 token 口径，见 1.6 节），已知问题，去污染范围，以及适用的训练阶段。数据卡不只是给外部看的文档，也是团队内部判断“这份数据能不能用于某个实验”的依据。MAI 按记忆率为每个数据源设 epoch 上限（4.4 节）[21]，这类上限就应该写进数据卡。

14.12 质量监控与数据飞轮

血缘让问题可以回溯，监控让问题能被及时发现，飞轮则把发现的问题变成下一轮数据。

每个阶段的统计面板

在处理管线的每一步之后，记录文档数、token 数与保留率，各类分数的分布，主题、语言与来源的分布，以及抽样示例。Data-Juicer 2.0 在每个算子之后跟踪统计量与语义标签，展示一串算子怎样逐步改变数据 [155]。

一条规则的保留率突然从 85% 变成 60%，几乎一定是上游数据或代码出了问题，而不是世界变了。

分布漂移检测

新快照、新数据源入库时，与已有数据比较 token 分布、主题分布与质量分分布；再训练一个分类器区分新旧数据，分类器越容易区分，漂移越大。对漂移明显的切片做抽样人工检查。

3.6 节的 AI 文本研究给了一个必须监控的漂移：到 2026 年 8 月，经过质量过滤的 Common Crawl token 中约 31% 被检测为 AI 生成，且比例逐个快照上升 [23]。这个比例应该作为每个快照的固定指标，验证集也要按人类与 AI 文本分开报告（15.2 节）。

数据飞轮

前面几章讨论了几种数据飞轮：UI-TARS-2 让 RL 模型生成轨迹，验证通过的进 SFT，失败的进继续预训练 [127]；Qwen-Image-2.0 把评测、bad case 与用户反馈归因后，分流到 RL、数据补偿与 prompt 工程三条通道 [53]；Z-Image 用模型诊断长尾短板，再定向检索与标注 [52]。

从系统角度看，飞轮需要三个组件：信号采集，即评测、线上日志与用户反馈，需要脱敏与授权；归因，把失败案例归到数据、训练或推理的具体环节；以及数据补偿管道，即向量检索、标注、质检与入库，能在几天内完成一轮。飞轮转得多快，最终取决于前面 11 节的系统有多快。

本章思考与进阶练习

14.1 解码与推理的配比

题目 一个视频 caption 管线中，CPU 解码与切分一个片段需要 2 核·秒，GPU 生成一条 caption 需要 0.08 GPU·秒（批处理后的均摊值）。要让 1000 张 GPU 满载，需要多少个 CPU 核？

解题思路 1000 张 GPU 每秒消费 $1000/0.08 = 12{,}500$ 个片段，CPU 每秒需要产出同样多的片段，即 $12{,}500 \times 2 = 25{,}000$ 核。加上 I/O 与调度余量，实际约 3 万核，CPU 与 GPU 之比约为 30:1。视频管线的 CPU 往往比 GPU 更紧张，与 600TB 案例中“解码饥饿”的教训一致。

14.2 去重的 shuffle 成本

题目 用 14 个 band 做 LSH，对 100 亿篇文档做模糊去重。每个 band 哈希值 8 字节，文档 ID 8 字节。shuffle 阶段要传输多少数据？怎样降低？

解题思路 每篇文档每个 band 产生一条（band 哈希, 文档 ID）记录，16 字节，总量为 $10^{10} \times 14 \times 16 \approx 2.24$ TB。降低的方法有三：按 band 分批处理，每次只 shuffle 一个 band，峰值内存降到 1/14；先

按快照分片去重（4.2 节），减少全局去重的数据量；先做精确哈希去重，Olmo 3 中这一步就去掉了约三分之二的文档。

14.3 进阶：沙箱的容量规划

题目 你计划做 agent RL，每步 512 个 prompt、每个 prompt 16 条 rollout，平均每条 rollout 20 分钟，训练一步 5 分钟。沙箱冷启动 90 秒。需要多少个并发沙箱？按需加载把冷启动降到 10 秒后呢？

解题思路 每步需要 $512 \times 16 = 8{,}192$ 条 rollout。要让训练不等待，每 5 分钟必须完成 8,192 条，即每秒约 27.3 条。每个沙箱的占用时间为运行 20 分钟加冷启动：90 秒冷启动时约 1,290 秒，并发数约 $27.3 \times 1{,}290 \approx 3.5$ 万；10 秒冷启动时约 1,210 秒，约 3.3 万。对 20 分钟的任务，冷启动只省了约 6% 的并发；但对 30 秒的工具调用任务，90 秒冷启动让每个沙箱的占用时间从 30 秒变成 120 秒，并发数是无冷启动时的 4 倍。按需加载的价值，主要在短任务与高创建速率上，这正是 DSec 每秒 5,000 次创建的场景。还要算上陈旧度：20 分钟的 rollout 跨越约 4 个版本，η 必须不小于 4。

第十五章　数据实验方法与评价

数据工作最常见的失败，往往不是方法错了，而是结论靠不住：代理模型上的排名到了大模型上反转，一次种子噪声被当成了收益，换一种评测方式排名就翻转，“3 倍数据效率”换个口径只剩 1.2 倍。

本章是全书的方法论收口，依次讨论：怎样用小实验预测大训练，怎样选评测、估计噪声，怎样设计消融，怎样定义数据效率，后训练数据的实验有什么特殊之处，以及怎样把模型行为追溯回数据。

15.1　代理模型实验

全书几乎每一个数据决策，都是先在小模型上做出来的。这一节回答的是：小模型的结论，什么时候可以相信。

代理实验的四种尺度

业界的代理实验大致分四种尺度，越往下越接近真实训练，成本也越高：

实验	规模	回答的问题	出处
代理模型群	30M 参数 × 3B token，几十到几百个	配比怎样设	Olmo 3、Olmix [13, 68]
单尺度排名	150M–1B，每个配方一个	哪份数据更好	DataDecide [167]
Microanneal	5B 候选 + 5B 基线，退火	这份数据值不值得加入	Olmo 3 [13]
集成测试	完整的 100B 中期训练 + SFT 后评测	整个配方是否成立	Olmo 3 [13]

合理的流程是逐级漏斗：大量便宜的实验做初筛，少数候选进入昂贵的验证。

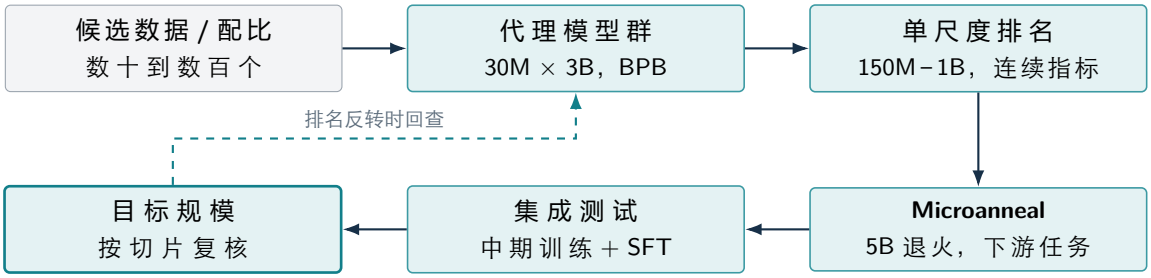


图 15.1　代理实验漏斗。读图：每往右一级，候选数减少一个量级，单个实验的成本增加一到两个量级；指标也从 BPB 逐步换成下游任务与 SFT 后评测。虚线表示目标规模上出现反转时，要回到代理阶段检查重复次数与多样性，而不是只换一个配比重跑。[13, 21, 68, 167]

DataDecide：小实验能预测多少

AI2 的 DataDecide 用 25 种预训练数据配方 × 14 种模型规模（4M–1B）系统地测量了“小规模结论能否外推”，最大的几个规模各跑 3 个种子 [167]。它有四条发现。第一，单尺度排名是强基线：在 150M

规模上比较两份数据，约 80% 的成对比较能正确预测 1B 规模的胜者。第二，规模律方法没有更好：测试的 8 种规模律外推方法，没有一种在“计算量–决策准确率”上优于单尺度排名。第三，连续指标更可预测：用基于似然的连续指标代替离散准确率，MMLU、ARC、HellaSwag、MBPP、HumanEval 在 1B 规模上的可预测性超过 80%，只需约 0.01% 的计算。第四，任务之间差异大：MMLU、ARC Easy 便宜就能预测，HellaSwag 需要多得多的计算，SocialIQA 在大多数规模上都难以预测；决定可预测性的，是种子间方差小、配方之间差异大。

它的局限是只覆盖了一种 token 与参数之比（约 5 倍 Chinchilla），以及选择题形式的评测。

代理实验什么时候会失效

代理实验的失效有四类典型原因。重复次数不同：代理只训 3B token，看不到小数据源被重复多次的损害，对策是在配比优化时施加按目标预算计算的重复上限（第五章）。能力的涌现阈值：小模型在某些任务上全是随机水平，对策是用连续指标，或换成该能力的“前置”任务。模型容量：大模型能吸收更多长尾知识，小模型更受益于高密度数据，对策是在 2–3 个规模上观察趋势，而不是只看一个点。训练阶段：某些数据的价值只在退火或 SFT 之后才体现，对策是用 microanneal 与集成测试补充（第七章）。

MAI-Thinking-1 给了一个产业级的反例：小规模上偏重 STEM 的配比胜过偏重代码的配比，放大到 23B 激活参数后结论反转 [21]。作者的解释是 STEM 来源多样性不足、模糊重复较多，大模型很快把它们“吃完”，开始受困于重复。这正是第一类与第三类原因的叠加：代理规模上，重复与容量两个变量都没有被暴露出来。

6.7 节也有一个方向相同的例子：合成数据的优势随模型规模增大而缩小，纯改写数据在大模型上落后于原始网页 [73]。只在小模型上做实验，会系统性地高估低多样性数据的价值。

15.2 评测选择与噪声

代理实验能不能用，一半取决于规模，另一半取决于用什么指标去量。同一组实验换一种评测方式，结论就可能翻转。

评测方式会翻转排名

daVinci-LLM 给过一个例子 [41]：在 MMLU 上，按似然评分，OLMo-2-7B 与 Qwen-2.5-3B 基本持平；按生成式（CoT）评分，OLMo-2-7B 领先 2.90 分。两种评测方式之间有 3.10 分的摆动，足以让排名反转。1.3 节 SuperBPE 的例子也是如此：两个 tokenizer 的 BPB 几乎一样，下游平均却相差 4 分 [11]。所以同一组数据实验，至少要用连续指标与生成式指标两种方式交叉检查。

指标的选择

四类常用指标各有适用的阶段：

指标	优点	缺点	适合
BPB / 困惑度	连续、低噪声、跨 tokenizer 可比	与下游能力不一定一致	配比回归、早期实验
选择题似然	连续，小模型上也有信号	只适用于选择题	代理模型
准确率	直观	离散、噪声大，小模型上接近随机	大模型的最终评测

指标	优点	缺点	适合
生成式评测	贴近真实使用	噪声最大、最贵	集成测试、SFT 之后

Olmo 3 的配比回归以 BPB 为目标 [13]，DataDecide 推荐用似然类连续指标做小规模决策 [167]，两者逻辑一致：小实验用低噪声指标，大实验用贴近使用的指标。

验证集也要分层

指标选对了，验证集本身也可能掩盖问题。3.6 节的 AI 文本研究中，在出现损害的实验里，有 95.5% 的损害被混合验证集掩盖了 [23]：AI 文本上的损失下降，抵消了人类文本上的损失上升，混合后的总损失看起来没有变化。

这个现象并不限于 AI 文本。验证集里只要混有两种分布，而数据改动对它们的影响方向相反，平均数就会把损害藏起来。所以验证集应按来源分层报告：人类文本与 AI 文本分开，各语言分开，各领域分开。15.1 节的排名反转也说明了分层的必要：只看总损失，很难发现是哪个领域先被“吃完”。

种子噪声与所需的种子数

一个数据改动是否有效，取决于效应量与噪声之比：

$$\mathrm{SNR} = \frac{|\mu_A - \mu_B|}{\sigma_{\text{seed}}}$$

其中 μ_A, μ_B 是两组的均值，σ_{seed} 是种子间的标准差。要以显著性水平 α、检验力 $1-\beta$ 检测到差异 Δ，每组需要的种子数约为

$$n \approx 2\left(\frac{(z_{1-\alpha/2} + z_{1-\beta})\,\sigma_{\text{seed}}}{\Delta}\right)^2$$

数值例子（教学假设）：种子间标准差 0.8 分，想检测 1.0 分的差异，取 $\alpha = 0.05$、检验力 80%，两个 z 值为 1.96 与 0.84，得

$$n \approx 2 \times \left(\frac{2.8 \times 0.8}{1.0}\right)^2 \approx 10$$

每组约 10 个种子；若只想检测 2 分的差异，约 3 个就够。这意味着 1 分以内的数据改动，在单种子实验中基本不可信，要么跑多种子，要么换用方差更小的指标。DataDecide 得出的也是这个结论：低方差、大差距的基准，才能做出可靠的决策 [167]。

噪声底

在大规模实验开始前，先测“噪声底”：用同一份数据、不同的种子跑几次，记录每个指标的波动范围。之后任何改动的效果，都先与噪声底比较，小于噪声底的“提升”不应该被报告。

噪声底也要随规模重测。小模型在生成式评测上的种子方差，可能比大模型大好几倍，用 1B 的噪声底去判断 7B 的实验，会把真实的差异当成噪声。

15.3 消融设计

指标与噪声解决的是“量得准不准”，消融设计解决的是“量的是不是那个东西”。数据实验最难控制的，是那些被改动连带改变的变量。

控制变量

一次数据消融，以下几项要保持相同，或者明确报告：训练 token 数，唯一 token 数与重复次数，计算量（模型规模 × token 数），tokenizer（不同时改用 BPB 比较，见 1.3 节），学习率调度（WSD 与余弦下的结论可能不同），序列长度与打包方式，以及评测集与评测方式（且已去污染）。

最常被忽视的是重复次数。6.3 节的 megadocs 研究诚实地报告了：部分收益来自“允许训练更久”，也就是真实数据多训了几个 epoch、合成数据的比例更高；控制这些因素后，收益就缩小了 [77]。

交互项：析因设计

数据改动之间往往有交互。质量过滤与改写的收益可能互相替代，因为改写本来就提高了质量；中期训练的推理数据与 SFT 的推理数据则可能互补（6.6 节的 front-loading）。检测交互要用析因设计，以 2×2 为例：

	不改写	改写
不过滤	y_{00}	y_{01}
过滤	y_{10}	y_{11}

交互项为

$$I = (y_{11} - y_{10}) - (y_{01} - y_{00})$$

即“过滤之后改写的收益”减去“不过滤时改写的收益”。I 显著为负，说明两个改动的收益不能简单相加：先上线一个，另一个的边际收益会缩水。2×2 设计需要 4 组实验，每组若跑 3 个种子就是 12 次训练，所以析因设计通常只在代理规模上做。

实验记录卡

每次数据实验都应留下一张记录卡，至少包括下表中的内容：

栏目	示例
假设	加入 PDF 数据会提升 STEM 任务
改动	PDF 占比从 0 提到 10%，同时下调网页比例
固定项	1B 模型、100B token、tokenizer v3、WSD
数据版本	各数据源的版本 hash，以及唯一 token 数与重复次数
评测	BPB（STEM 子集）+ 选择题似然 + 生成式评测，已去污染，验证集按来源分层
种子与噪声底	种子数，各指标的种子间标准差

栏目	示例
结果与切片分析	均值 ± 标准差，哪些主题变好、哪些变差
结论与下一步	—

切片分析一栏最重要。Olmo 3 的配比优化让 54 个任务中的 13 个变差（5.4 节）[13]，只报平均数，就会掩盖这些代价。

15.4 数据效率的度量

消融的结论最后常被压缩成一个数：“数据效率提高了 N 倍”。这个数的口径，比它的大小更重要。

定义

数据效率倍数定义为

$$\mathrm{Eff} = \frac{D_{\mathrm{base}}(L^*)}{D_{\mathrm{new}}(L^*)}$$

其中 L^* 是目标指标，$D_{\mathrm{base}}(L^*)$ 与 $D_{\mathrm{new}}(L^*)$ 分别是基线与新方法达到 L^* 所需的 token 数。

各项研究报告的倍数

几项研究报告的倍数与口径如下：

研究	倍数	目标与口径
BeyondWeb [72]	7.7 倍（相对 RedPajama）、2.7 倍（相对 Nemotron-Synth）	8B 模型，达到 RedPajama 训练 180B token 的 14 项平均准确率
Olmix [68]	3.05 倍	达到自然分布配比的最终表现
DoReMi [66]	2.6 倍	达到基线的下游准确率所需的步数
Megadocs [77]	1.8 倍（latent thoughts，32 次生成）	300M 模型，200M 真实 token 的数据受限设置，按验证损失
Data Mixing Laws [67]	相当于多训练 48% 的步数	1B 模型，100B token

同一个 BeyondWeb，换一个基线，倍数就从 7.7 变成 2.7。

倍数之间为什么不能直接比较

这些倍数有四处口径差异：目标不同，验证损失、单个基准与多个基准的平均，结论可能不同；基线不同，相对原始网页的倍数远大于相对另一种合成方法的倍数；规模不同，数据受限设置（megadocs）与数据充足设置的含义完全不同；是否计入数据成本，合成数据的生成成本、过滤的计算成本，通常没有计入。所以报告数据效率时，要同时写明目标指标、基线、模型规模、训练 token 数，以及是否计入数据处理成本。

计入数据成本的总效率

把数据成本算进去，总效率为

$$\mathrm{Eff}_{\mathrm{total}} = \frac{C_{\mathrm{train}}(D_{\mathrm{base}})}{C_{\mathrm{train}}(D_{\mathrm{new}}) + C_{\mathrm{data}}}$$

其中 C_{train} 是训练计算量，C_{data} 是生成或处理数据的计算量。

数值例子（教学假设）：基线训练需要 10^{23} FLOPs；用合成数据后，训练只需 0.5×10^{23} FLOPs，即 2 倍数据效率；合成数据的生成用了 0.2×10^{23} FLOPs。总效率为 $1/(0.5 + 0.2) \approx 1.43$ 倍。

合成数据可以被多次训练复用，生成成本应该在多个实验与多个模型之间分摊，只训练一次时总效率最低。3.6 节 AI 文本研究的“1.6 倍计算”[23]，是同一个公式的反方向：数据变差，同样可以折算成额外的训练计算。

15.5 后训练数据的实验

预训练的数据实验以 token 与损失为单位；后训练的数据量小、指标噪声大，数据还会随模型变化，前面几节的方法要做三处调整。

损失归一化会改变“谁在主导”

SFT 数据的消融，结论会被损失的归一化方式悄悄改变。8.8 节的算例中，一条 8K token 的长样本与 40 条 200 token 的短样本混在一起：按 token 平均时长样本占 50% 的梯度，按样本平均时只占约 2.4%。比较两份 SFT 数据时，归一化方式必须固定并写进记录卡，否则“加入长推理数据有效”的结论，可能只是归一化换了。

RL 数据随策略漂移

RL 数据的筛选依赖当前模型。10.2 节的通过率筛选只保留通过率不是 0 也不是 1 的题，信息量为 $P_{\mathrm{info}}(p) = 1 - p^G - (1-p)^G$，其中 p 是题目的通过率，G 是每题的 rollout 数。同一份题库，换一个起点模型，留下来的题就不同。

所以 RL 数据的消融要固定三样东西：起点 checkpoint、筛选时所用的模型版本，以及验证器的版本。比较两份 RL 题库时，应在同一个起点上各自重新筛选，再比较最终效果；直接拿一个模型筛出来的题库去训另一个模型，比较的就不只是数据了。

按计算量对齐，而不是按步数

后训练的方法之间，每一步的成本差别很大。10.12 节的 Thinking Machines 实验中，用 OPD 把数学推理模型训练到 AIME'24 70%，用了 1,800 GPU 时，同等效果的 RL 基线用了 17,920 GPU 时 [130]。按步数比较，两者看起来差不多；按 GPU 时比较，相差约 10 倍。

因此后训练的数据或方法比较，应画“效果–GPU 时”曲线，并把 rollout、打分（包括 GRM 评审）与教师前向的成本一起算进去。异步 RL 中还要记录陈旧度（14.6 节），否则两次实验的数据分布并不相同。

15.6 数据归因

实验回答的是“这份数据有没有用”，归因回答的是反方向的问题：“模型的这个行为，是从哪些数据学来的”。

从输出回溯到训练文档

三类方法覆盖了不同的规模与问题。n-gram 精确匹配在后缀数组索引中检索输出的长片段，可以覆盖万亿 token 级的数据（infini-gram、OLMoTrace）[61, 64]，回答“这段话是从哪里学来的”。embedding 检索找出与输出语义相近的训练样本，适合十亿级的数据，回答“哪些数据与这个行为相关”。影响函数估计去掉某个样本后，模型在测试样本上的损失变化，计算昂贵、需要近似，回答“哪些样本对这个预测贡献最大”。

影响函数及其近似

影响函数估计的是：如果把训练样本 z 的权重提高一点，测试样本 z_{test} 的损失会怎样变化 [168]：

$$\mathcal{I}(z, z_{\text{test}}) = -\nabla_\theta \ell(z_{\text{test}})^\top H^{-1} \nabla_\theta \ell(z)$$

其中 ℓ 是损失，θ 是模型参数，H 是训练损失的海森矩阵。大模型上无法精确计算 H^{-1}，常用三种近似：TRAK 用随机投影把梯度降到低维，并在多个模型上做集成 [169]；LESS 用 LoRA 梯度、随机投影与 Adam 预条件来选择指令数据（3.8 节）[50]；最粗的做法是忽略 H^{-1}，退化为梯度内积，也就是梯度相似度。近似越粗，结果越接近“语义相似度”，越难区分相关与因果。

归因的用途与局限

归因在四个场景中有用：调试，在模型复读模板、泄漏隐私时找到源头数据（14.11 节）；去污染审计，检查模型对评测题的回答是否来自训练集里的原题；数据估值，估计每个数据源对目标能力的贡献，指导配比与采购；以及数据选择，即 LESS 类方法。

它的局限同样明确。大模型的能力往往来自大量样本的共同作用，单个样本的影响很小，归因结果噪声大；精确匹配只能找到逐字学来的内容，找不到“理解后泛化”的来源。归因给出的是相关性证据，要确认因果，仍需要做去掉或加入数据的对照训练，也就是回到 15.3 节的消融。

本章思考与进阶练习

15.1 效应量与种子数

题目 你的基准在 1B 规模上的种子间标准差为 1.2 分。你想检测 1.5 分的提升（$\alpha = 0.05$，检验力 80%），每组需要几个种子？如果改用标准差只有 0.3 个单位的连续指标，又需要几个？

解题思路 准确率需要 $n \approx 2 \times (2.8 \times 1.2/1.5)^2 \approx 10$ 个种子。连续指标要先把 1.5 分的效应换算到该指标的单位：若对应 0.4 个单位，$n \approx 2 \times (2.8 \times 0.3/0.4)^2 \approx 8.8$，约 9 个；若对应 0.6 个单位，只需约 4 个。关键不是指标本身的方差，而是“效应 / 方差”之比；选指标时，应在已知有差异的数据对上实测这个比值。

15.2 被平均数掩盖的损害

题目 你在网页数据中加入了一批新来源，混合验证集上的损失几乎不变。之后发现这批来源中约一半是 AI 生成文本。你应该怎样重新评估这次改动?

解题思路 把验证集拆成人类文本与 AI 文本两部分，分别比较改动前后的损失：AI 文本上的损失下降，可能正好抵消了人类文本上的上升 [23]。再按语言、领域切片，确认损害集中在哪里。若人类文本上的损失确实上升，按 3.6 节的建议，人类数据不足时宁可重复人类文本，也不要加入来源不明的 AI 文本。最后把“人类文本验证损失”加入以后每次实验的固定指标。

15.3 进阶：评测方式的选择偏差

题目 你的数据改动在似然评分下提升了 2 分，在生成式评分下下降了 1 分。可能的原因是什么? 应该相信哪一个?

解题思路 可能的原因有三：新数据提高了模型对正确选项的“熟悉度”（例如选择题格式的合成数据），但没有提高推理能力；新数据改变了生成风格，更啰嗦或格式不稳定，生成式评分的答案抽取失败率上升；两种评测的题目子集不同。诊断时检查生成式评分中的格式错误率，在 SFT 之后再比较，看差异是否消失，并按题型切片。最终以贴近真实使用的生成式评分为准，同时报告两者并说明原因。6.6 节中“SFT 风格数据带来的 few-shot 提升，部分来自格式熟悉度”，正是这种情况。

附录 A　术语与接口速查

术语	含义	首次出现
BPE / Unigram	两种子词分词算法：按频次贪心合并 / 按概率选择切分	1.1
BPB	bits-per-byte，每字节的比特数，跨 tokenizer 可比的损失	1.3
Fertility / ratio	同一内容在某语言下与英文下的 token 数之比	1.2
样本级 mask	打包后只允许文档内注意力的块对角 mask	1.4
Loss mask	指定哪些 token 计入损失	1.4
原始 / 唯一 / 训练 token	去重前的量、全局去重后的量、计入重复次数后的量	1.6
WARC / WAT / WET	Common Crawl 的原始响应、元数据与链接图、官方纯文本	2.1
L0–L9	按模型介入深度划分的数据处理层级	3.1
质量分类器	fastText 或 embedding 回归头，把“像好文档”学成分数	3.3
MinHash / LSH	用签名估计 Jaccard 相似度 / 分带找候选重复	4.2
语义去重	在 embedding 空间中识别改写、翻译后的重复	4.3
去污染	从训练数据中排除评测题及其改写	4.5
π_b / ρ_b	数据桶的配比 / 重复次数	5.1
温度采样	按 $\lvert D_b\rvert^{1/\tau}$ 拉平各桶的比例	5.1
WSD	warmup-stable-decay 学习率调度	5.2
代理模型群	用大量小模型回归“配比 $\rightarrow$ 指标”的关系	5.4
D'	重复数据的等效唯一数据量	5.8
种子锚定改写	以真实文档为种子、受控生成的合成数据	6.1
megadoc	把同源的多个改写版本拼成一篇长文	6.3
front-loading	把推理等 SFT 风格数据前移到预训练	6.6
中期训练 / 退火	学习率衰减段的定向训练 / 用小规模退火评估数据价值	7.1
FIM	fill-in-the-middle，前缀–后缀–中段的代码补全格式	7.4
Agentic CPT	在中期训练中前移 agent 行为	7.5
拒绝采样	采样多个回答，只保留通过验证的	8.3
Bradley–Terry	把偏好写成两个奖励之差的 sigmoid	9.1
DPO	直接在策略上做偏好优化，用对数概率比充当隐式奖励	9.1
GRM	生成式奖励模型，先写理由再给判断	9.3
Rubric	把“好回答”拆成可逐条检查的标准	9.4
Reward hacking	代理奖励上升、真实质量下降的现象	9.6
验证器 $\mathcal{V}$ / 环境 $\mathcal{E}$	把轨迹映射成奖励的程序 / 响应动作的执行环境	10.1
GRPO / P_{info}	组内归一化优势的 RL 算法 / 一道题产生非零梯度的概率	10.2
FAIL_TO_PASS / PASS_TO_PASS	SWE 任务中修复后应通过 / 修复前后都应通过的测试	10.5
难解易验	验证远比求解简单的任务构造原则	10.7

术语	含义	首次出现
OPD / MOPD	on-policy 蒸馏 / 多教师 on-policy 蒸馏	10.12
交错数据	保留图文相对位置的多模态文档	11.3
latent token	视频生成模型在 VAE latent 上计的 token	12.4
伪动作 / IDM	从人类视频重建的动作标签 / 逆动力学模型	13.4
陈旧度 η	异步 RL 中训练数据落后当前模型的版本数	14.6
血缘	从训练样本回溯到原始文档的版本链	14.11
噪声底	同一数据、不同种子下各指标的波动范围	15.2
数据效率倍数	基线与新方法达到同一指标所需 token 数之比	15.4

一手资料与延伸阅读

[1] Neural Machine Translation of Rare Words with Subword Units (BPE). https://arxiv.org/abs/1508.07909

[2] Tokenization Counts: the Impact of Tokenization on Arithmetic in Frontier LLMs. https://arxiv.org/abs/2402.14903

[3] DeepSeek-V3 Technical Report. https://arxiv.org/abs/2412.19437

[4] Fishing for Magikarp: Automatically Detecting Under-trained Tokens in Large Language Models. https://arxiv.org/abs/2405.05417

[5] Scaling Laws with Vocabulary: Larger Models Deserve Larger Vocabularies. https://arxiv.org/abs/2407.13623

[6] The Llama 3 Herd of Models. https://arxiv.org/abs/2407.21783

[7] DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence. https://arxiv.org/abs/2606.19348

[8] Qwen3.5: Towards Native Multimodal Agents（Qwen 官方博客）。https://www.alibabacloud.com/blog/qwen3-5-towards-native-multimodal-agents_602894

[9] Gemma 4 Technical Report. https://arxiv.org/abs/2607.02770

[10] Language Model Tokenizers Introduce Unfairness Between Languages. https://arxiv.org/abs/2305.15425

[11] SuperBPE: Space Travel for Language Models. https://arxiv.org/abs/2503.13423

[12] Byte Latent Transformer: Patches Scale Better Than Tokens. https://arxiv.org/abs/2412.09871

[13] Olmo 3. https://arxiv.org/abs/2512.13961

[14] Kimi K3 Technical Report. https://arxiv.org/abs/2607.24653

[15] DeepSeek-V4（Hugging Face 博客解读）。https://huggingface.co/blog/deepseekv4

[16] GLM-5: from Vibe Coding to Agentic Engineering. https://arxiv.org/abs/2602.15763

[17] Nemotron 3 Ultra: Open, Efficient Mixture-of-Experts Hybrid Mamba-Transformer Model for Agentic Reasoning. https://arxiv.org/abs/2606.15007

[18] Kimi K2.5: Visual Agentic Intelligence. https://arxiv.org/abs/2602.02276

[19] Qwen3-VL Technical Report. https://arxiv.org/abs/2511.21631

[20] Nemotron-CC: Transforming Common Crawl into a Refined Long-Horizon Pretraining Dataset. https://arxiv.org/abs/2412.02595

[21] MAI-Thinking-1 Technical Report. https://microsoft.ai/pdf/mai-thinking-1.pdf

[22] Will We Run Out of Data? Limits of LLM Scaling Based on Human-Generated Data. https://arxiv.org/abs/2211.04325

[23] How Much Is an AI Token Worth? https://arxiv.org/abs/2609.40295

[24] The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale（含 FineWeb-Edu）. https://arxiv.org/abs/2406.17557

[25] Nemotron-CC-Math: A 133 Billion-Token-Scale High Quality Math Pretraining Dataset. https://arxiv.org/abs/2508.15096

[26] DataComp-LM: In Search of the Next Generation of Training Sets for Language Models (DCLM). https://arxiv.org/abs/2406.11794

[27] FinePDFs（Hugging Face 数据集与说明）. https://huggingface.co/datasets/HuggingFaceFW/finepdfs

[28] Qwen3 Technical Report. https://arxiv.org/abs/2505.09388

[29] olmOCR: Unlocking Trillions of Tokens in PDFs with Vision Language Models. https://arxiv.org/abs/2502.18443

[30] MinerU2.5: A Decoupled Vision-Language Model for Efficient High-Resolution Document Parsing. https://arxiv.org/abs/2509.22186

[31] StarCoder 2 and The Stack v2: The Next Generation. https://arxiv.org/abs/2402.19173

[32] Seed-Coder: Let the Code Model Curate Data for Itself. https://arxiv.org/abs/2506.03524

[33] SmolLM2: When Smol Goes Big（含 Stack-Edu）. https://arxiv.org/abs/2502.02737

[34] DeepSeek-Coder: When the Large Language Model Meets Programming. https://arxiv.org/abs/2401.14196

[35] How Frontier Labs Train LLMs（个人博客，二手整理；仅用于 Grok 4.5 与 MiniMax 预训练不用合成数据两条信息）。https://jxzhangjhu.github.io/blog/2026/how-frontier-labs-train-llms/

[36] FineWeb2: One Pipeline to Scale Them All. https://arxiv.org/abs/2506.20920

[37] LAION-BVD. https://arxiv.org/abs/2608.24845

[38] HunyuanVideo 1.5 Technical Report. https://arxiv.org/abs/2511.18870

[39] MiMo-Audio: Audio Language Models are Few-Shot Learners. https://arxiv.org/abs/2512.23808

[40] Kandinsky 6.0 Video: Foundation Models for Synchronized Video and Audio Generation. https://arxiv.org/abs/2610.05608

[41] daVinci-LLM: Towards the Science of Pretraining. https://arxiv.org/abs/2603.27164

[42] Scaling Language Models: Methods, Analysis & Insights from Training Gopher. https://arxiv.org/abs/2112.11446

[43] Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer (C4). https://arxiv.org/abs/1910.10683

[44] MADLAD-400: A Multilingual And Document-Level Large Audited Dataset. https://arxiv.org/abs/2309.04662

[45] QuRating: Selecting High-Quality Data for Training Language Models. https://arxiv.org/abs/2402.09739

[46] Organize the Web: Constructing Domains Enhances Pre-Training Data Curation (WebOrganizer). https://arxiv.org/abs/2502.10341

[47] AI can Autonomously Evolve Pretraining Data Curation (DataEvolve / Darwin-CC). https://arxiv.org/abs/2603.14420

[48] CCNet: Extracting High Quality Monolingual Datasets from Web Crawl Data. https://arxiv.org/abs/1911.00359

[49] Data Selection for Language Models via Importance Resampling (DSIR). https://arxiv.org/abs/2302.03169

[50] LESS: Selecting Influential Data for Targeted Instruction Tuning. https://arxiv.org/abs/2402.04333

[51] Rho-1: Not All Tokens Are What You Need. https://arxiv.org/abs/2404.07965

[52] Z-Image: An Efficient Image Generation Foundation Model with Single-Stream Diffusion Transformer. https://arxiv.org/abs/2511.22699

[53] Qwen-Image-2.0 Technical Report. https://arxiv.org/abs/2605.10730

[54] Deduplicating Training Data Makes Language Models Better. https://arxiv.org/abs/2107.06499

[55] SemDeDup: Data-efficient Learning at Web-scale through Semantic Deduplication. https://arxiv.org/abs/2303.09540

[56] D4: Improving LLM Pretraining via Document De-Duplication and Diversification. https://arxiv.org/abs/2308.12284

[57] NeMo Curator：视频整理管线文档. https://docs.nvidia.com/nemo/curator/latest/curate-video.md

[58] Scaling Laws and Interpretability of Learning from Repeated Data. https://arxiv.org/abs/2205.10487

[59] Quantifying Memorization Across Neural Language Models. https://arxiv.org/abs/2202.07646

[60] Kimi K2: Open Agentic Intelligence. https://arxiv.org/abs/2507.20534

[61] Infini-gram: Scaling Unbounded n-gram Language Models to a Trillion Tokens. https://arxiv.org/abs/2401.17377

[62] Rethinking Benchmark and Contamination for Language Models with Rephrased Samples. https://arxiv.org/abs/2311.04850

[63] Detecting Pretraining Data from Large Language Models (Min-K% Prob). https://arxiv.org/abs/2310.16789

[64] OLMoTrace: Tracing Language Model Outputs Back to Trillions of Training Tokens. https://arxiv.org/abs/2504.07096

[65] RegMix: Data Mixture as Regression for Language Model Pre-training. https://arxiv.org/abs/2407.01492

[66] DoReMi: Optimizing Data Mixtures Speeds Up Language Model Pretraining. https://arxiv.org/abs/2305.10429

[67] Data Mixing Laws: Optimizing Data Mixtures by Predicting Language Modeling Performance. https://arxiv.org/abs/2403.16952

[68] Olmix: A Framework for Data Mixing Throughout LM Development. https://arxiv.org/abs/2602.12237

[69] Beyond Chinchilla-Optimal: Accounting for Inference in Language Model Scaling Laws. https://arxiv.org/abs/2401.00448

[70] Scaling Data-Constrained Language Models. https://arxiv.org/abs/2305.16264

[71] How Can We Synthesize High-Quality Pretraining Data? A Systematic Study of Prompt Design, Generator Model, and Source Data (FinePhrase). https://arxiv.org/abs/2604.13977

[72] BeyondWeb: Lessons from Scaling Synthetic Data for Trillion-scale Pretraining. https://arxiv.org/abs/2508.10975

[73] Demystifying Synthetic Data in LLM Pre-training: A Systematic Study of Scaling Laws, Benefits, and Pitfalls. https://arxiv.org/abs/2510.01631

[74] Rephrasing the Web: A Recipe for Compute and Data-Efficient Language Modeling (WRAP). https://arxiv.org/abs/2401.16380

[75] Recycling the Web: A Method to Enhance Pre-training Data Quality and Quantity for Language Models (REWIRE). https://arxiv.org/abs/2506.04689

[76] Rewriting Pre-Training Data Boosts LLM Performance in Math and Code (SwallowMath / SwallowCode). https://arxiv.org/abs/2505.02881

[77] Data-efficient Pre-training by Scaling Synthetic Megadocs. https://arxiv.org/abs/2603.18534

[78] Textbooks Are All You Need (phi-1). https://arxiv.org/abs/2306.11644

[79] Cosmopedia: how to create large-scale synthetic data for pre-training. https://huggingface.co/blog/cosmopedia

[80] Scaling Synthetic Data Creation with 1,000,000,000 Personas (Persona Hub). https://arxiv.org/abs/2406.20094

[81] Beyond Rephrasing: Book-Level Organization Improves Synthetic Textbook Data for Mid-Training. https://arxiv.org/abs/2607.28109

[82] Synthetic Continued Pretraining (EntiGraph). https://arxiv.org/abs/2409.07431

[83] Front-Loading Reasoning: The Synergy between Pretraining and Post-Training Data. https://arxiv.org/abs/2510.03264

[84] AI Models Collapse When Trained on Recursively Generated Data (Nature, 2024). https://www.nature.com/articles/s41586-024-07566-y

[85] Is Model Collapse Inevitable? Breaking the Curse of Recursion by Accumulating Real and Synthetic Data. https://arxiv.org/abs/2404.01413

[86] Scaling Agents via Continual Pre-training (AgentFounder). https://arxiv.org/abs/2509.13310

[87] OctoThinker: Mid-training Incentivizes Reinforcement Learning Scaling. https://arxiv.org/abs/2506.20512

[88] How to Train Long-Context Language Models (Effectively) (ProLong). https://arxiv.org/abs/2410.02660

[89] NExtLong: Toward Effective Long-Context Training without Long Documents. https://arxiv.org/abs/2501.12766

[90] EntropyLong: Effective Long-Context Training via Predictive Uncertainty. https://arxiv.org/abs/2510.02330

[91] Keye-VL-2.0 Technical Report. https://arxiv.org/abs/2606.10651

[92] DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. https://arxiv.org/abs/2501.12948

[93] LIMO: Less is More for Reasoning. https://arxiv.org/abs/2502.03387

[94] s1: Simple Test-Time Scaling. https://arxiv.org/abs/2501.19393

[95] OpenThoughts: Data Recipes for Reasoning Models. https://arxiv.org/abs/2506.04178

[96] Magpie: Alignment Data Synthesis from Scratch by Prompting Aligned LLMs with Nothing. https://arxiv.org/abs/2406.08464

[97] OpenThoughts-Agent. https://arxiv.org/abs/2606.24855

[98] HelpSteer2: Open-source Dataset for Training Top-performing Reward Models. https://arxiv.org/abs/2406.08673

[99] UltraFeedback: Boosting Language Models with Scaled AI Feedback. https://arxiv.org/abs/2310.01377

[100] Tülu 3: Pushing Frontiers in Open Language Model Post-Training. https://arxiv.org/abs/2411.15124

[101] The Delta Learning Hypothesis: Preference Tuning on Weak Data can Yield Strong Gains. https://arxiv.org/abs/2507.06187

[102] Skywork-Reward-V2: Scaling Preference Data Curation via Human-AI Synergy. https://arxiv.org/abs/2507.01352

[103] Generative Verifiers: Reward Modeling as Next-Token Prediction. https://arxiv.org/abs/2408.15240

[104] DeepSeek V4: ten teachers, one student（M. Labonne 解读）. https://maximelabonne.substack.com/p/deepseek-v4-ten-teachers-one-student

[105] Rubrics as Rewards: Reinforcement Learning Beyond Verifiable Domains. https://arxiv.org/abs/2507.17746

[106] ARES: Automated Rubric Synthesis for Scalable LLM Reinforcement Learning. https://arxiv.org/abs/2605.23454

[107] QUBRIC. https://arxiv.org/abs/2606.03968

[108] QUEST: Training Frontier Deep Research Agents with Fully Synthetic Tasks. https://arxiv.org/abs/2605.24218

[109] GLM-5V-Turbo Technical Report. https://arxiv.org/abs/2604.26752

[110] A Long Way to Go: Investigating Length Correlations in RLHF. https://arxiv.org/abs/2310.03716

[111] Agent World Model: Infinity Synthetic Environments for Agentic Reinforcement Learning. https://arxiv.org/abs/2602.10090

[112] Scaling Laws for Reward Model Overoptimization. https://arxiv.org/abs/2210.10760

[113] GLM-5.2（Z.ai 官方博客）。https://huggingface.co/blog/zai-org/glm-52-blog

[114] DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models. https://arxiv.org/abs/2512.02556

[115] MiMo-V2-Flash Technical Report. https://arxiv.org/abs/2601.02780

[116] MiniMax M2.5 发布博客（RL 环境规模、Forge 框架）. https://www.minimax.io/blog/minimax-m25

[117] Seed1.5-VL Technical Report. https://arxiv.org/abs/2505.07062

[118] Kimi k1.5: Scaling Reinforcement Learning with LLMs. https://arxiv.org/abs/2501.12599

[119] DAPO: An Open-Source LLM Reinforcement Learning System at Scale. https://arxiv.org/abs/2503.14476

[120] DeepSeekMath-V2: Towards Self-Verifiable Mathematical Reasoning. https://arxiv.org/abs/2511.22570

[121] Training Software Engineering Agents and Verifiers with SWE-Gym. https://arxiv.org/abs/2412.21139

[122] R2E-Gym: Procedural Environments and Hybrid Verifiers for Scaling Open-Weights SWE Agents. https://arxiv.org/abs/2504.07164

[123] SWE-smith: Scaling Data for Software Engineering Agents. https://arxiv.org/abs/2504.21798

[124] SWE-rebench V2: Language-Agnostic SWE Task Collection at Scale. https://arxiv.org/abs/2602.23866

[125] ScaleEnv: Scaling Environment Synthesis from Scratch for Generalist Interactive Tool-Use Agent Training. https://arxiv.org/abs/2602.06820

[126] Environments Hub（Prime Intellect 博客）。https://primeintellect.ai/blog/environments

[127] UI-TARS-2 Technical Report: Advancing GUI Agent with Multi-Turn Reinforcement Learning. https://arxiv.org/abs/2509.02544

[128] TreeCUA：树形探索的计算机使用 agent 数据合成（美团，2026）. https://arxiv.org/abs/2602.09662

[129] PivotRL: High Accuracy Agentic Post-Training at Low Compute Cost. https://arxiv.org/abs/2603.21383

[130] On-Policy Distillation（Thinking Machines Lab 博客）。https://thinkingmachines.ai/blog/on-policy-distillation

[131] Nemotron-Cascade 2. https://arxiv.org/abs/2603.19220

[132] SAO: Single-rollout Asynchronous Optimization. https://arxiv.org/abs/2607.07508

[133] 20/20 VLM（DatologyAI，数据整理对 VLM 的影响）. https://arxiv.org/abs/2605.11405

[134] FineVision: Open Data Is All You Need. https://arxiv.org/abs/2510.17269

[135] LLaVA-OneVision-1.5: Fully Open Framework for Democratized Multimodal Training. https://arxiv.org/abs/2509.23661

[136] Molmo and PixMo: Open Weights and Open Data for State-of-the-Art Vision-Language Models. https://arxiv.org/abs/2409.17146

[137] OBELICS: An Open Web-Scale Filtered Dataset of Interleaved Image-Text Documents. https://arxiv.org/abs/2306.16527

[138] MINT-1T: Scaling Open-Source Multimodal Data by 10x. https://arxiv.org/abs/2406.11271

[139] Wan: Open and Advanced Large-Scale Video Generative Models. https://arxiv.org/abs/2503.20314

[140] Seedance 1.0: Exploring the Boundaries of Video Generation Models. https://arxiv.org/abs/2506.09113

[141] Seedance 2.0. https://arxiv.org/abs/2604.14148

[142] Cosmos World Foundation Model Platform for Physical AI. https://arxiv.org/abs/2501.03575

[143] Matrix-Game 3.0. https://arxiv.org/abs/2604.08995

[144] Qwen3-Omni Technical Report. https://arxiv.org/abs/2509.17765

[145] Kimi-Audio Technical Report. https://arxiv.org/abs/2504.18425

[146] Emilia: A Large-Scale, Extensive, Multilingual, and Diverse Dataset for Speech Generation. https://arxiv.org/abs/2501.15907

[147] MOSS-TTS Technical Report. https://arxiv.org/abs/2603.18090

[148] Data Pyramid for Embodied Manipulation: A Survey. https://arxiv.org/abs/2607.24744

[149] HumanScale: Egocentric Human Video Can Outperform Real-Robot Data for Embodied Pre-training. https://arxiv.org/abs/2606.20521

[150] EgoScale: Scaling Dexterous Manipulation with Diverse Egocentric Human Data. https://arxiv.org/abs/2602.16710

[151] GR00T N1: An Open Foundation Model for Generalist Humanoid Robots. https://arxiv.org/abs/2503.14734

[152] Isaac GR00T N1.7（NVIDIA 官方仓库 README）。https://github.com/NVIDIA/Isaac-GR00T

[153] π*0.6: a VLA that Learns from Experience（Physical Intelligence 官方博客，Recap 方法）。https://www.pi.website/blog/pistar06

[154] Matrix-Game: Interactive World Foundation Model. https://arxiv.org/abs/2506.18701

[155] Data-Juicer 2.0: Cloud-Scale Adaptive Data Processing for and with Foundation Models. https://arxiv.org/abs/2501.14755

[156] Video Captioning at Scale: 600 TB in 95 Minutes with Anyscale on CoreWeave（厂商博客，自报数据）. https://wf.coreweave.com/blog/video-captioning-at-scale-600-tb-in-95-minutes-with-anyscale-on-coreweave

[157] DeepSeek Elastic Compute (DSec): A Sandbox Infrastructure for Effective Agentic Training at Scale. https://arxiv.org/abs/2609.22978

[158] WebDataset. https://github.com/webdataset/webdataset

[159] MosaicML Streaming（MDS 格式）. https://github.com/mosaicml/streaming

[160] Lance（列式多模态数据格式）. https://github.com/lancedb/lance

[161] Datatrove（Hugging Face 数据处理库）. https://github.com/huggingface/datatrove

[162] NeMo Curator（NVIDIA 数据整理库）. https://github.com/NVIDIA-NeMo/Curator

[163] Efficient Memory Management for Large Language Model Serving with PagedAttention (vLLM). https://arxiv.org/abs/2309.06180

[164] SGLang: Efficient Execution of Structured Language Model Programs. https://arxiv.org/abs/2312.07104

[165] AReaL: A Large-Scale Asynchronous Reinforcement Learning System for Language Reasoning. https://arxiv.org/abs/2505.24298

[166] OLMo: Accelerating the Science of Language Models. https://arxiv.org/abs/2402.00838

[167] DataDecide: How to Predict Best Pretraining Data with Small Experiments. https://arxiv.org/abs/2504.11393

[168] Understanding Black-box Predictions via Influence Functions. https://arxiv.org/abs/1703.04730

[169] TRAK: Attributing Model Behavior at Scale. https://arxiv.org/abs/2303.14186